

2. Toolbox: Authentication, Access Control, and Cryptography

Chapter topics:

- Authentication, capabilities, and limitations
 - The three bases of authentication: knowledge, characteristics, possessions
 - Strength of an authentication mechanism
 - Implementation of access control
 - Employing encryption
 - Symmetric and asymmetric encryption
 - Message digests
 - Signatures and certificates
-

Just as doctors have stethoscopes and carpenters have measuring tapes and squares, security professionals have tools they use frequently. Three of these security tools are authentication, access control, and cryptography. In this chapter we introduce these tools, and in later chapters we use these tools repeatedly to address a wide range of security issues.

In some sense, security hasn't changed since sentient beings began accumulating things worth protecting. A system owner establishes a security policy, formally or informally, explicitly or implicitly—perhaps as simple as “no one is allowed to take my food”—and begins taking measures to enforce that policy. The character of the threats changes as the protagonist moves from the jungle to the medieval battlefield to the modern battlefield to the Internet, as does the nature of the available protections, but their strategic essence remains largely constant: An attacker wants something a defender has, so the attacker goes after it. The defender has a number of options—fight, build a barrier or alarm system, run and hide, diminish the target's attractiveness to the attacker—and these options all have analogues in modern computer security. The specifics change, but the broad strokes remain the same.

In this chapter, we lay the foundation for computer security by studying those broad strokes. We look at a number of ubiquitous security strategies, identify the threats against which each of those strategies is effective, and give examples of representative countermeasures. Throughout the rest of this book, as we delve into the specific technical security measures used in operating systems, programming, websites and browsers, and networks, we revisit these same strategies again and again. Years from now, when we're all using technology that hasn't even been imagined yet, this chapter should be just as relevant as it is today.

A security professional analyzes situations by finding threats and vulnerabilities to the confidentiality, integrity, and/or availability of a computing system. Often, controlling these threats and vulnerabilities involves a policy that specifies *who* (which subjects) can

access *what* (which objects) *how* (by which means). We introduced that framework in [Chapter 1](#). But now we want to delve more deeply into how such a policy works. To be effective the policy enforcement must determine *who* accurately. That is, if policy says Adam can access something, security fails if someone else impersonates Adam. Thus, to enforce security policies properly, we need ways to determine beyond a reasonable doubt that a subject's identity is accurate. The property of accurate identification is called authentication. The first critical tool for security professionals is authentication and its techniques and technologies.

When we introduced security policies we did not explicitly state the converse: A subject is allowed to access an object in a particular mode but, unless authorized, all other subjects are *not* allowed to access the object. A policy without such limits is practically useless. What good does it do to say one subject can access an object if any other subject can do so without being authorized by policy. Consequently, we need ways to restrict access to only those subjects on the *yes* list, like admitting theatre patrons to a play (with tickets) or checking in invitees to a party (on a guest list). Someone or something controls access, for example, an usher collects tickets or a host manages the guest list. Allowing exactly those accesses authorized is called access control. Mechanisms to implement access control are another fundamental computer security tool.

Suppose you were trying to limit access to a football match being held on an open park in a populous city. Without a fence, gate, or moat, you could not limit who could see the game. But suppose you had super powers and could cloak the players in invisibility uniforms. You would issue special glasses only to people allowed to see the match; others might look but see nothing. Although this scenario is pure fantasy, such an invisibility technology does exist, called encryption. Simply put, encryption is a tool by which we can transform data so only intended receivers (who have keys, the equivalent of anti-cloaking glasses) can deduce the concealed bits. The third and final fundamental security tool in this chapter is encryption.

In this chapter we describe these tools and then give a few examples to help you understand how the tools work. But most applications of these tools come in later chapters, where we elaborate on their use in the context of a more complete security situation. In most of this chapter we dissect our three fundamental security tools: authentication, access control, and encryption.

2.1 Authentication

Your neighbor recognizes you, sees you frequently, and knows you are someone who should be going into your home. Your neighbor can also notice someone different, especially if that person is doing something suspicious, such as snooping around your doorway, peering up and down the walk, or picking up a heavy stone. Coupling these suspicious events with hearing the sound of breaking glass, your neighbor might even call the police.

Computers have replaced many face-to-face interactions with electronic ones. With no vigilant neighbor to recognize that something is awry, people need other mechanisms to separate authorized from unauthorized parties. For this reason, the basis of computer security is controlled access: *someone* is authorized to take *some action* on *something*. We

examine access control later in this chapter. But for access control to work, we need to be sure who the “someone” is. In this section we introduce authentication, the process of ascertaining or confirming an identity.

A computer system does not have the cues we do with face-to-face communication that let us recognize our friends. Instead computers depend on data to recognize others. Determining who a person really is consists of two separate steps:

- **Identification** is the act of asserting who a person is.
- **Authentication** is the act of proving that asserted identity: that the person is who she says she is.

Identification is asserting who a person is.

Authentication is proving that asserted identity.

We have phrased these steps from the perspective of a person seeking to be recognized, using the term “person” for simplicity. In fact, such recognition occurs between people, computer processes (executing programs), network connections, devices, and similar active entities. In security, all these entities are called **subjects**.

The two concepts of identification and authentication are easily and often confused. Identities, like names, are often well known, public, and not protected. On the other hand, authentication is necessarily protected. If someone’s identity is public, anyone can claim to be that person. What separates the pretenders from the real person is proof by authentication.

Identification Versus Authentication

Identities are often well known, predictable, or guessable. If you send email to someone, you implicitly send along your email account ID so the other person can reply to you. In an online discussion you may post comments under a screen name as a way of linking your various postings. Your bank account number is printed on checks you write; your debit card account number is shown on your card, and so on. In each of these cases you reveal a part of your identity. Notice that your identity is more than just your name: Your bank account number, debit card number, email address, and other things are ways by which people and processes identify you.

Some account IDs are not hard to guess. Some places assign user IDs as the user’s last name followed by first initial. Others use three initials or some other scheme that outsiders can easily predict. Often for online transactions your account ID is your email address, to make it easy for you to remember. Other accounts identify you by telephone, social security, or some other identity number. With too many accounts to remember, you may welcome places that identify you by something you know well because you use it often. But using it often also means other people can know or guess it as well. For these reasons, many people could easily, although falsely, claim to be you by presenting one of your known identifiers.

Identities are typically public or well known. Authentication should be private.

Authentication, on the other hand, should be reliable. If identification asserts your identity, authentication confirms that you are who you purport to be. Although identifiers may be widely known or easily determined, authentication should be private. However, if the authentication process is not strong enough, it will not be secure. Consider, for example, how one political candidate's email was compromised by a not-private-enough authentication process as described in [Sidebar 2-1](#).

Sidebar 2-1 Vice Presidential Candidate Sarah Palin's Email Exposed

During the 2008 U.S. presidential campaign, vice presidential candidate Sarah Palin's personal email account was hacked. Contents of email messages and Palin's contacts list were posted on a public bulletin board. A 20-year-old University of Tennessee student, David Kernell, was subsequently convicted of unauthorized access to obtain information from her computer and sentenced to a year and a day.

How could a college student have accessed the computer of a high-profile public official who at the time was governor of Alaska and a U.S. vice presidential candidate under protection of the U.S. Secret Service? Easy: He simply pretended to be her. But surely nobody (other than, perhaps, comedian Tina Fey) could successfully impersonate her. Here is how easy the attack was.

Governor Palin's email account was gov.palin@yahoo.com. The account ID was well known because of news reports of an earlier incident involving Palin's using her personal account for official state communications; even without the publicity the account name would not have been hard to guess.

But the password? No, the student didn't guess the password. All he had to do was pretend to be Palin and claim she had forgotten her password. Yahoo asked Kernell the security questions Palin had filed with Yahoo on opening the account: birth date (found from Wikipedia), postcode (public knowledge, especially because she had gotten public attention for not using the official governor's mansion), and where she met her husband (part of her unofficial biography circulating during the campaign: she and her husband met in high school). With those three answers, Kernell was able to change her password (to "popcorn," something appealing to most college students). From that point on, not only was Kernell effectively Palin, the real Palin could not access her own email account because she did not know the new password.

Authentication mechanisms use any of three qualities to confirm a user's identity:

- Something the user *knows*. Passwords, PIN numbers, passphrases, a secret handshake, and mother's maiden name are examples of what a user may know.
- Something the user *is*. These authenticators, called biometrics, are based on a physical characteristic of the user, such as a fingerprint, the pattern of a person's voice, or a face (picture). These authentication methods are old (we recognize friends in person by their faces or on a telephone by their voices) but are just starting to be used in computer authentications.

- Something the user *has*. Identity badges, physical keys, a driver's license, or a uniform are common examples of things people have that make them recognizable.

Two or more forms can be combined; for example, a bank card and a PIN combine something the user has (the card) with something the user knows (the PIN).

Authentication is based on something you know, are, or have.

Although passwords were the first form of computer authentication and remain popular, these other forms are becoming easier to use, less expensive, and more common. In the following sections we examine each of these forms of authentication.

Authentication Based on Phrases and Facts: Something You Know

Password protection seems to offer a relatively secure system for confirming identity-related information, but human practice sometimes degrades its quality. Let us explore vulnerabilities in authentication, focusing on the most common authentication parameter, the password. In this section we consider the nature of passwords, criteria for selecting them, and ways of using them for authentication. As you read the following discussion of password vulnerabilities, think about how well these identity attacks would work against security questions and other authentication schemes with which you may be familiar. And remember how much information about us is known—sometimes because we reveal it ourselves—as described in [Sidebar 2-2](#).

Sidebar 2-2 Facebook Pages Answer Security Questions

George Bronk, a 23-year-old resident of Sacramento, California, pleaded guilty on 13 January 2011 to charges including computer intrusion, false impersonation, and possession of child pornography. His crimes involved impersonating women with data obtained from their Facebook accounts.

According to an Associated Press news story [[THO11](#)], Bronk scanned Facebook pages for pages showing women's email addresses. He then read their Facebook profiles carefully for clues that could help him answer security questions, such as a favorite color or a father's middle name. With these profile clues, Bronk then turned to the email account providers. Using the same technique as Kernell, Bronk pretended to have forgotten his (her) password and sometimes succeeded at answering the security questions necessary to recover a forgotten password. He sometimes used the same technique to obtain access to Facebook accounts.

After he had the women's passwords, he perused their sent mail folders for embarrassing photographs; he sometimes mailed those to a victim's contacts or posted them on her Facebook page. He carried out his activities from December 2009 to October 2010. When police confiscated his computer and analyzed its contents, they found 3200 Internet contacts and 172 email files containing explicit photographs; police sent mail to all the contacts to ask if they had been victimized, and 46 replied that they had. The victims lived in England, Washington, D.C., and 17 states from California to New Hampshire.

The California attorney general's office advised those using email and social-networking sites to pick security questions and answers that aren't posted on public sites, or to add numbers or other characters to common security answers. Additional safety tips are on the attorney general's website.

Password Use

The use of passwords is fairly straightforward, as you probably already know from experience. A user enters some piece of identification, such as a name or an assigned user ID; this identification can be available to the public or can be easy to guess because it does not provide the real protection. The protection system then requests a password from the user. If the password matches the one on file for the user, the user is authenticated and allowed access. If the password match fails, the system requests the password again, in case the user mistyped.

Even though passwords are widely used, they suffer from some difficulties of use:

- *Use.* Supplying a password for each access to an object can be inconvenient and time consuming.
- *Disclosure.* If a user discloses a password to an unauthorized individual, the object becomes immediately accessible. If the user then changes the password to re-protect the object, the user must inform any other legitimate users of the new password because their old password will fail.
- *Revocation.* To revoke one user's access right to an object, someone must change the password, thereby causing the same problems as disclosure.
- *Loss.* Depending on how the passwords are implemented, it may be impossible to retrieve a lost or forgotten password. The operators or system administrators can certainly intervene and provide a new password, but often they cannot determine what password a user had chosen previously. If the user loses (or forgets) the password, administrators must assign a new one.

Attacking and Protecting Passwords

How secure are passwords themselves? Passwords are somewhat limited as protection devices because of the relatively small number of bits of information they contain. Worse, people pick passwords that do not even take advantage of the number of bits available: Choosing a well-known string, such as `qwerty`, `password`, or `123456` reduces an attacker's uncertainty or difficulty essentially to zero.

Knight and Hartley [[KNI98](#)] list, in order, 12 steps an attacker might try in order to determine a password. These steps are in increasing degree of difficulty (number of guesses), and so they indicate the amount of work to which the attacker must go in order to derive a password. Here are their password guessing steps:

- no password
- the same as the user ID
- is, or is derived from, the user's name
- on a common word list (for example, `password`, `secret`, `private`) plus common names and patterns (for example, `qwerty`, `aaaaaa`)

- contained in a short college dictionary
- contained in a complete English word list
- contained in common non-English-language dictionaries
- contained in a short college dictionary with capitalizations (PaSsWoRd) or substitutions (digit 0 for letter O, and so forth)
- contained in a complete English dictionary with capitalizations or substitutions
- contained in common non-English dictionaries with capitalization or substitutions
- obtained by brute force, trying all possible combinations of alphabetic characters
- obtained by brute force, trying all possible combinations from the full character set

Although the last step will always succeed, the steps immediately preceding it are so time consuming that they will deter all but the most dedicated attacker for whom time is not a limiting factor.

Every password can be guessed; password strength is determined by how many guesses are required.

We now expand on some of these approaches.

Dictionary Attacks

Several network sites post dictionaries of phrases, science fiction character names, places, mythological names, Chinese words, Yiddish words, and other specialized lists. These lists help site administrators identify users who have chosen weak passwords, but the same dictionaries can also be used by attackers of sites that do not have such attentive administrators. The COPS [[FAR90](#)], Crack [[MUF92](#)], and SATAN [[FAR95](#)] utilities allow an administrator to scan a system for weak passwords. But these same utilities, or other homemade ones, allow attackers to do the same. Now Internet sites offer so-called password recovery software as freeware or shareware for under \$20. (These are password-cracking programs.)

People think they can be clever by picking a simple password and replacing certain characters, such as 0 (zero) for letter O, 1 (one) for letter I or L, 3 (three) for letter E or @ (at) for letter A. But users aren't the only people who could think up these substitutions.

Inferring Passwords Likely for a User

If Sandy is selecting a password, she is probably not choosing a word completely at random. Most likely Sandy's password is something meaningful to her. People typically choose personal passwords, such as the name of a spouse, child, other family member, or pet. For any given person, the number of such possibilities is only a dozen or two. Trying this many passwords by computer takes well under a second! Even a person working by hand could try ten likely candidates in a minute or two.

Thus, what seemed formidable in theory is in fact quite vulnerable in practice, and the

likelihood of successful penetration is frighteningly high. Morris and Thompson [[MOR79](#)] confirmed our fears in their report on the results of having gathered passwords from many users, shown in [Table 2-1](#). [Figure 2-1](#) (based on data from that study) shows the characteristics of the 3,289 passwords gathered. The results from that study are distressing, and the situation today is likely to be the same. Of those passwords, 86 percent could be uncovered in about one week's worth of 24-hour-a-day testing, using the very generous estimate of 1 millisecond per password check.

Number	Percentage	Structure
15	<1%	Single ASCII character
72	2%	Two ASCII characters
464	14%	Three ASCII characters
477	14%	Four alphabetic letters
706	21%	Five alphabetic letters, all the same case
605	18%	Six lowercase alphabetic letters
492	15%	Words in dictionaries or lists of names
2831	86%	Total of all categories above

TABLE 2-1 Password Characteristics

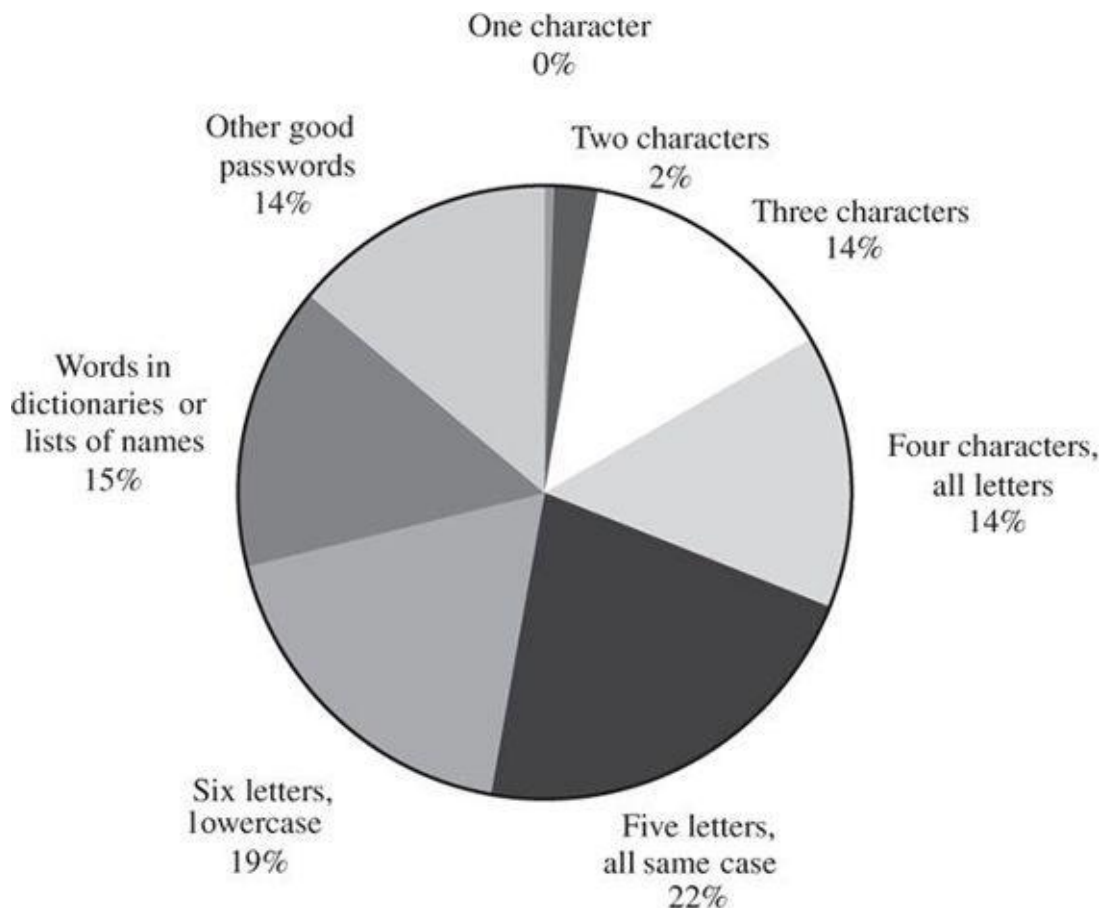


FIGURE 2-1 Distribution of Password Types

Lest you dismiss these results as dated (they were reported in 1979), Klein repeated the experiment in 1990 [[KLE90](#)] and Spafford in 1992 [[SPA92a](#)]. Each collected approximately 15,000 passwords. Klein reported that 2.7 percent of the passwords were guessed in only 15 minutes of machine time (at the speed of 1990s computers), and 21 percent were guessed within a week! Spafford found that the average password length was 6.8 characters and that 28.9 percent consisted of only lowercase alphabetic characters.

Then, in 2002 the British online bank Egg found its users still choosing weak passwords [[BUX02](#)]. A full 50 percent of passwords for their online banking service were family members' names: 23 percent children's names, 19 percent a spouse or partner, and 9 percent their own. Alas, pets came in at only 8 percent, while celebrities and football (soccer) stars tied at 9 percent each. And in 1998, Knight and Hartley [[KNI98](#)] reported that approximately 35 percent of passwords are deduced from syllables and initials of the account owner's name. In December 2009 the computer security firm Imperva analyzed 34 million Facebook passwords that had previously been disclosed accidentally, reporting that

- about 30 per cent of users chose passwords of fewer than seven characters.
- nearly 50 per cent of people used names, slang words, dictionary words or trivial passwords—consecutive digits, adjacent keyboard keys and so on.
- most popular passwords included 12345, 123456, 1234567, password, and iloveyou, in the top ten.

Two friends we know told us their passwords as we helped them administer their systems, and their passwords would both have been among the first we would have guessed. But, you say, these are amateurs unaware of the security risk of a weak password. At a recent meeting, a security expert related this experience: He thought he had chosen a solid password, so he invited a class of students to ask him questions and offer guesses as to his password. He was amazed that they asked only a few questions before they had deduced the password. And this was a security expert!

The conclusion we draw from these incidents is that people choose weak and easily guessed passwords more frequently than some might expect. Clearly, people find something in the password process that is difficult or unpleasant: Either people are unable to choose good passwords, perhaps because of the pressure of the situation, or they fear they will forget solid passwords. In either case, passwords are not always good authenticators.

Guessing Probable Passwords

Think of a word. Is the word you thought of long? Is it uncommon? Is it hard to spell or to pronounce? The answer to all three of these questions is probably no.

Penetrators searching for passwords realize these very human characteristics and use them to their advantage. Therefore, penetrators try techniques that are likely to lead to rapid success. If people prefer short passwords to long ones, the penetrator will plan to try all passwords but to try them in order by length. There are only $26^1 + 26^2 + 26^3 = 18,278$ (not case sensitive) passwords of length 3 or less. Testing that many passwords would be difficult but possible for a human, but repetitive password testing is an easy computer

application. At an assumed rate of one password per millisecond, all of these passwords can be checked in 18.278 seconds, hardly a challenge with a computer. Even expanding the tries to 4 or 5 characters raises the count only to 475 seconds (about 8 minutes) or 12,356 seconds (about 3.5 hours), respectively.

This analysis assumes that people choose passwords such as vxlag and msms as often as they pick enter and beer. However, people tend to choose names or words they can remember. Many computing systems have spelling checkers that can be used to check for spelling errors and typographic mistakes in documents. These spelling checkers sometimes carry online dictionaries of the most common English words. One contains a dictionary of 80,000 words. Trying all of these words as passwords takes only 80 seconds at the unrealistically generous estimate of one guess per millisecond.

Nobody knows what the most popular password is, although some conjecture it is “password.” Other common ones are user, abc123, aaaaaa (or aaaaa or aaaaaaa), 123456, and asdfg or qwerty (the arrangement of keys on a keyboard). Lists of common passwords are easy to find (for example, http://blog.jimmyr.com/Password_analysis_of_databases_that_were_hacked_28_2009.php). See also <http://threatpost.com/password-is-no-longer-the-worst-password/103746> for a list of the most common passwords, obtained in a data breach from Adobe, Inc. From these examples you can tell that people often use anything simple that comes to mind as a password, so human attackers might succeed by trying a few popular passwords.

Common passwords—such as qwerty, password, 123456—are used astonishingly often.

Defeating Concealment

Easier than guessing a password is just to read one from a table, like the sample table shown in [Table 2-2](#). The operating system authenticates a user by asking for a name and password, which it then has to validate, most likely by comparing to a value stored in a table. But that table then becomes a treasure trove for evil-doers: Obtaining the table gives access to all accounts because it contains not just one but all user IDs and their corresponding passwords.

Identity	Password
Jane	qwerty
Pat	aaaaaa
Phillip	oct31witch
Roz	aaaaaa
Herman	guessme
Claire	aq3wm\$oto!4

TABLE 2-2 Sample Password Table

Operating systems stymie that approach by storing passwords not in their public form but in a concealed form (using encryption, which we describe later in this chapter), as shown in [Table 2-3](#). When a user creates a password, the operating system accepts and immediately conceals it, storing the unreadable version. Later when the user attempts to authenticate by entering a user ID and password, the operating system accepts whatever is typed, applies the same concealing function, and compares the concealed version with what is stored. If the two forms match, the authentication passes.

Identity	Password
Jane	0x471aa2d2
Pat	0x13b9c32f
Phillip	0x01c142be
Roz	0x13b9c32f
Herman	0x5202aae2
Claire	0x488b8c27

TABLE 2-3 Sample Password Table with Concealed Password Values

Operating systems store passwords in hidden (encrypted) form so that compromising the id–password list does not give immediate access to all user accounts.

We used the term “conceal” in the previous paragraph because sometimes the operating system performs some form of scrambling that is not really encryption, or sometimes it is a restricted form of encryption. The only critical point is that the process be one-way: Converting a password to its concealment form is simple, but going the other way (starting with a concealed version and deriving the corresponding password) is effectively impossible. (For this reason, on some websites if you forget your password, the system can reset your password to a new, random value, but it cannot tell you what your forgotten password was.)

For active authentication, that is, entering identity and authenticator to be able to access a system, most systems lock out a user who fails a small number of successive login attempts. This failure count prevents an attacker from attempting more than a few guesses. (Note, however, that this lockout feature gives an attacker a way to prevent access by a legitimate user: simply enter enough incorrect passwords to cause the system to block the account.) However, if the attacker obtains an encrypted password table and learns the concealment algorithm, a computer program can easily test hundreds of thousands of guesses in a matter of minutes.

As numerous studies in this chapter confirmed, people often use one of a few predictable passwords. The interceptor can create what is called a rainbow table, a list of the concealed forms of the common passwords, as shown in [Table 2-4](#). Searching for matching entries in an intercepted password table, the intruder can learn that Jan’s

password is 123456 and Mike's is qwerty. The attacker sorts the table to make lookup fast.

Original Password	Encrypted Password
asdfg	0x023c94fc
p@55w0rd	0x04ff38d9
aaaaaa	0x13b9c32f
password	0x2129f30d
qwerty	0x471aa2d2
12345678	0x4f2c4dd8
123456	0x5903c34d
aaaaa	0x8384a8c8
	etc.

TABLE 2-4 Sample Rainbow Table for Common Passwords

Rainbow table: precomputed list of popular values, such as passwords

Scrambled passwords have yet another vulnerability. Notice in [Table 2-2](#) that Pat and Roz both chose the same password. Both copies will have the same concealed value, so someone who intercepts the table can learn that users Pat and Roz have the same password. Knowing that, the interceptor can also guess that Pat and Roz both chose common passwords, and start trying the usual ones; when one works, the other will, too.

To counter both these threats, some systems use an extra piece called the **salt**. A salt is an extra data field different for each user, perhaps the date the account was created or a part of the user's name. The salt value is joined to the password before the combination is transformed by concealment. In this way, Pat+aaaaaa has a different concealment value from Roz+aaaaaa, as shown in [Table 2-5](#). Also, an attacker cannot build a rainbow table because the common passwords now all have a unique component, too.

Identity	ID+password (not stored in table)	Stored Authentication Value
Jane	Jan+qwerty	0x1d46e346
Pat	Pat+aaaaaa	0x2d5d3e44
Phillip	Phi+oct31witch	0xc23c04d8
Roz	Roz+aaaaaa	0xe30f4d27
Herman	Her+guessme	0x8127f48d
Claire	Cla+aq3wm\$oto!4	0x5209d942

TABLE 2-5 Sample Password Table with Personalized Concealed Password Values

Salt: user-specific component joined to an encrypted password to distinguish identical passwords

Exhaustive Attack

In an **exhaustive** or **brute force** attack, the attacker tries all possible passwords, usually in some automated fashion. Of course, the number of possible passwords depends on the implementation of the particular computing system. For example, if passwords are words consisting of the 26 characters A–Z and can be of any length from 1 to 8 characters, there are 26^1 passwords of 1 character, 26^2 passwords of 2 characters, and 26^8 passwords of 8 characters. Therefore, the system as a whole has $26^1 + 26^2 + \dots + 26^8 = 26^9 - 1 \sim 5 \times 10^{12}$ or five million million possible passwords. That number seems intractable enough. If we were to use a computer to create and try each password at a rate of checking one password per millisecond, it would take on the order of 150 years to test all eight-letter passwords. But if we can speed up the search to one password per microsecond, the work factor drops to about two months. This amount of time is reasonable for an attacker to invest if the reward is large. For instance, an intruder may try brute force to break the password on a file of credit card numbers or bank account information.

But the break-in time can be made even more tractable in a number of ways. Searching for a single particular password does not necessarily require all passwords to be tried; an intruder need try only until the correct password is identified. If the set of all possible passwords were evenly distributed, an intruder would likely need to try only half of the password space: the expected number of searches to find any particular password. However, an intruder can also use to advantage the uneven distribution of passwords. Because a password has to be remembered, people tend to pick simple passwords; therefore, the intruder should try short combinations of characters before trying longer ones. This feature reduces the average time to find a match because it reduces the subset of the password space searched before finding a match. And as we described earlier, the attacker can build a rainbow table of the common passwords, which reduces the attack effort to a simple table lookup.

All these techniques to defeat passwords, combined with usability issues, indicate that we need to look for other methods of authentication. In the next section we explore how to implement strong authentication as a control against impersonation attacks. For another example of an authentication problem, see [Sidebar 2-3](#).

Good Passwords

Chosen carefully, passwords can be strong authenticators. The term “password” implies a single word, but you can actually use a nonexistent word or a phrase. So 2Brn2Bti? could be a password (derived from “to be or not to be, that is the question”) as could “PayTaxesApril15th.” Note that these choices have several important characteristics: The strings are long, they are chosen from a large set of characters, and they do not appear in a dictionary. These properties make the password difficult (but, of course, not impossible) to determine. If we do use passwords, we can improve their security by a few simple practices:

- *Use characters other than just a–z.* If passwords are chosen from the letters a–z, there are only 26 possibilities for each character. Adding digits expands the number of possibilities to 36. Using both uppercase and lowercase letters plus digits expands the number of possible characters to 62. Although this change seems small, the effect is large when someone is testing a full space of all possible combinations of characters. It takes about 100 hours to test all 6-letter words chosen from letters of one case only, but it takes about 2 years to test all 6-symbol passwords from upper- and lowercase letters and digits. Although 100 hours is reasonable, 2 years is oppressive enough to make this attack far less attractive.

Sidebar 2-3 Will the Real Earl of Buckingham Please Step Forward?

A man claiming to be the Earl of Buckingham was identified as Charlie Stopford, a man who had disappeared from his family in Florida in 1983 and assumed the identity of Christopher Buckingham, an 8-month-old baby who died in 1963. Questioned in England in 2005 after a check of passport details revealed the connection to the deceased Buckingham baby, Stopford was arrested when he didn’t know other correlating family details [[PAN06](#)]. (His occupation at the time of his arrest? Computer security consultant.)

The British authorities knew he was not Christopher Buckingham, but what was his real identity? The answer was discovered only because his family in the United States thought it recognized him from photos and a news story: Stopford was a husband and father who had disappeared more than 20 years earlier. Because he had been in the U.S. Navy (in military intelligence, no less) and his adult fingerprints were on file, authorities were able to make a positive identification.

As for the title he appropriated for himself, there has been no Earl of Buckingham since 1687.

In modern society we are accustomed to a full paper trail documenting events from birth through death, but not everybody fits neatly into that model. Consider the case of certain people who for various reasons need to change their identity.

When the government changes someone's identity (for example, when a witness goes into hiding), the new identity includes school records, addresses, employment records, and so forth.

How can we authenticate the identity of war refugees whose home country may no longer exist, let alone civil government and a records office? What should we do to authenticate children born into nomadic tribes that keep no formal birth records? How does an adult confirm an identity after fleeing a hostile territory without waiting at the passport office for two weeks for a document?

- *Choose long passwords.* The combinatorial explosion of password guessing difficulty begins around length 4 or 5. Choosing longer passwords makes it less likely that a password will be uncovered. Remember that a brute force penetration can stop as soon as the password is found. Some penetrators will try the easy cases—known words and short passwords—and move on to another target if those attacks fail.
- *Avoid actual names or words.* Theoretically, there are 26^6 , or about 300 million 6-letter “words” (meaning any combination of letters), but there are only about 150,000 words in a good collegiate dictionary, ignoring length. By picking one of the 99.95 percent nonwords, you force the attacker to use a longer brute-force search instead of the abbreviated dictionary search.
- *Use a string you can remember.* Password choice is a double bind. To remember the password easily, you want one that has special meaning to you. However, you don't want someone else to be able to guess this special meaning. One easy-to-remember password is UcnB2s. That unlikely looking jumble is a simple transformation of “you can never be too secure.” The first letters of words from a song, a few letters from different words of a private phrase, or something involving a memorable basketball score are examples of reasonable passwords. But don't be too obvious. Password-cracking tools also test replacements like 0 (zero) for o or O (letter “oh”) and 1 (one) for l (letter “ell”) or \$ for S (letter “ess”). So I10v3U is already in the search file.
- *Use variants for multiple passwords.* With accounts, websites, and subscriptions, an individual can easily amass 50 or 100 passwords, which is clearly too many to remember. Unless you use a trick. Start with a phrase as in the previous suggestion: Ih1b2s (I have one brother, two sisters). Then append some patterns involving the first few vowels and consonants of the entity for the password: Ih1b2sIvs for vIsa, Ih1b2sAfc for fAcbook, and so forth.
- *Change the password regularly.* Even if you have no reason to suspect that someone has compromised the password, you should change it from time to time. A penetrator may break a password system by obtaining an old list or working exhaustively on an encrypted list.
- *Don't write it down.* Note: This time-honored advice is relevant only if physical security is a serious risk. People who have accounts on many machines and servers, and with many applications or sites, may have trouble remembering

all the access codes. Setting all codes the same or using insecure but easy-to-remember passwords may be more risky than writing passwords on a reasonably well-protected list. (Obviously, you should not tape your PIN to your bank card or post your password on your computer screen.)

- *Don't tell anyone else.* The easiest attack is **social engineering**, in which the attacker contacts the system's administrator or a user to elicit the password in some way. For example, the attacker may phone a user, claim to be "system administration," and ask the user to verify the user's password. Under no circumstances should you ever give out your private password; legitimate administrators can circumvent your password if need be, and others are merely trying to deceive you.

These principles lead to solid password selection, but they lead to a different problem: People choose simple passwords because they have to create and remember so many passwords. Bank accounts, email access, library services, numerous websites, and other applications all seem to require a password. We cannot blame users for being tempted to use one simple password for all of them when the alternative is trying to remember dozens if not hundreds of strong passwords, as discussed in [Sidebar 2-4](#).

Sidebar 2-4 Usability in the Small versus Usability in the Large

To an application developer seeking a reasonable control, a password seems to be a straightforward mechanism for protecting an asset. But when many applications require passwords, the user's simple job of remembering one or two passwords is transformed into the nightmare of keeping track of a large number of them. Indeed, a visit to <http://www.passwordbook.com> suggests that users often have difficulty managing a collection of passwords. The site introduces you to a password and login organizer that is cheaply and easily purchased. In the words of the vendor, it is "The complete password manager for the busy Web master or network administrator ... Safe and easy, books don't crash! Now you can manage all your passwords in one hardbound book."

Although managing one password or token for an application might seem easy (we call it "usability in the small"), managing many passwords or tokens at once becomes a daunting task ("usability in the large"). The problem of remembering a large variety of items has been documented in the psychology literature since the 1950s, when Miller [[MIL56](#)] pointed out that people remember things by breaking them into memorable chunks, whether they are digits, letters, words, or some other identifiable entity. Miller initially documented how young adults had a memory span of seven (plus or minus two) chunks. Subsequent research revealed that the memory span depends on the nature of the chunk: longer chunks led to shorter memory spans: seven for digits, six for letters, and five for words. Other factors affect a person's memory span, too. Cowan [[COW01](#)] suggests that we assume a working memory span of four chunks for young adults, with shorter spans for children and senior citizens. For these reasons, usability should inform not only the choice of appropriate password construction (the small) but also the security architecture itself (the large).

Other Things Known

Passwords, or rather something only the user knows, are one form of strong authentication. Passwords are easy to create and administer, inexpensive to use, and easy to understand. However, users too often choose passwords that are easy for them to remember, but not coincidentally easy for others to guess. Also, users can forget passwords or tell them to others. Passwords come from the authentication factor of something the user knows, and unfortunately people's brains are imperfect.

Consequently, several other approaches to "something the user knows" have been proposed. For example, [Sidebar 2-5](#) describes authentication approaches employing a user's knowledge instead of a password. However, few user knowledge authentication techniques have been well tested and few scale up in any useful way; these approaches are still being researched.

Sidebar 2-5 Using Personal Patterns for Authentication

Lamandé [[LAM10](#)] reports that the GrIDSure authentication system (<http://www.gridsure.com>) has been integrated into Microsoft's Unified Access Gateway (UAG) platform. This system allows a user to authenticate herself with a one-time passcode based on a pattern of squares chosen from a grid. When the user wishes access, she is presented with a grid containing randomly assigned numbers; she then enters as her passcode the numbers that correspond to her chosen pattern. Because the displayed grid numbers change each time the grid is presented, the pattern enables the entered passcode to be a one-time code. GrIDSure is an attempt to scale a "user knowledge" approach from usability in the small to usability in the large. Many researchers (see, for example, [[SAS07](#), [BON08](#), and [BID09](#)]) have examined aspects of GrIDSure's security and usability, with mixed results. It remains to be seen how the use of GrIDSure compares with the use of a collection of traditional passwords.

Similarly, the ImageShield product from Confident Technologies (www.confidenttechnologies.com) asks a user to enroll by choosing three categories from a list; the categories might be cats, cars, and flowers, for example. Then at authentication time, the user is shown a grid of pictures, some from the user's categories and others not. Each picture has a 1-character letter or number. The user's one-time access string is the characters attached to the images from the user's preselected categories. So, if the pictures included a cat with label A, a flower with label 7, and seven other images, the user's access value would be A7. The images, characters and positions change for each access, so the authenticator differs similarly.

Authentication schemes like this are based on simple puzzles that the user can solve easily but that an imposter would be unable to guess successfully. However, with novel authentication schemes, we have to be aware of the phenomenon of usability in the small and the large: Can a user remember squares on a grid and categories of pictures and a favorite vacation spot and the formula $2a+c$ and many other nonobvious things?

Security Questions

Instead of passwords, some companies use questions to which (presumably) only the right person would know the answer. Such questions include mother's maiden name, street name from childhood, model of first automobile, and name of favorite teacher. The user picks relevant questions and supplies the answers when creating an identity.

The problem with such questions is that the answers to some can be determined with little difficulty, as was the case for Sarah Palin's email account. With the number of public records available online, mother's maiden name and street name are often available, and school friends could guess a small number of possible favorite teachers. Anitra Babic and colleagues [[BAB09](#)] documented the weakness of many of the supposedly secret question systems in current use. Joseph Bonneau and Sören Preibusch [[BON10](#)] did a detailed survey of website authentication methods and found little uniformity, many weaknesses, and no apparent correlation between the value of a site's data and its authentication requirements.

Passwords are becoming oppressive as many websites now ask users to log in. But when faced with a system that is difficult to handle, users often take the easy route: choosing an easy password and reusing it on many sites. To overcome that weakness, some systems use a form of authentication that cannot be stolen, duplicated, forgotten, lent, or lost: properties of the user, as we discuss in the next section. The technology for passing personal characteristics to a remote server requires more than a keyboard and pointing device, but such approaches are becoming more feasible, especially as password table breaches increase.

Authentication Based on Biometrics: Something You Are

Biometrics are biological properties, based on some physical characteristic of the human body. The list of biometric authentication technologies is still growing. Now devices can recognize the following biometrics:

- fingerprint
- hand geometry (shape and size of fingers)
- retina and iris (parts of the eye)
- voice
- handwriting, signature, hand motion
- typing characteristics
- blood vessels in the finger or hand
- face
- facial features, such as nose shape or eye spacing

Authentication with biometrics has advantages over passwords because a biometric cannot be lost, stolen, forgotten, or shared and is always available, always at hand, so to speak. These characteristics are difficult, if not impossible, to forge.

Examples of Biometric Authenticators

Many physical characteristics are possibilities as authenticators. In this section we

present examples of two of them, one for the size and shape of the hand, and one for the patterns of veins in the hand.

[Figure 2-2](#) shows a hand geometry reader. The user places a hand on the sensors, which detect lengths and widths of fingers, curvature, and other characteristics.

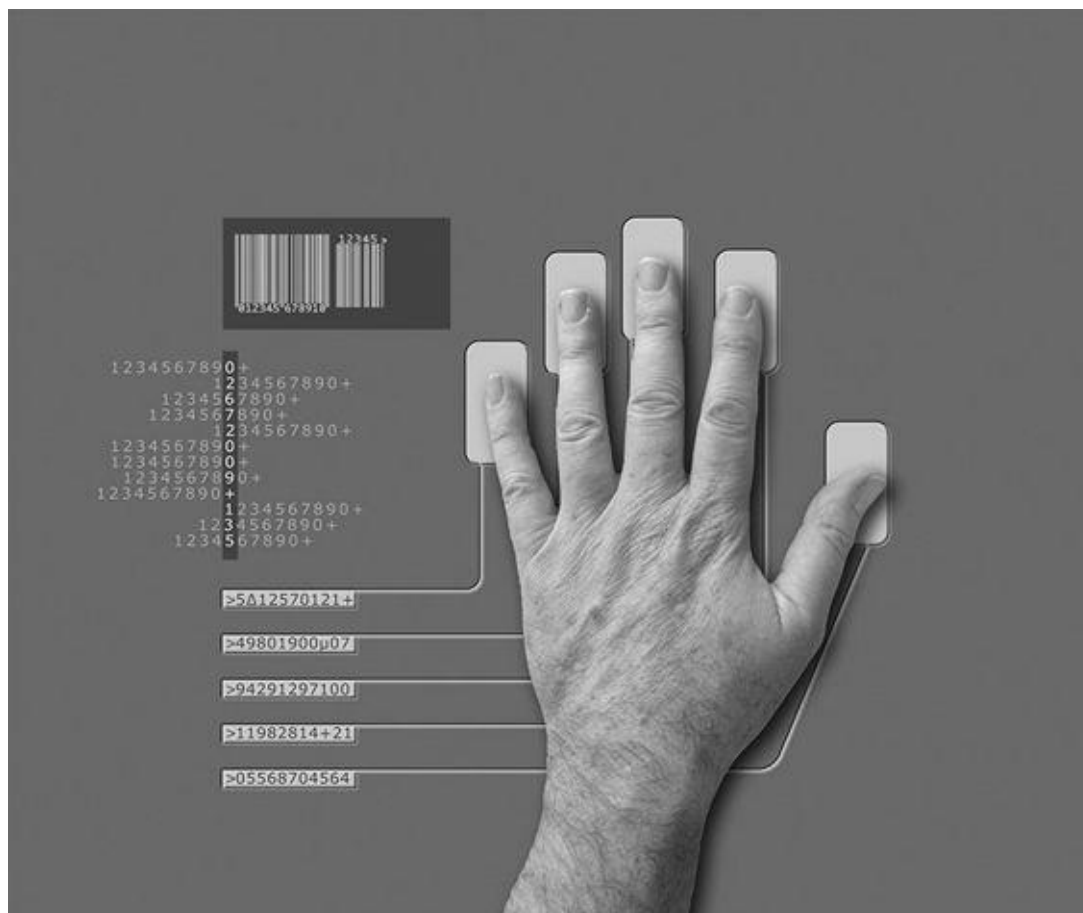


FIGURE 2-2 Hand Geometry Reader (Graeme Dawes/Shutterstock)

An authentication device from Fujitsu reads the pattern of veins in the hand. This device does not require physical contact between the hand and the reader, which is an advantage for hygiene. The manufacturer claims a false acceptance rate of 0.00008 percent and false rejection rate of 0.01 percent, with a response time of less than one second. [Figure 2-3](#) shows this device embedded in a computer mouse, so the user is automatically authenticated.



FIGURE 2-3 Hand Vein Reader (Permission for image provided courtesy of Fujitsu)

Problems with Use of Biometrics

Biometrics come with several problems:

- Biometrics are relatively *new*, and some people find their use *intrusive*. For example, people in some cultures are insulted by having to submit to fingerprinting, because they think that only criminals are fingerprinted. Hand geometry and face recognition (which can be done from a camera across the room) are scarcely invasive, but people have real concerns about peering into a laser beam or sticking a finger into a slot. (See [[SCH06a](#)] for some examples of people resisting biometrics.)
- Biometric recognition devices are *costly*, although as the devices become more popular, their cost per device should go down. Still, outfitting every user's workstation with a reader can be expensive for a large company with many employees.
- Biometric readers and comparisons can become a *single point of failure*. Consider a retail application in which a biometric recognition is linked to a payment scheme: As one user puts it, "If my credit card fails to register, I can always pull out a second card, but if my fingerprint is not recognized, I have only that one finger." (Fingerprint recognition is specific to a single finger; the pattern of one finger is not the same as another.) Manual laborers can actually rub off their fingerprints over time, and a sore or irritation may confound a fingerprint reader. Forgetting a password is a user's fault; failing biometric authentication is not.
- All biometric readers use *sampling* and establish a *threshold* for acceptance of a close match. The device has to sample the biometric, measure often hundreds of key points, and compare that set of measurements with a template. Features vary slightly from one reading to the next, for example, if your face is tilted, if you press one side of a finger more than another, or if your voice is affected by a sinus infection. Variation reduces accuracy.
- Although equipment accuracy is improving, *false readings* still occur. We label a **false positive** or **false accept** a reading that is accepted when it should be rejected (that is, the authenticator does not match) and a **false negative** or **false reject** one that rejects when it should accept. Often, reducing a false positive rate increases false negatives, and vice versa. [Sidebar 2-6](#) explains why we can never eliminate all false positives and negatives. The consequences for a false negative are usually less than for a false positive, so an acceptable system may have a false positive rate of 0.001 percent but a false negative rate of 1 percent. However, if the population is large and the asset extremely valuable, even these small percentages can lead to catastrophic results.

False positive: incorrectly confirming an identity.

False negative: incorrectly denying an identity.

Sidebar 2-6 What False Positives and Negatives Really Mean

Screening systems must be able to judge the degree to which their matching schemes work well. That is, they must be able to determine if they are effectively identifying those people who are sought while not harming those people who are not sought. When a screening system compares something it has (such as a stored fingerprint) with something it is measuring (such as a finger's characteristics), we call this a dichotomous system or test: There either is a match or there is not.

We can describe the dichotomy by using a Reference Standard, as depicted in [Table 2-6](#), below. The Reference Standard is the set of rules that determines when a positive test means a positive result. We want to avoid two kinds of errors: false positives (when there is a match but should not be) and false negatives (when there is no match but should be).

	Is the Person Claimed	Is Not the Person Claimed
Test is Positive (There is a match.)	True Positive	False Positive
Test is Negative (There is no match.)	False Negative	True Negative

TABLE 2-6 Reference Standard for Describing Dichotomous Tests

We can measure the success of the screen by using four standard measures: sensitivity, prevalence, accuracy, and specificity. To see how they work, we assign variables to the entries in [Table 2-6](#), as shown in [Table 2-7](#).

	Is the Person Claimed	Is Not the Person Claimed
Test is Positive	True Positive = a	False Positive = b
Test is Negative	False Negative = c	True Negative = d

TABLE 2-7 Reference Standard with Variables

Sensitivity measures the degree to which the screen selects those whose names correctly match the person sought. It is the proportion of positive results among all possible correct matches and is calculated as $a / (a + c)$. *Specificity* measures the proportion of negative results among all people who are not sought; it is calculated as $d / (b + d)$. Sensitivity and specificity describe how well a test discriminates between cases with and without a certain condition.

Accuracy or efficacy measures the degree to which the test or screen correctly flags the condition or situation; it is measured as $(a + d) / (a + b + c + d)$. *Prevalence* tells us how common a certain condition or situation is. It is measured as $(a + c) / (a + b + c + d)$.

Sensitivity and specificity are statistically related: When one increases, the other decreases. Thus, you cannot simply say that you are going to reduce or remove false positives; such an action is sure to increase the false negatives. Instead, you have to find a balance between an acceptable number of false

positives and false negatives. To assist us, we calculate the *positive predictive value* of a test: a number that expresses how many times a positive match actually represents the identification of the sought person. The positive predictive value is $a / (a + b)$. Similarly, we can calculate the *negative predictive value* of the test as $d / (c + d)$. We can use the predictive values to give us an idea of when a result is likely to be positive or negative. For example, a positive result of a condition that has high prevalence is likely to be positive. However, a positive result for an uncommon condition is likely to be a false positive.

The sensitivity and specificity change for a given test, depending on the level of the test that defines a match. For example, the test could call it a match only if it is an exact match: only 'Smith' would match 'Smith.' Such a match criterion would have fewer positive results (that is, fewer situations considered to match) than one that uses Soundex to declare that two names are the same: 'Smith' is the same as 'Smythe,' 'Smeth,' 'Smitt,' and other similarly sounding names. Consequently, the two tests vary in their sensitivity. The Soundex criterion is less strict and is likely to produce more positive matches; therefore, it is the more sensitive but less specific test. In general, consider the range of sensitivities that can result as we change the test criteria. We can improve the sensitivity by making the criterion for a positive test less strict. Similarly, we can improve the specificity by making the criterion for a positive test stricter.

A *receiver operating characteristic (ROC) curve* is a graphical representation of the trade-off between the false negative and false positive rates. Traditionally, the graph of the ROC shows the false positive rate ($1 - \text{specificity}$) on the x-axis and the true positive rate (sensitivity or $1 - \text{the false negative rate}$) on the y-axis. The accuracy of the test corresponds to the area under the curve. An area of 1 represents the perfect test, whereas an area of 0.5 is a worthless test. Ideally, we want a test to be as far left and as high on the graph as possible, representing a test with a high rate of true positives and a low rate of false positives. That is, the larger the area under the curve, the more the test is identifying true positives and minimizing false positives. [Figure 2-4](#) shows examples of ROC curves and their relationship to sensitivity and specificity.

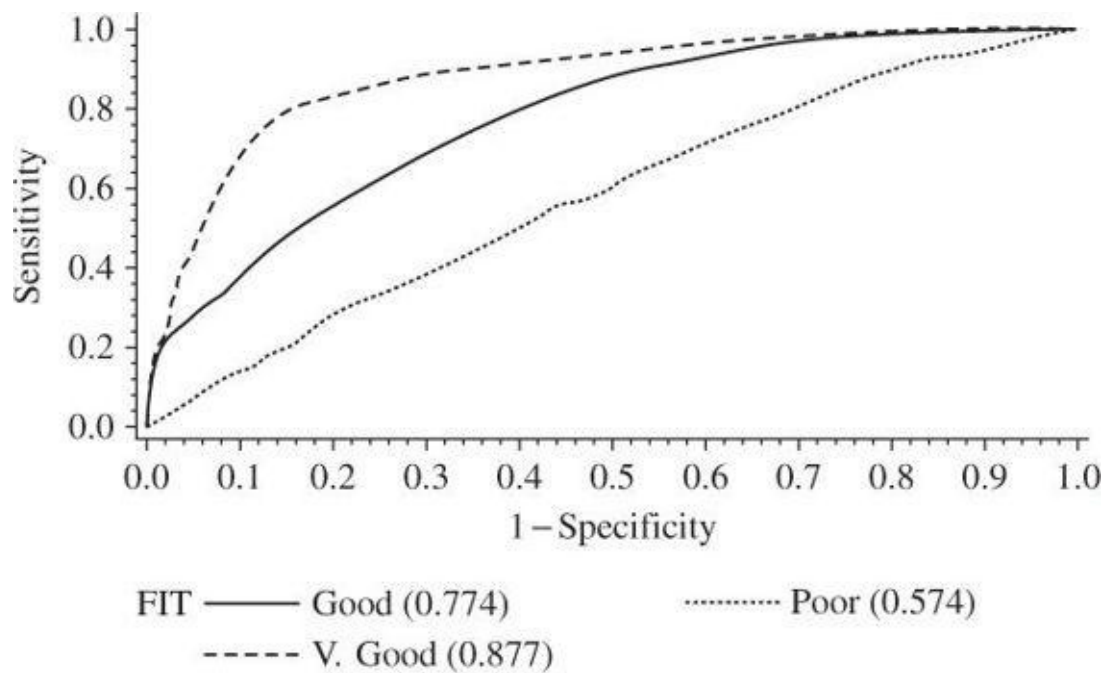


FIGURE 2-4 ROC Curves

For a matching or screening system, as for any test, system administrators must determine what levels of sensitivity and specificity are acceptable. The levels depend on the intention of the test, the setting, the prevalence of the target criterion, alternative methods for accomplishing the same goal, and the costs and benefits of testing.

- The *speed* at which a recognition must be done limits accuracy. We might ideally like to take several readings and merge the results or evaluate the closest fit. But authentication is done to allow a user to do something: Authentication is not the end goal but a gate keeping the user from the goal. The user understandably wants to get past the gate and becomes frustrated and irritated if authentication takes too long.
- Although we like to think of biometrics as unique parts of an individual, *forgeries* are possible. Some examples of forgeries are described in [Sidebar 2-7](#).

Biometrics depend on a physical characteristic that can vary from one day to the next or as people age. Consider your hands, for example: On some days, the temperature, your activity level, or other factors may cause your hands to swell, thus distorting your hands' physical characteristics. But an authentication should not fail just because the day is hot. Biometric recognition also depends on how the sample is taken. For hand geometry, for example, you place your hand on a template, but measurements will vary slightly depending on exactly how you position your hand.

Sidebar 2-7 Biometric Forgeries

The most famous fake was an artificial fingerprint produced by researchers in Japan using cheap and readily available gelatin. The researchers used molds made by pressing live fingers against them or by processing fingerprint images from prints on glass surfaces. The resulting “gummy fingers” were frequently accepted by 11 particular fingerprint devices with optical or capacitive sensors.

[[MAT02](#)]

According to another story from BBC news (13 Mar 2013) a doctor in Brazil was caught with sixteen fingers: ten authentic and six made of silicone that she used to log in to the hospital's time-card system on behalf of fellow doctors.

In a study published in 2014 [[BOW14](#)], researchers looked at whether contact lenses can be used to fool authentication devices that look at the pattern of the iris (the colored ring of the eye). The goal of the research was to determine whether iris recognition systems reliably detect true positives; that is, whether a subject will be reliably authenticated by the system. The researchers demonstrated that tinted contact lenses can fool the system into denying a match when one really exists. A subject might apply contact lenses in order to not be noticed as a wanted criminal, for example. Although difficult and uncommon, forgery will be an issue whenever the reward for a false result is high enough.

Authentication with biometrics uses a pattern or template, much like a baseline, that represents measurement of the characteristic. When you use a biometric for authentication, a current set of measurements is taken and compared to the template. The current sample need not exactly match the template, however. Authentication succeeds if the match is “close enough,” meaning it is within a predefined tolerance, for example, if 90 percent of the values match or if each parameter is within 5 percent of its expected value. Measuring, comparing, and assessing closeness for the match takes time, certainly longer than the “exact match or not” comparison for passwords. (Consider the result described in [Sidebar 2-8](#).) Therefore, the speed and accuracy of biometrics is a factor in determining their suitability for a particular environment of use.

Biometric matches are not exact; the issue is whether the rate of false positives and false negatives is acceptable.

Remember that identification is stating an identity, whereas authentication is confirming the identity, as depicted in [Figure 2-5](#). Biometrics are reliable for authentication but are much less reliable for identification. The reason is mathematical. All biometric readers operate in two phases. First, a user registers with the reader, during which time a characteristic of the user (for example, the geometry of the hand) is captured and reduced to a set of data points. During registration, the user may be asked to present the hand several times so that the registration software can adjust for variations, such as how the hand is positioned. Registration produces a pattern, called a **template**, of the data points particular to a specific user. In the second phase the user later seeks authentication from the system, during which time the system remeasures the hand and compares the new measurements with the stored template. If the new measurement is close enough to the template, the system accepts the authentication; otherwise, the system rejects it. [Sidebar 2-9](#) points out the problem in confusing identification and authentication.

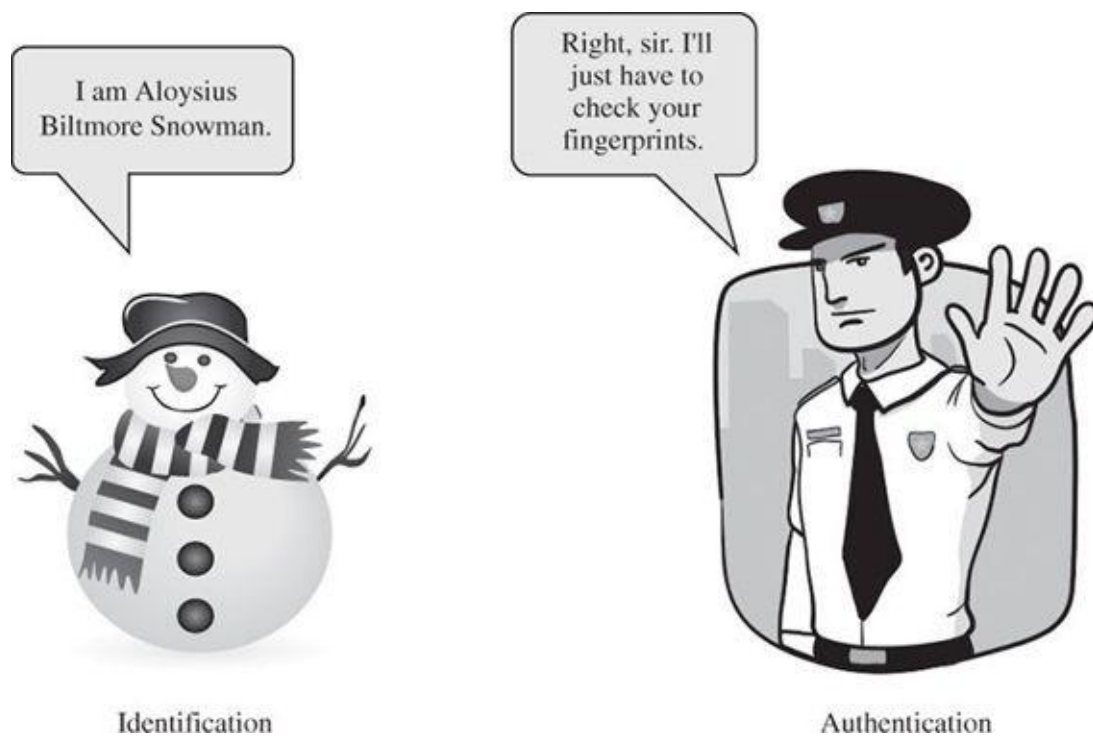


FIGURE 2-5 Identification and Authentication (courtesy of Lfoxy/Shutterstock [left]; Schotter Studio/Shutterstock [right])

Sidebar 2-8 Fingerprint Capture—Not So Fast!

Recording or capturing fingerprints should be a straightforward process. Some countries use fingerprints to track foreign visitors who enter the country, and so they want to know the impact on processing visitors at the border. On television and in the movies it seems as if obtaining a good fingerprint image takes only a second or two.

Researchers at the U.S. National Institute of Standards and Technology (NIST) performed a controlled experiment involving over 300 subjects generally representative of the U.S. population [THE07]. They found that contrary to television, obtaining a quality sample of all ten fingers takes between 45 seconds and a minute.

Sidebar 2-9 DNA for Identification or Authentication

In December 1972, a nurse in San Francisco was sexually assaulted and brutally murdered in her apartment. The landlady, who confronted a man as he rushed out of the apartment, gave a physical description to the police. At the crime scene, police collected evidence, including DNA samples of the assumed murderer. After months of investigation, however, police were unable to focus in on a suspect and the case was eventually relegated to the pile of unsolved cases.

Thirty years later, the San Francisco Police Department had a grant to use DNA to solve open cases and, upon reopening the 1972 case, they found one slide with a deteriorated DNA sample. For investigative purposes, scientists isolate 13 traits, called markers, in a DNA sample. The odds of two different people matching on all 13 markers is 1 in 1 quadrillion (1×10^{15}). However, as

described in a *Los Angeles Times* story by Jason Felch and Maura Dolan [[FEL08](#)], the old sample in this case had deteriorated and only 5½ of 13 markers were reliable. With only that many markers, the likelihood that two people would match drops to 1 in 1.1 million, and remember that for the purpose here, two people's DNA matching means at least one sample is not the criminal's.

Next, the police wanted to compare the sample with the California state database of DNA samples of convicted criminals. But to run such a comparison, administrators require at least 7 markers and police had only 5½. To search the database, police used values from two other markers that were too faint to be considered conclusive. With seven markers, police polled the database of 338,000 and came up with one match, a man subsequently tried and convicted of this crime, a man whose defense attorneys strongly believe is innocent. He had no connection to the victim, his fingerprints did not match any collected at the crime scene, and his previous conviction for a sex crime had a different pattern.

The issue is that police are using the DNA match as an identifier, not an authenticator. If police have other evidence against a particular suspect and the suspect's DNA matches that found at the crime scene, the likelihood of a correct identification increases. However, if police are looking only to find anyone whose DNA matches a sample, the likelihood of a false match rises dramatically. Remember that with a 1 in 1.1 million *false* match rate, if you assembled 1.1 million people, you would expect that one would falsely match your sample, or with 0.5 million people you would think the likelihood of a match to be approximately 1 in 2. The likelihood of a false match falls to 1 in 1.1 million people only if you examine just one person.

Think of this analogy: If you buy one lottery ticket in a 1.1 million ticket lottery, your odds of winning are 1 in 1.1 million. If you buy two tickets, your odds increase to 2 in 1.1 million, and if you buy 338,000 tickets your odds become 338,000 in 1.1 million, or roughly 1 in 3. For this reason, when seeking identification, not authentication, both the FBI's DNA advisory board and a panel of the National Research Council recommend multiplying the general probability (1 in 1.1 million) by the number of samples in the database to derive the likelihood of a random—innocent—match.

Although we do not know whether the person convicted in this case was guilty or innocent, the reasoning reminds us to be careful to distinguish between identification and authentication.

Accuracy of Biometrics

We think of biometrics—or any authentication technology—as binary: A person either passes or fails, and if we just set the parameters correctly, most of the right people will pass and most of the wrong people will fail. That is, the mechanism does not discriminate. In fact, the process is biased, caused by the balance between sensitivity and selectivity: Some people are more likely to pass and others more likely to fail. [Sidebar 2-10](#) describes how this can happen.

Until recently police and the justice system assumed that fingerprints are unique.

However, there really is no mathematical or scientific basis for this assumption. In fact, fingerprint identification has been shown to be fallible, and both human and computerized fingerprint comparison systems have also shown failures. Part of the comparison problem relates to the fact that not an entire fingerprint is compared, only characteristics at significant ridges on the print. Thus, humans or machines examine only salient features, called the template of that print.

Biometric authentication means a subject matches a template closely enough. “Close” is a system parameter that can be tuned.

Unless every template is unique, that is, no two people have the same values, the system cannot uniquely identify subjects. However, as long as an imposter is unlikely to have the same biometric template as the real user, the system can authenticate. In authentication we do not look through all templates to see who might match a set of measured features; we simply determine whether one person’s features match his stored template. Biometric authentication is feasible today; biometric identification is largely still a research topic.

Measuring the accuracy of biometric authentication is difficult because the authentication is not unique. In an experimental setting, for any one subject or collection of subjects we can compute the false negative and false positive rates because we know the subjects and their true identities. But we cannot extrapolate those results to the world and ask how many other people could be authenticated as some person. We are limited because our research population and setting may not reflect the real world. Product vendors make many claims about the accuracy of biometrics or a particular biometric feature, but few independent researchers have actually tried to substantiate the claims. In one experiment described in [Sidebar 2-11](#), expert fingerprint examiners, the people who testify about fingerprint evidence at trials, failed some of the time.

Sidebar 2-10 Are There Unremarkable People?

Are there people for whom a biometric system simply does not work? That is, are there people, for example, whose features are so indistinguishable they will always pass as someone else?

Doddington et al. [[DOD98](#)] examined systems and users to find specific examples of people who tend to be falsely rejected unusually often, those against whose profiles other subjects tend to match unusually often, and those who tend to match unusually many profiles.

To these classes Yager and Dunstone [[YAG10](#)] added people who are likely to match and cause high rates of false positives and those people who are unlikely to match themselves or anyone else. The authors then studied different biometric analysis algorithms in relation to these difficult cases.

Yager and Dunstone cited a popular belief that 2 percent of the population have fingerprints that are inherently hard to match. After analyzing a large database of fingerprints (the US-VISIT collection of fingerprints from foreign visitors to the United States) they concluded that few, if any, people are intrinsically hard to match, and certainly not 2 percent.

They examined specific biometric technologies and found that some of the errors related to the technology, not to people. For example, they looked at a database of people iris recognition systems failed to match, but they found that many of those people were wearing glasses when they enrolled in the system; they speculate that the glasses made it more difficult for the system to extract the features of an individual's iris pattern. In another case, they looked at a face recognition system. They found that people the system failed to match came from one particular ethnic group and speculated that the analysis algorithm had been tuned to distinctions of faces of another ethnic group. Thus, they concluded that matching errors are more likely the results of enrollment issues and algorithm weaknesses than of any inherent property of the people's features.

Still, for the biometric systems they studied, they found that for a specific characteristic and analysis algorithm, some users' characteristics perform better than other users' characteristics. This research reinforces the need to implement such systems carefully so that inherent limitations of the algorithm, computation, or use do not disproportionately affect the outcome.

Sidebar 2-11 Fingerprint Examiners Make Mistakes

A study supported by the U.S. Federal Bureau of investigation [[ULE11](#)] addressed the validity of expert evaluation of fingerprints. Experimenters presented 169 professional examiners with pairs of fingerprints from a pool of 744 prints to determine whether the prints matched. This experiment was designed to measure the accuracy (degree to which two examiners would reach the same conclusion) and reliability (degree to which one examiner would reach the same conclusion twice). A total of 4,083 fingerprint pairs were examined.

Of the pairs examined, six were incorrectly marked as matches, for a false positive rate of 0.01 percent. Although humans are recognized as fallible, frustratingly we cannot predict when they will be so. Thus, in a real-life setting, these false positives could represent six noncriminals falsely found guilty. The false negative rate was significantly higher, 7.5 percent, perhaps reflecting conservatism on the part of the examiners: The examiners were more likely to be unconvinced of a true match than to be convinced of a nonmatch.

The issue of false positives in fingerprint matching gained prominence after a widely publicized error related to the bombings in 2004 of commuter trains in Madrid, Spain. Brandon Mayfield, a U.S. lawyer living in Oregon, was arrested because the FBI matched his fingerprint with a print found on a plastic bag containing detonator caps at the crime scene. In 2006 the FBI admitted it had incorrectly classified the fingerprints as "an absolutely incontrovertible match."

Authentication is essential for a computing system because accurate user identification is the key to individual access rights. Most operating systems and computing system administrators have applied reasonable but stringent security measures to lock out unauthorized users before they can access system resources. But, as reported in [Sidebar 2-12](#), sometimes an inappropriate mechanism is forced into use as an authentication device.

Losing or forgetting a biometric authentication is virtually impossible because biometrics rely on human characteristics. But the characteristics can change over time (think of hair color or weight); therefore, biometric authentication may be less precise than knowledge-based authentication. You either know a password or you don't. But a fingerprint can be a 99 percent match or 95 percent or 82 percent, part of the variation depending on factors such as how you position your finger as the print is read, whether your finger is injured, and if your hand is cold or your skin is dry or dirty. Stress can also affect biometric factors, such as voice recognition, potentially working against security. Imagine a critical situation in which you need to access your computer urgently but your being agitated affects your voice. If the system fails your authentication and offers you the chance to try again, the added pressure may make your voice even worse, which threatens availability.

Biometrics can be reasonably quick and easy, and we can sometimes adjust the sensitivity and specificity to balance false positive and false negative results. But because biometrics require a device to read, their use for remote authentication is limited. The third factor of authentication, something you *have*, offers strengths and weaknesses different from the other two factors.

Sidebar 2-12 Using Cookies for Authentication

On the web, cookies are often used for authentication. A cookie is a pair of data items sent to the web browser by the visited website. The data items consist of a key and a value, designed to represent the current state of a session between a visiting user and the visited website. Once the cookie is placed on the user's system (usually in a directory with other cookies), the browser continues to use it for subsequent interaction between the user and that website. Each cookie is supposed to have an expiration date, but that date can be far in the future, and it can be modified later or even ignored.

For example, *The Wall Street Journal's* website, wsj.com, creates a cookie when a user first logs in. In subsequent transactions, the cookie acts as an identifier; the user no longer needs a password to access that site. (Other sites use the same or a similar approach.)

Users must be protected from exposure and forgery. That is, users may not want the rest of the world to know what sites they have visited. Neither will they want someone to examine information or buy merchandise online by impersonation and fraud. And furthermore, on a shared computer, one user can act as someone else if the receiving site uses a cookie to perform automatic authentication.

Sit and Fu [[SIT01](#)] point out that cookies were not designed for protection. There is no way to establish or confirm a cookie's integrity, and not all sites encrypt the information in their cookies.

Sit and Fu also point out that a server's operating system must be particularly vigilant to protect against eavesdropping: "Most [web traffic] exchanges do not use [encryption] to protect against eavesdropping; anyone on the network between the two computers can overhear the traffic. Unless a server takes strong

precautions, an eavesdropper can steal and reuse a cookie, impersonating a user indefinitely.” (In [Chapter 6](#) we describe how encryption can be used to protect against such eavesdropping.)

Authentication Based on Tokens: Something You Have

Something you have means that you have a physical object in your possession. One physical authenticator with which you are probably familiar is a key. When you put your key in your lock, the ridges in the key interact with pins in the lock to let the mechanism turn. In a sense the lock authenticates you for authorized entry because you possess an appropriate key. Of course, you can lose your key or duplicate it and give the duplicate to someone else, so the authentication is not perfect. But it is precise: Only your key works, and your key works only your lock. (For this example, we intentionally ignore master keys.)

Other familiar examples of tokens are badges and identity cards. You may have an “affinity card”: a card with a code that gets you a discount at a store. Many students and employees have identity badges that permit them access to buildings. You must have an identity card or passport to board an airplane or enter a foreign country. In these cases you possess an object that other people recognize to allow you access or privileges.

Another kind of authentication token has data to communicate invisibly. Examples of this kind of token include credit cards with a magnetic stripe, credit cards with an embedded computer chip, or access cards with passive or active wireless technology. You introduce the token into an appropriate reader, and the reader senses values from the card. If your identity and values from your token match, this correspondence adds confidence that you are who you say you are.

We describe different kinds of tokens next.

Active and Passive Tokens

As the names imply, **passive tokens** do nothing, and active ones take some action. A photo or key is an example of a passive token in that the contents of the token never change. (And, of course, with photos permanence can be a problem, as people change hair style or color and their faces change over time.)

An **active token** can have some variability or interaction with its surroundings. For example, some public transportation systems use cards with a magnetic strip. When you insert the card into a reader, the machine reads the current balance, subtracts the price of the trip and rewrites a new balance for the next use. In this case, the token is just a repository to hold the current value. Another form of active token initiates a two-way communication with its reader, often by wireless or radio signaling. These tokens lead to the next distinction among tokens, static and dynamic interaction.

Passive tokens do not change. Active tokens communicate with a sensor.

Static and Dynamic Tokens

The value of a **static token** remains fixed. Keys, identity cards, passports, credit and

other magnetic-stripe cards, and radio transmitter cards (called RFID devices) are examples of static tokens. Static tokens are most useful for onsite authentication: When a guard looks at your picture badge, the fact that you possess such a badge and that your face looks (at least vaguely) like the picture causes the guard to pass your authentication and allow you access.

We are also interested in remote authentication, that is, in your being able to prove your identity to a person or computer somewhere else. With the example of the picture badge, it may not be easy to transmit the image of the badge and the appearance of your face for a remote computer to compare. Worse, distance increases the possibility of forgery: A local guard could tell if you were wearing a mask, but a guard might not detect it from a remote image. Remote authentication is susceptible to the problem of the token having been forged.

Tokens are vulnerable to an attack called skimming. **Skimming** is the use of a device to copy authentication data surreptitiously and relay it to an attacker. Automated teller machines (ATMs) and point-of-sale credit card readers are particularly vulnerable to skimming.¹ At an ATM the thief attaches a small device over the slot into which you insert your bank card. Because all bank cards conform to a standard format (so you can use your card at any ATM or merchant), the thief can write a simple piece of software to copy and retain the information recorded on the magnetic stripe on your bank card. Some skimmers also have a tiny camera to record your key strokes as you enter your PIN on the keypad. Either instantaneously (using wireless communication) or later (collecting the physical device), the thief thus obtains both your account number and its PIN. The thief simply creates a dummy card with your account number recorded and, using the PIN for authentication, visits an ATM and withdraws cash from your account or purchases things with a cloned credit card.

¹ Note that this discussion refers to the magnetic-stripe cards popular in the United States. Most other countries use embedded computer chip cards that are substantially less vulnerable to skimming.

Another form of copying occurs with passwords. If you have to enter or speak your password, someone else can look over your shoulder or overhear you, and now that authenticator is easily copied or forged. To overcome copying of physical tokens or passwords, we can use dynamic tokens. A **dynamic token** is one whose value changes. Although there are several different forms, a dynamic authentication token is essentially a device that generates an unpredictable value that we might call a pass number. Some devices change numbers at a particular interval, for example, once a minute; others change numbers when you press a button, and others compute a new number in response to an input, sometimes called a challenge. In all cases, it does not matter if someone else sees or hears you provide the pass number, because that one value will be valid for only one access (yours), and knowing that one value will not allow the outsider to guess or generate the next pass number.

Dynamic tokens have computing power on the token to change their internal state.

Dynamic token generators are useful for remote authentication, especially of a person to a computer. An example of a dynamic token is the SecurID token from RSA Laboratories,

shown in [Figure 2-6](#). To use a SecurID token, you enter the current number displayed on the token when prompted by the authenticating application. Each token generates a distinct, virtually unpredictable series of numbers that change every minute, so the authentication system knows what number to expect from your token at any moment. In this way, two people can have SecurID tokens, but each token authenticates only its assigned owner. Entering the number from another token does not pass your authentication. And because the token generates a new number every minute, entering the number from a previous authentication fails as well.

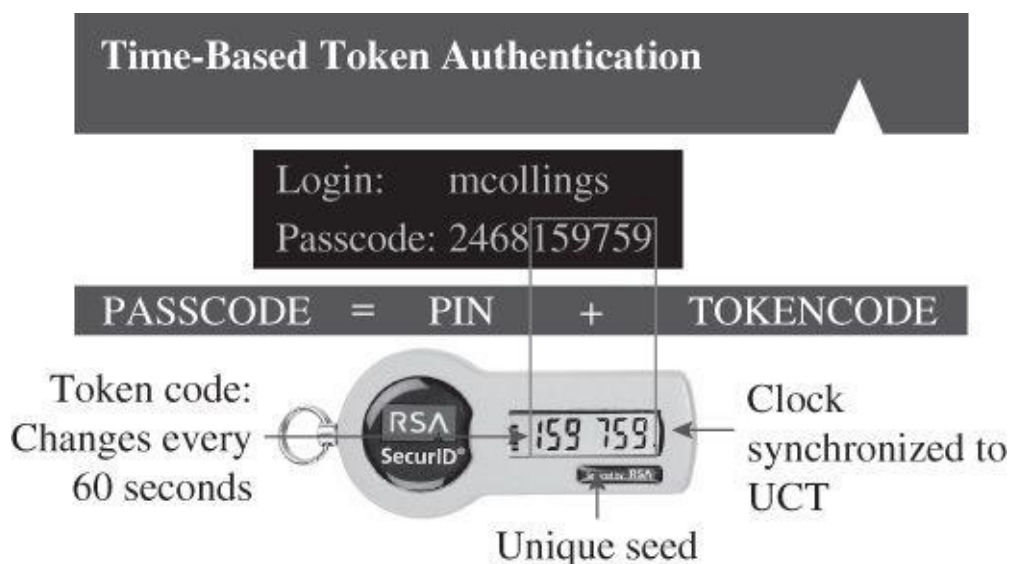


FIGURE 2-6 SecurID Token (Photo courtesy of RSA, the security division of EMS and copyright © RSA Corporation, all rights reserved.)

We have now examined the three bases of authentication: something you know, are, or have. Used in an appropriate setting, each can offer reasonable security. In the next sections we look at some ways of enhancing the basic security from these three forms.

Federated Identity Management

If these different forms of authentication seem confusing and overwhelming, they can be. Consider that some systems will require a password, others a fingerprint scan, others an active token, and others some combination of techniques. As you already know, remembering identities and distinct passwords for many systems is challenging. People who must use several different systems concurrently (email, customer tracking, inventory, and sales, for example) soon grow weary of logging out of one, into another, refreshing a login that has timed out, and creating and updating user profiles. Users rightly call for computers to handle the bookkeeping.

A **federated identity management** scheme is a union of separate identification and authentication systems. Instead of maintaining separate user profiles, a federated scheme maintains one profile with one authentication method. Separate systems share access to the authenticated identity database. Thus, authentication is performed in one place, and separate processes and systems determine that an already authenticated user is to be activated. Such a process is shown in [Figure 2-7](#).

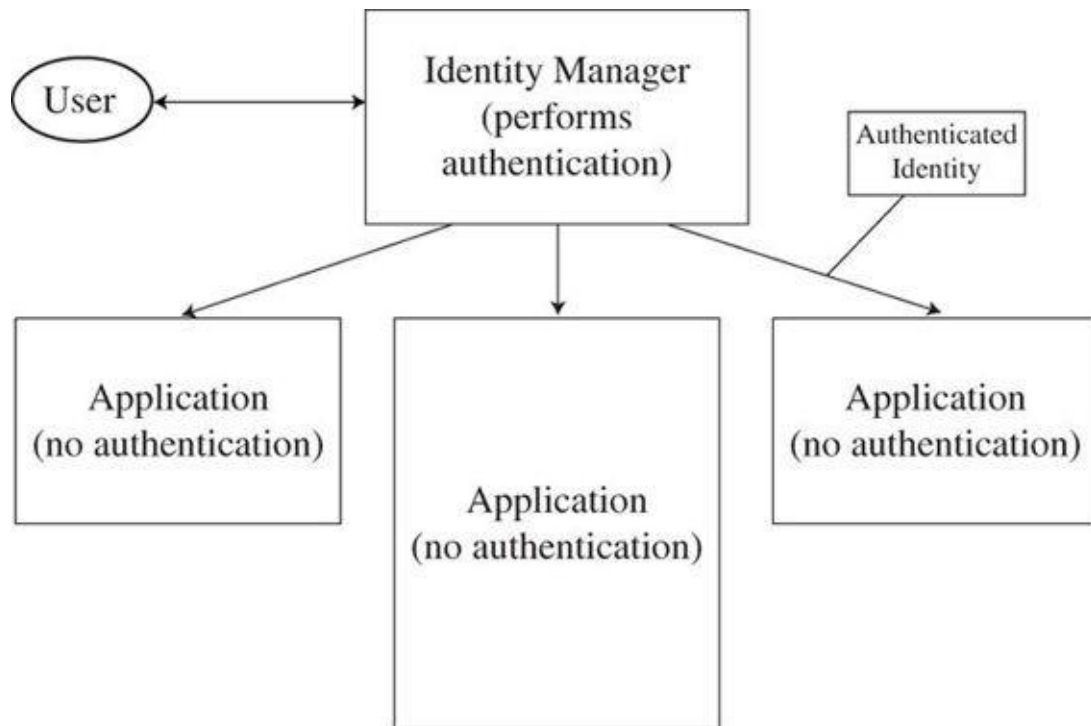


FIGURE 2-7 Federated Identity Manager

Federated identity management unifies the identification and authentication process for a group of systems.

Closely related is a **single sign-on** process, depicted in [Figure 2-8](#). Think of an umbrella procedure to which you log in once per session (for example, once a day). The umbrella procedure maintains your identities and authentication codes for all the different processes you access. When you want to access email, for example, instead of your completing a user ID and password screen, the single sign-on process passes those details to the email handler, and you resume control after the authentication step has succeeded.

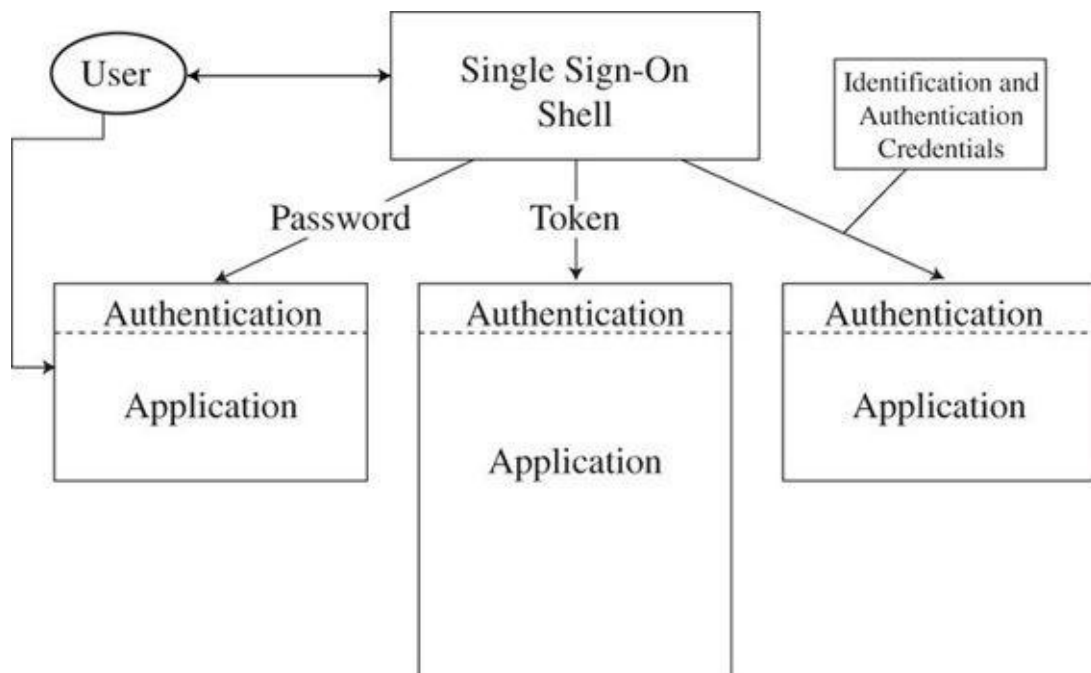


FIGURE 2-8 Single Sign-On

The difference between these two approaches is that federated identity management

involves a single identity management module that replaces identification and authentication in all other systems. Thus all these systems invoke the identity management module. With single sign-on, systems still call for individual identification and authentication, but the umbrella task performs those interactions on behalf of the user.

Single sign-on takes over sign-on and authentication to several independent systems for a user.

Multifactor Authentication

The single-factor authentication approaches discussed in this chapter offer advantages and disadvantages. For example, a token works only as long as you do not give it away (or lose it or have it stolen), and password use fails if someone can see you enter your password by peering over your shoulder. We can compensate for the limitation of one form of authentication by combining it with another form.

Identity cards, such as a driver's license, often contain a picture and signature. The card itself is a token, but anyone seeing that card can compare your face to the picture and confirm that the card belongs to you. Or the person can ask you to write your name and can compare signatures. In that way, the authentication is both token based and biometric (because your appearance and the way you sign your name are innate properties of you). Notice that your credit card has a space for your signature on the back, but in the United States few merchants compare that signature to the sales slip you sign. Having authentication factors available does not necessarily mean we use them.

As long as the process does not become too onerous, authentication can use two, three, four, or more factors. For example, to access something, you must type a secret code, slide your badge, and hold your hand on a plate.

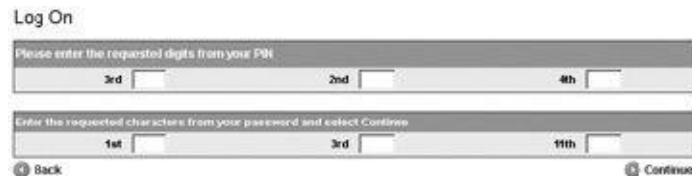
Combining authentication information is called **multifactor authentication**. Two forms of authentication (which is, not surprisingly, known as **two-factor authentication**) are presumed to be better than one, assuming of course that the two forms are strong. But as the number of forms increases, so also does the user's inconvenience. Each authentication factor requires the system and its administrators and the users to manage more security information. We assume that more factors imply higher confidence, although few studies support that assumption. And two kinds of authentication imply two pieces of software and perhaps two kinds of readers, as well as the time to perform two authentications. Indeed, even if multifactor authentication is superior to single factor, we do not know which value of n makes n -factor authentication optimal. From a usability point of view, large values of n may lead to user frustration and reduced security, as shown in [Sidebar 2-13](#).

Secure Authentication

Passwords, biometrics, and tokens can all participate in secure authentication. Of course, simply using any or all of them is no guarantee that an authentication approach will be secure. To achieve true security, we need to think carefully about the problem we are trying to solve and the tools we have; we also need to think about blocking possible attacks and attackers.

Sidebar 2-13 When More Factors Mean Less Security

Dave Concannon's blog at www.aeofsteel.com/tag/ulsterbank describes his frustration at using Ulsterbank's online banking system. The logon process involves several steps. First, the user supplies a customer identification number (the first authentication factor). Next, a separate user ID is required (factor 2). Third, the PIN is used to supply a set of digits (factor 3), as shown in the figure below: The system requests three different digits chosen at random (in the figure, the third, second, and fourth digits are to be entered). Finally, the system requires a passphrase of at least ten characters, some of which must be numbers (factor 4).



The screenshot shows a 'Log On' window with two main sections. The top section is titled 'Please enter the requested digits from your PIN' and contains three input fields labeled '3rd', '2nd', and '4th'. The bottom section is titled 'Enter the requested characters from your password and select Continue' and contains three input fields labeled '1st', '3rd', and '11th'. At the bottom left is a 'Back' button and at the bottom right is a 'Continue' button.

In his blog, Concannon rails about the difficulties not only of logging on but also of changing his password. With four factors to remember, Ulsterbank users will likely, in frustration, write down the factors and carry them in their wallets, thereby reducing the banking system's security.

Suppose we want to control access to a computing system. In addition to a name and password, we can use other information available to authenticate users. Suppose Adams works in the accounting department during the shift between 8:00 a.m. and 5:00 p.m., Monday through Friday. Any legitimate access attempt by Adams should be made during those times, through a workstation in the accounting department offices. By limiting Adams to logging in under those conditions, the system protects against two problems:

- Someone from outside might try to impersonate Adams. This attempt would be thwarted by either the time of access or the port through which the access was attempted.
- Adams might attempt to access the system from home or on a weekend, planning to use resources not allowed or to do something that would be too risky with other people around.

Limiting users to certain workstations or certain times of access can cause complications (as when a user legitimately needs to work overtime, a person has to access the system while out of town on business, or a particular workstation fails). However, some companies use these authentication techniques because the added security they provide outweighs inconvenience. As security analysts, we need to train our minds to recognize qualities that distinguish normal, allowed activity.

As you have seen, security practitioners have a variety of authentication mechanisms ready to use. None is perfect; all have strengths and weaknesses, and even combinations of mechanisms are imperfect. Often the user interface seems simple and foolproof (what could be easier than laying a finger on a glass plate?), but as we have described, underneath that simplicity lies uncertainty, ambiguity, and vulnerability. Nevertheless, in this section you have seen types and examples so that when you develop systems and

applications requiring authentication, you will be able to draw on this background to pick an approach that achieves your security needs.

2.2 Access Control

In this section we discuss how to protect general objects, such as files, tables, access to hardware devices or network connections, and other resources. In general, we want a flexible structure, so that certain users can use a resource in one way (for example, read-only), others in a different way (for example, allowing modification), and still others not at all. We want techniques that are robust, easy to use, and efficient.

We start with the basic access control paradigm, articulated by Scott Graham and Peter Denning [[GRA72](#)]: A subject is permitted to access an object in a particular mode, and only such authorized accesses are allowed.

- *Subjects* are human users, often represented by surrogate programs running on behalf of the users.
- *Objects* are things on which an action can be performed: Files, tables, programs, memory objects, hardware devices, strings, data fields, network connections, and processors are examples of objects. So too are users, or rather programs or processes representing users, because the operating system (a program representing the system administrator) can act on a user, for example, allowing a user to execute a program, halting a user, or assigning privileges to a user.
- *Access modes* are any controllable actions of subjects on objects, including, but not limited to, read, write, modify, delete, execute, create, destroy, copy, export, import, and so forth.

Effective separation will keep unauthorized subjects from unauthorized access to objects, but the separation gap must be crossed for authorized subjects and modes. In this section we consider ways to allow all and only authorized accesses.

Access control: limiting who can access what in what ways

Access Policies

Access control is a mechanical process, easily implemented by a table and computer process: A given subject either can or cannot access a particular object in a specified way. Underlying the straightforward decision is a complex and nuanced decision of which accesses should be allowed; these decisions are based on a formal or informal security policy.

Access control decisions are (or should not be) made capriciously. Pat gets access to this file because she works on a project that requires the data; Sol is an administrator and needs to be able to add and delete users for the system. Having a basis simplifies making similar decisions for other users and objects. A policy also simplifies establishing access control rules, because they just reflect the existing policy.

Thus, before trying to implement access control, an organization needs to take the time to develop a higher-level security policy, which will then drive all the access control rules.

Effective Policy Implementation

Protecting objects involves several complementary goals.

- *Check every access.* We may want to revoke a user's privilege to access an object. If we have previously authorized the user to access the object, we do not necessarily intend that the user should retain indefinite access to the object. In fact, in some situations, we may want to prevent further access immediately after we revoke authorization, for example, if we detect a user being impersonated. For this reason, we should aim to check every access by a user to an object.
- *Enforce least privilege.* The principle of least privilege states that a subject should have access to the smallest number of objects necessary to perform some task. Even if extra information would be useless or harmless if the subject were to have access, the subject should not have that additional access. For example, a program should not have access to the absolute memory address to which a page number reference translates, even though the program could not use that address in any effective way. Not allowing access to unnecessary objects guards against security weaknesses if a part of the protection mechanism should fail.

Least privilege: access to the fewest resources necessary to complete some task

- *Verify acceptable usage.* Ability to access is a yes-or-no decision. But equally important is checking that the activity to be performed on an object is appropriate. For example, a data structure such as a stack has certain acceptable operations, including push, pop, clear, and so on. We may want not only to control who or what has access to a stack but also to be assured that all accesses performed are legitimate stack accesses.

Tracking

Implementing an appropriate policy is not the end of access administration. Sometimes administrators need to revisit the access policy to determine whether it is working as it should. Has someone been around for a long time and so has acquired a large number of no-longer-needed rights? Do so many users have access to one object that it no longer needs to be controlled? Or should it be split into several objects so that individuals can be allowed access to only the pieces they need? Administrators need to consider these kinds of questions on occasion to determine whether the policy and implementation are doing what they should. We explore the management side of defining security policies in [Chapter 10](#), but we preview some issues here because they have a technical bearing on access control.

Granularity

By **granularity** we mean the fineness or specificity of access control. It is a spectrum: At one end you can control access to each individual bit or byte, each word in a document, each number on a spreadsheet, each photograph in a collection. That level of specificity is generally excessive and cumbersome to implement. The finer the granularity, the larger

number of access control decisions that must be made, so there is a performance penalty. At the other extreme you simply say Adam has complete access to computer C1. That approach may work if the computer is for Adam's use alone, but if computer C1 is shared, then the system has no basis to control or orchestrate that sharing. Thus, a reasonable midpoint must apply.

Typically a file, a program, or a data space is the smallest unit to which access is controlled. However, note that applications can implement their own access control. So, for example, as we describe in [Chapter 7](#), a database management system can have access to a complete database, but it then carves the database into smaller units and parcels out access: This user can see names but not salaries, that user can see only data on employees in the western office.

Hardware devices, blocks of memory, the space on disk where program code is stored, specific applications, all these are likely objects over which access is controlled.

Access Log

After making an access decision, the system acts to allow that access and leaves the user and the object to complete the transaction. Systems also record which accesses have been permitted, creating what is called an **audit log**. This log is created and maintained by the system, and it is preserved for later analysis. Several reasons for logging access include the following:

- Records of accesses can help plan for new or upgraded equipment, by showing which items have had heavy use.
- If the system fails, these records can show what accesses were in progress and perhaps help identify the cause of failure.
- If a user misuses objects, the access log shows exactly which objects the user did access.
- In the event of an external compromise, the audit log may help identify how the assailant gained access and which data items were accessed (and therefore revealed or compromised). These data for after-the-fact forensic analysis have been extremely helpful in handling major incidents.

As part of the access control activity, the system builds and protects this audit log. Obviously, granularity matters: A log that records each memory byte accessed is too lengthy to be of much practical value, but a log that says "8:01 user turned on system; 17:21 user turned off system" probably contains too little detail to be helpful.

In the next section we consider protection mechanisms appropriate for general objects of unspecified types, such as the kinds of objects listed above. To make the explanations easier to understand, we sometimes use an example of a specific object, such as a file. Note, however, that a general mechanism can be used to protect any of the types of objects listed.

Limited Privilege

Limited privilege is the act of restraining users and processes so that any harm they can do is not catastrophic. A system that prohibits all accesses to anything by anyone certainly

achieves both confidentiality and integrity, but it completely fails availability and usefulness. Thus, we seek a midpoint that balances the need for some access against the risk of harmful, inappropriate access. Certainly, we do not expect users or processes to cause harm. But recognizing that not all users are ethical or even competent and that not all processes function as intended, we want to limit exposure from misbehaving users or malfunctioning processes. Limited privilege is a way to constrain that exposure.

Limited privilege is a management concept, not a technical control. The process of analyzing users and determining the privileges they require is a necessary first step to authorizing within those limits. After establishing the limits, we turn to access control technology to enforce those limits. In [Chapter 3](#) we again raise the concept of limited privilege when we describe program design and implementation that ensures security. Security design principles first written by Jerome Saltzer and Michael Schroeder [[SAL75](#)] explain the advantage of limiting the privilege with which users and their programs run.

Implementing Access Control

Access control is often performed by the operating system. Only the operating system can access primitive objects, such as files, to exercise control over them, and the operating system creates and terminates the programs that represent users (subjects). However, current hardware design does not always support the operating system in implementing well-differentiated or fine-grained access control. The operating system does not usually see inside files or data objects, for example, so it cannot perform row- or element-level access control within a database. Also, the operating system cannot easily differentiate among kinds of network traffic. In these cases, the operating system defers to a database manager or a network appliance in implementing some access control aspects. With limited kinds of privileges to allocate, the operating system cannot easily both control a database manager and allow the database manager to control users. Thus, current hardware design limits some operating system designs.

Reference Monitor

James Anderson and his study committee [[AND72](#)] gave name and structure to the digital version of a concept that has existed for millennia. To protect their medieval fortresses, rulers had one heavily protected gate as the sole means of ingress. Generals surrounded troop emplacements with forts and sentry guards. Bankers kept cash and other valuables in safes with impregnable doors to which only a select few trusted people had the combinations. Fairy tale villains locked damsels away in towers. All these examples show strong access control because of fail-safe designs.

In Anderson's formulation for computers, access control depends on a combination of hardware and software that is

- always invoked; validates every access attempt
- immune from tampering
- assuredly correct

Reference monitor: access control that is always invoked, tamperproof, and verifiable

Anderson called this construct a **reference monitor**. It should be obvious why these three properties are essential.

A reference monitor is a notion, not a tool you can buy to plug into a port. It could be embedded in an application (to control the application’s objects), part of the operating system (for system-managed objects) or part of an appliance. Still, you will see these same three properties appear repeatedly in this book. To have an effective reference monitor, we need to consider effective and efficient means to translate policies, the basis for validation, into action. How we represent a policy in binary data has implications for how efficient and even how effective the mediation will be.

In the next sections we present several models of how access rights can be maintained and implemented by the reference monitor.

Access Control Directory

One simple way to protect an object is to use a mechanism that works like a file directory. Imagine we are trying to protect files (the set of objects) from users of a computing system (the set of subjects). Every file has a unique owner who possesses “control” access rights (including the rights to declare who has what access) and to revoke access of any person at any time. Each user has a file directory, which lists all the files to which that user has access.

Clearly, no user can be allowed to write in the file directory, because that would be a way to forge access to a file. Therefore, the operating system must maintain all file directories, under commands from the owners of files. The obvious rights to files are the common read, write, and execute that are familiar on many shared systems. Furthermore, another right, owner, is possessed by the owner, permitting that user to grant and revoke access rights. [Figure 2-9](#) shows an example of a file directory.

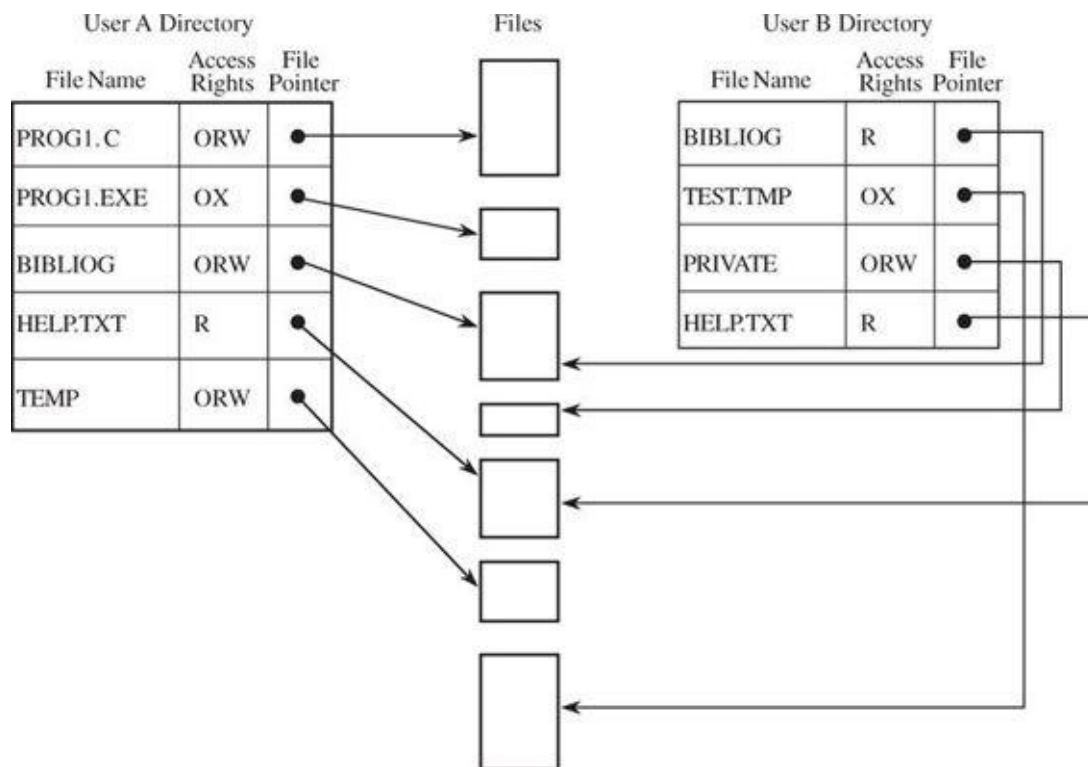


FIGURE 2-9 Directory Access Rights

This approach is easy to implement because it uses one list per user, naming all the

objects that a user is allowed to access. However, several difficulties can arise. First, the list becomes too large if many shared objects, such as libraries of subprograms or a common table of users, are accessible to all users. The directory of each user must have one entry for each such shared object, even if the user has no intention of accessing the object. Deletion must be reflected in all directories.

A second difficulty is revocation of access. If owner A has passed to user B the right to read file F, an entry for F is made in the directory for B. This granting of access implies a level of trust between A and B. If A later questions that trust, A may want to revoke the access right of B. The operating system can respond easily to the single request to delete the right of B to access F, because that action involves deleting one entry from a specific directory. But if A wants to remove the rights of everyone to access F, the operating system must search each individual directory for the entry F, an activity that can be time consuming on a large system. For example, large systems or networks of smaller systems can easily have 5,000 to 10,000 active accounts. Moreover, B may have passed the access right for F to another user C, a situation known as **propagation of access rights**, so A may not know that C's access exists and should be revoked. This problem is particularly serious in a network.

A third difficulty involves pseudonyms. Owners A and B may have two different files named F, and they may both want to allow access by S. Clearly, the directory for S cannot contain two entries under the same name for different files. Therefore, S has to be able to uniquely identify the F for A (or B). One approach is to include the original owner's designation as if it were part of the file name, with a notation such as A:F (or B:F).

Suppose, however, that S would like to use a name other than F to make the file's contents more apparent. The system could allow S to name F with any name unique to the directory of S. Then, F from A could be called Q to S. As shown in [Figure 2-10](#), S may have forgotten that Q is F from A, and so S requests access again from A for F. But by now A may have more trust in S, so A transfers F with greater rights than before. This action opens up the possibility that one subject, S, may have two distinct sets of access rights to F, one under the name Q and one under the name F. In this way, allowing pseudonyms can lead to multiple permissions that are not necessarily consistent. Thus, the directory approach is probably too simple for most object protection situations.

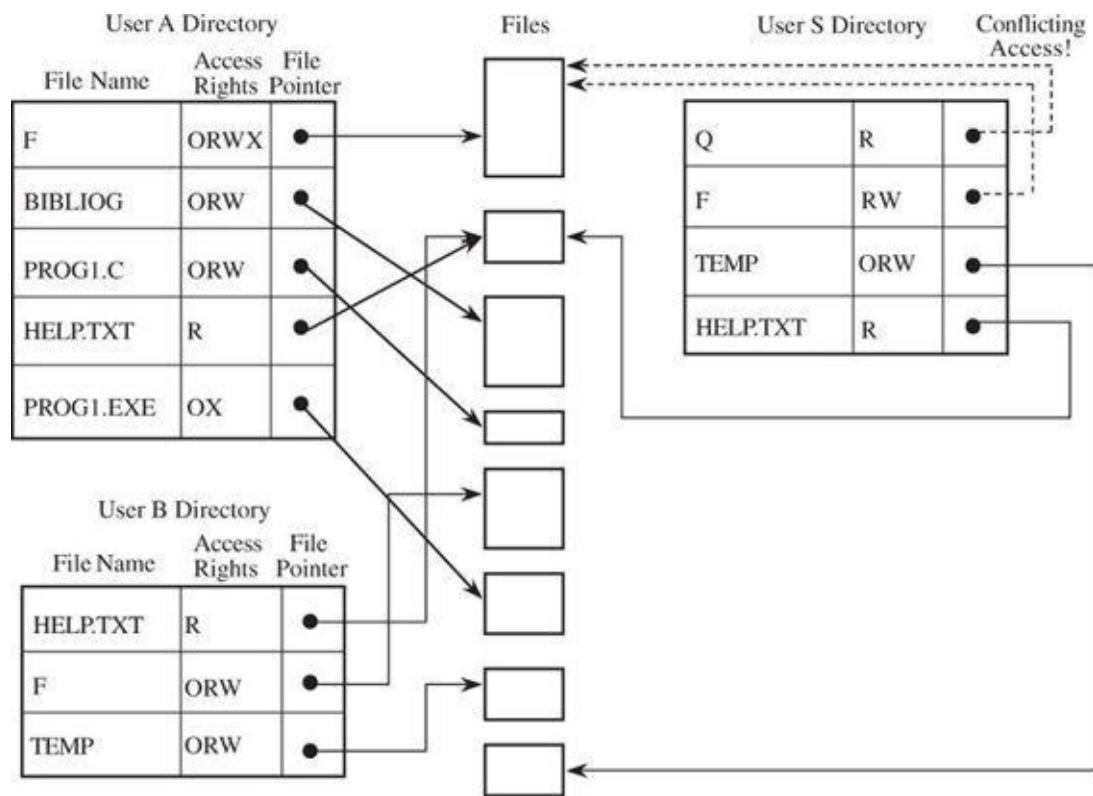


FIGURE 2-10 Ambiguous Access Rights

Access Control Matrix

We can think of the directory as a listing of objects accessible by a single subject, and the access list as a table identifying subjects that can access a single object. The data in these two representations are equivalent, the distinction being the ease of use in given situations.

As an alternative, we can use an **access control matrix**, shown in [Figure 2-11](#), a table in which each row represents a subject, each column represents an object, and each entry is the set of access rights for that subject to that object.

		objects		
		File A	Printer	System Clock
subjects	User W	Read Write Own	Write	Read
	Admin		Write Control	Control

FIGURE 2-11 Access Control Matrix

A more detailed example representation of an access control matrix is shown in [Table 2-8](#). Access rights shown in that table are O, own; R, read; W, write; and X, execute. In general, the access control matrix is sparse (meaning that most cells are empty): Most subjects do not have access rights to most objects. The access matrix can be represented as

a list of triples, each having the form $\langle \text{subject, object, rights} \rangle$, as shown in [Table 2-9](#).

	Bibliog	Temp	F	Help .txt	C_ Comp	Linker	Clock	Printer
USER A	ORW	ORW	ORW	R	X	X	R	W
USER B	R	—	—	R	X	X	R	W
USER S	RW	—	R	R	X	X	R	W
USER T	—	—	R	X	X	X	R	W
SYS MGR	—	—	—	RW	OX	OX	ORW	O
USER SVCS	—	—	—	O	X	X	R	W

TABLE 2-8 Access Control Matrix

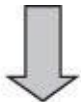
Subject	Object	Right
USER A	Bibliog	ORW
USER B	Bibliog	R
USER S	Bibliog	RW
USER A	Temp	ORW
USER A	F	ORW
USER S	F	R
<i>etc.</i>		

TABLE 2-9 List of Access Control Triples

This representation may be more efficient than the access control matrix because there is no triple for any empty cell of the matrix (such as $\langle \text{USER T, Bibliog, —} \rangle$). Even though the triples can be sorted by subject or object as needed, searching a large number of these triples is inefficient enough that this implementation is seldom used.

Access Control List

An alternative representation is the **access control list**; as shown in [Figure 2-12](#), this representation corresponds to columns of the access control matrix. There is one such list for each object, and the list shows all subjects who should have access to the object and what their access is. This approach differs from the directory list because there is one access control list per object; a directory is created for each subject. Although this difference seems small, there are some significant advantages to this approach.



	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

FIGURE 2-12 Access Control List

The access control list representation can include default rights. Consider subjects A and S, both of whom have access to object F. The operating system will maintain just one access list for F, showing the access rights for A and S, as shown in [Figure 2-13](#). The access control list can include general default entries for any users. In this way, specific users can have explicit rights, and all other users can have a default set of rights. With this organization, all possible users of the system can share a public file or program without the need for an entry for the object in the individual directory of each user.

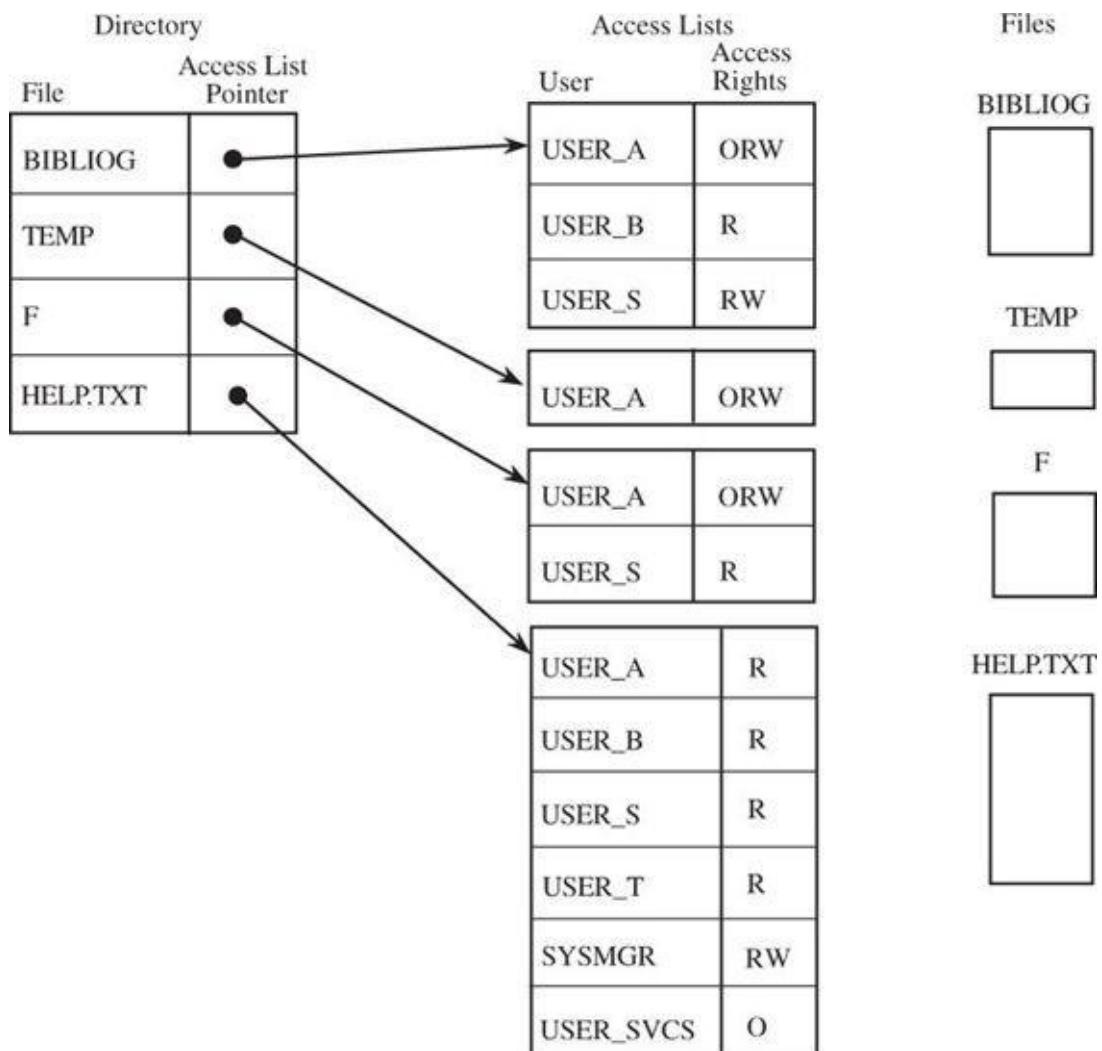


FIGURE 2-13 Access Control List with Two Subjects

The Multics operating system used a form of access control list in which each user belonged to three protection classes: a user, a group, and a compartment. The user designation identified a specific subject, and the group designation brought together subjects who had a common interest, such as their being coworkers on a project. The compartment confined an untrusted object; a program executing in one compartment could not access objects in another compartment without specific permission. The compartment was also a way to collect objects that were related, such as all files for a single project.

To see how this type of protection might work, suppose every user who initiates access to the system identifies a group and a compartment with which to work. If Adams logs in as user Adams in group Decl and compartment Art2, only objects having Adams-Decl-Art2 in the access control list are accessible in the session.

By itself, this kind of mechanism would be too restrictive to be usable. Adams cannot create general files to be used in any session. Worse yet, shared objects would not only have to list Adams as a legitimate subject but also have to list Adams under all acceptable groups and all acceptable compartments for each group.

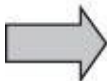
The solution is the use of wild cards, meaning placeholders that designate “any user” (or “any group” or “any compartment”). An access control list might specify access by Adams-Decl-Art1, giving specific rights to Adams if working in group Decl on compartment Art1. The list might also specify Adams-*-Art1, meaning that Adams can access the object from any group in compartment Art1. Likewise, a notation of *-Decl-* would mean “any user in group Decl in any compartment.” Different placements of the wildcard notation * have the obvious interpretations.

Unix uses a similar approach with user-group-world permissions. Every user belongs to a group of related users—students in a common class, workers on a shared project, or members of the same department. The access permissions for each object are a triple (u,g,w) in which u is for the access rights of the user, g is for other members of the group, and w is for all other users in the world.

The access control list can be maintained in sorted order, with * sorted as coming after all specific names. For example, Adams-Decl-* would come after all specific compartment designations for Adams. The search for access permission continues just until the first match. In the protocol, all explicit designations are checked before wild cards in any position, so a specific access right would take precedence over a wildcard right. The last entry on an access list could be *-*-*, specifying rights allowable to any user not explicitly on the access list. With this wildcard device, a shared public object can have a very short access list, explicitly naming the few subjects that should have access rights different from the default.

Privilege List

A **privilege list**, sometimes called a **directory**, is a row of the access matrix, showing all those privileges or access rights for a given subject (shown in [Figure 2-14](#)). One advantage of a privilege list is ease of revocation: If a user is removed from the system, the privilege list shows all objects to which the user has access so that those rights can be removed from the object.



	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

FIGURE 2-14 Privilege Control List

Capability

So far, we have examined protection schemes in which the operating system must keep track of all the protection objects and rights. But other approaches put some of the burden on the user. For example, a user may be required to have a ticket or pass that enables access, much like a ticket or identification card that cannot be duplicated.

More formally, we say that a **capability** is an unforgeable token that gives the possessor certain rights to an object. The Multics [SAL74], CAL [LAM76], and Hydra [WUL74] systems used capabilities for access control. As shown in Figure 2-15, a capability is just one access control triple of a subject, object, and right. In theory, a subject can create new objects and can specify the operations allowed on those objects. For example, users can create objects such as files, data segments, or subprocesses and can also specify the acceptable kinds of operations, such as read, write, and execute. But a user can also create completely new objects, such as new data structures, and can define types of accesses previously unknown to the system.

	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

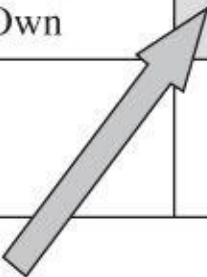


FIGURE 2-15 Capability

Capability: Single- or multi-use ticket to access an object or service

Think of capability as a ticket giving permission to a subject to have a certain type of access to an object. For the capability to offer solid protection, the ticket must be

unforgeable. One way to make it unforgeable is to not give the ticket directly to the user. Instead, the operating system holds all tickets on behalf of the users. The operating system returns to the user a pointer to an operating system data structure, which also links to the user. A capability can be created only by a specific request from a user to the operating system. Each capability also identifies the allowable accesses.

Alternatively, capabilities can be encrypted under a key available only to the access control mechanism. If the encrypted capability contains the identity of its rightful owner, user A cannot copy the capability and give it to user B.

One possible access right to an object is transfer or **propagate**. A subject having this right can pass copies of capabilities to other subjects. In turn, each of these capabilities also has a list of permitted types of accesses, one of which might also be transfer. In this instance, process A can pass a copy of a capability to B, who can then pass a copy to C. B can prevent further distribution of the capability (and therefore prevent further dissemination of the access right) by omitting the transfer right from the rights passed in the capability to C. B might still pass certain access rights to C, but not the right to propagate access rights to other subjects.

As a process executes, it operates in a domain or local name space. The **domain** is the collection of objects to which the process has access. A domain for a user at a given time might include some programs, files, data segments, and I/O devices such as a printer and a terminal. An example of a domain is shown in [Figure 2-16](#).

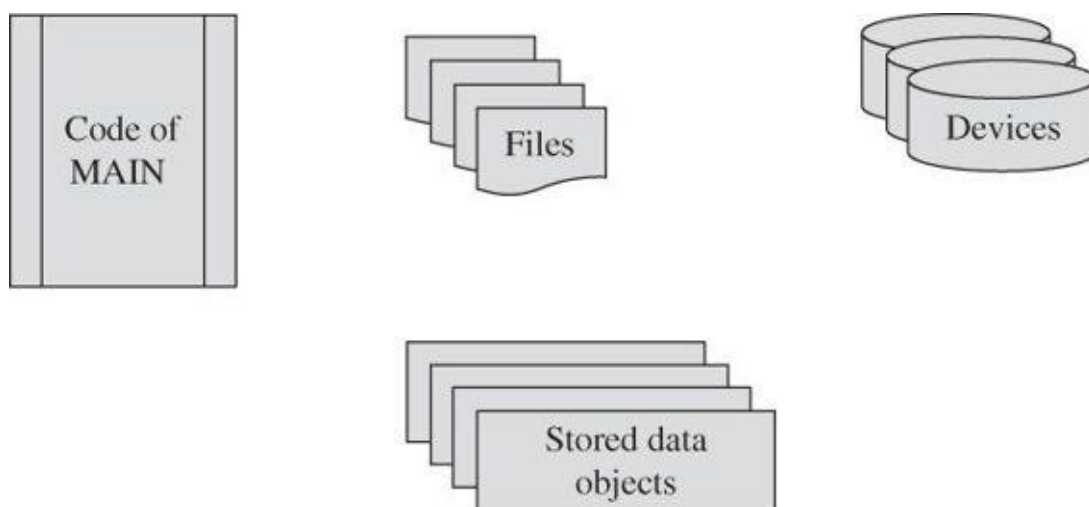


FIGURE 2-16 Example of a Domain

As execution continues, the process may call a subprocedure, passing some of the objects to which it has access as arguments to the subprocedure. The domain of the subprocedure is not necessarily the same as that of its calling procedure; in fact, a calling procedure may pass only some of its objects to the subprocedure, and the subprocedure may have access rights to other objects not accessible to the calling procedure, as shown in [Figure 2-17](#). The caller may also pass only some of its access rights for the objects it passes to the subprocedure. For example, a procedure might pass to a subprocedure the right to read but not to modify a particular data value.

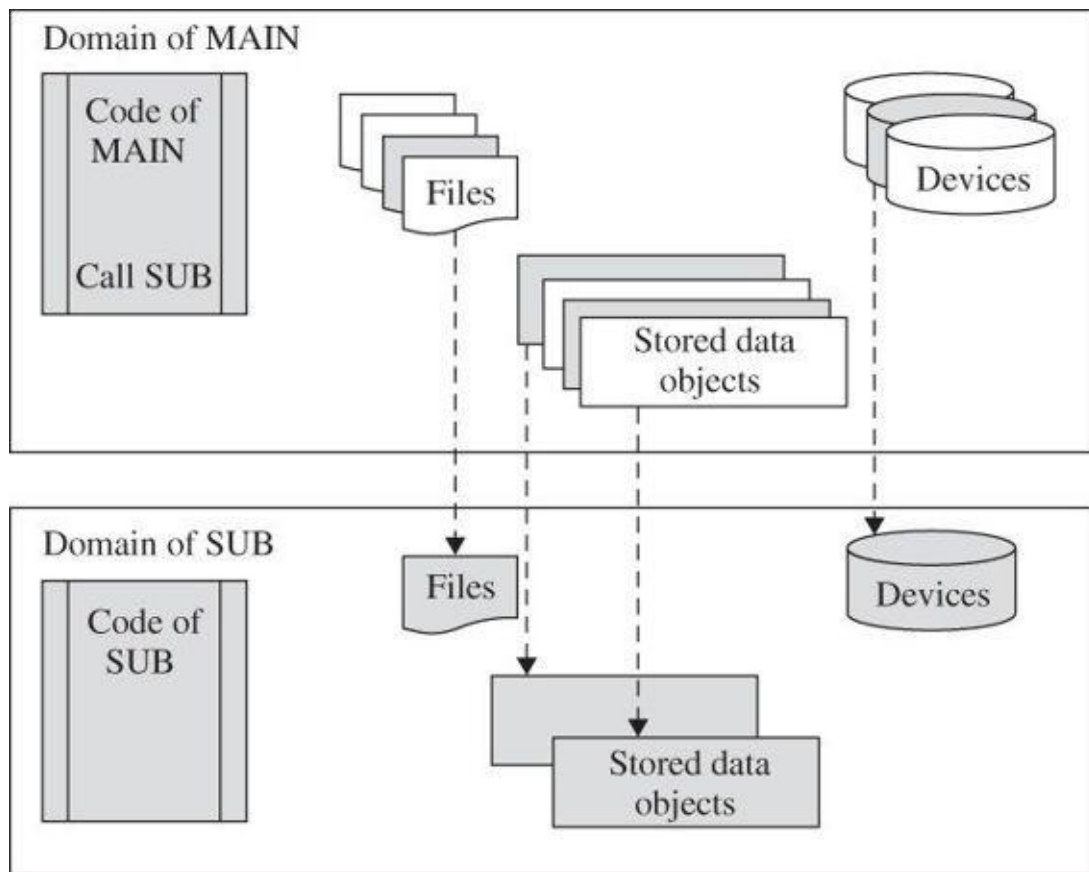


FIGURE 2-17 Passing Objects to a Domain

Because each capability identifies a single object in a domain, the collection of capabilities defines the domain. When a process calls a subprocedure and passes certain objects to the subprocedure, the operating system forms a stack of all the capabilities of the current procedure. The operating system then creates new capabilities for the subprocedure.

Operationally, capabilities are a straightforward way to keep track of the access rights of subjects to objects during execution. The capabilities are backed up by a more comprehensive table, such as an access control matrix or an access control list. Each time a process seeks to use a new object, the operating system examines the master list of objects and subjects to determine whether the object is accessible. If so, the operating system creates a capability for that object.

Capabilities must be stored in memory inaccessible to normal users. One way of accomplishing this is to store capabilities in segments not pointed at by the user's segment table or to enclose them in protected memory as from a pair of base/bounds registers. Another approach is to use a tagged architecture machine to identify capabilities as structures requiring protection.

During execution, only the capabilities of objects that have been accessed by the current process are kept readily available. This restriction improves the speed with which access to an object can be checked. This approach is essentially the one used in Multics, as described in [\[FAB74\]](#).

Capabilities can be revoked. When an issuing subject revokes a capability, no further access under the revoked capability should be permitted. A capability table can contain pointers to the active capabilities spawned under it so that the operating system can trace

what access rights should be deleted if a capability is revoked. A similar problem is deleting capabilities for users who are no longer active.

These three basic structures, the directory, access control matrix and its subsets, and capability, are the basis of access control systems implemented today. Quite apart from the mechanical implementation of the access control matrix or its substructures, two access models relate more specifically to the objective of access control: relating access to a subject's role or the context of the access. We present those models next.

Procedure-Oriented Access Control

One goal of access control is restricting not just what subjects have access to an object, but also what they can *do* to that object. Read versus write access can be controlled rather readily by most applications and operating systems, but more complex control is not so easy to achieve.

By **procedure-oriented** protection, we imply the existence of a procedure that controls access to objects (for example, by performing its own user authentication to strengthen the basic protection provided by the basic operating system). In essence, the procedure forms a capsule around the object, permitting only certain specified accesses.

Procedures can perform actions specific to a particular object in implementing access control.

Procedures can ensure that accesses to an object be made through a trusted interface. For example, neither users nor general operating system routines might be allowed direct access to the table of valid users. Instead, the only accesses allowed might be through three procedures: one to add a user, one to delete a user, and one to check whether a particular name corresponds to a valid user. These procedures, especially add and delete, could use their own checks to make sure that calls to them are legitimate.

Procedure-oriented protection implements the principle of information hiding because the means of implementing an object are known only to the object's control procedure. Of course, this degree of protection carries a penalty of inefficiency. With procedure-oriented protection, there can be no simple, fast access checking, even if the object is frequently used.

Role-Based Access Control

We have not yet distinguished among kinds of users, but we want some users (such as administrators) to have significant privileges, and we want others (such as regular users or guests) to have lower privileges. In companies and educational institutions, this can get complicated when an ordinary user becomes an administrator or a baker moves to the candlestick makers' group. **Role-based access control** lets us associate privileges with groups, such as all administrators can do this or candlestick makers are forbidden to do that. Administering security is easier if we can control access by job demands, not by person. Access control keeps up with a person who changes responsibilities, and the system administrator does not have to choose the appropriate access control settings for someone. For more details on the nuances of role-based access control, see [[FER03](#)].

Access control by role recognizes common needs of all members of a set of subjects.

In conclusion, our study of access control mechanisms has intentionally progressed from simple to complex. Historically, as the mechanisms have provided greater flexibility, they have done so with a price of increased overhead. For example, implementing capabilities that must be checked on each access is far more difficult than implementing a simple directory structure that is checked only on a subject's first access to an object. This complexity is apparent to both the user and implementer. The user is aware of additional protection features, but the naïve user may be frustrated or intimidated at having to select protection options with little understanding of their usefulness. The implementation complexity becomes apparent in slow response to users. The balance between simplicity and functionality is a continuing struggle in security.

2.3 Cryptography

Next we introduce the third of our basic security tools, cryptography. In this chapter we present only the rudiments of the topic, just enough so you can see how it can be used and what it can achieve. We leave the internals for [Chapter 12](#) at the end of this book. We do that because most computer security practitioners would be hard-pressed to explain or implement good cryptography from scratch, which makes our point that you do not need to understand the internals of cryptography just to use it successfully. As you read this chapter you may well ask why something is done in a particular way or how something really works. We invite you to jump to [Chapter 12](#) for the details. But this chapter focuses on the tool and its uses, leaving the internal workings for the future.

Encryption or cryptography—the name means secret writing—is probably the strongest defense in the arsenal of computer security protection. Well-disguised data cannot easily be read, modified, or fabricated. Simply put, encryption is like a machine: you put data in one end, gears spin and lights flash, and you receive modified data out the other end. In fact, some encryption devices used during World War II operated with actual gears and rotors, and these devices were effective at deterring (although not always preventing) the opposite side from reading the protected messages. Now the machinery has been replaced by computer algorithms, but the principle is the same: A transformation makes data difficult for an outsider to interpret.

Cryptography conceals data against unauthorized access.

We begin by examining what encryption does and how it works. We introduce the basic principles of encryption algorithms, introducing two types of encryption with distinct uses. Because weak or flawed encryption creates only the illusion of protection, we also look at how encryption can fail. We briefly describe techniques used to break through the protective cover to void security. Building on these basic types of encryption, we show how to combine them to securely address several general problems of computing and communicating.

Problems Addressed by Encryption

Sometimes we describe encryption in the context of sending secret messages. This framing is just for ease of description: The same concepts apply to protecting a file of data or sensitive information in memory. So when we talk about sending a message, you should also think of protecting any digital object for access only by authorized people.

Consider the steps involved in sending messages from a **sender**, *S*, to a **recipient**, *R*. If *S* entrusts the message to *T*, who then delivers it to *R*, *T* then becomes the **transmission medium**. If an outsider, *O*, wants to access the message (to read, change, or even destroy it), we call *O* an **interceptor** or **intruder**. Any time after *S* transmits the message via *T*, it is vulnerable to exploitation, and *O* might try to access it in any of the following ways:

- *block* it, by preventing its reaching *R*, thereby affecting the availability of the message
- *intercept* it, by reading or listening to the message, thereby affecting the confidentiality of the message
- *modify* it, by seizing the message and changing it in some way, affecting the message's integrity
- *fabricate* an authentic-looking message, arranging for it to be delivered as if it came from *S*, thereby also affecting the integrity of the message

As you can see, a message's vulnerabilities reflect the four possible security failures we identified in [Chapter 1](#). Fortunately, encryption is a technique that can address all these problems. Encryption is a means of maintaining secure data in an insecure environment. In this book, we study encryption as a security technique, and we see how it is used in protecting programs, databases, networks, and electronic communications.

Terminology

Encryption is the process of encoding a message so that its meaning is not obvious; **decryption** is the reverse process, transforming an encrypted message back into its normal, original form. Alternatively, the terms **encode** and **decode** or **encipher** and **decipher** are used instead of encrypt and decrypt.² That is, we say we encode, encrypt, or encipher the original message to hide its meaning. Then, we decode, decrypt, or decipher it to reveal the original message. A system for encryption and decryption is called a **cryptosystem**.

² There are slight differences in the meanings of these three pairs of words, although they are not significant in the context of this discussion. Strictly speaking, **encoding** is the process of translating entire words or phrases to other words or phrases, whereas **enciphering** is translating letters or symbols individually; **encryption** is the group term that covers both encoding and enciphering.

The original form of a message is known as **plaintext**, and the encrypted form is called **ciphertext**. This relationship is shown in [Figure 2-18](#). Think of encryption as a form of opaque paint that obscures or obliterates the plaintext, preventing it from being seen or interpreted accurately. Then, decryption is the process of peeling away the paint to reveal the original plaintext again.

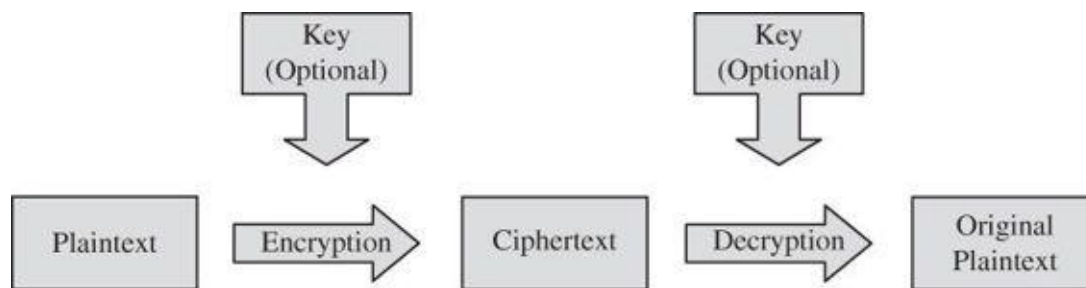


FIGURE 2-18 Plaintext and Ciphertext

Ciphertext: encrypted material; plaintext: material in intelligible form

We use a formal notation to describe the transformations between plaintext and ciphertext. For example, we write $C = E(P)$ and $P = D(C)$, where C represents the ciphertext, E is the encryption rule, P is the plaintext, and D is the decryption rule. What we seek is a cryptosystem for which $P = D(E(P))$. In other words, we want to be able to convert the plaintext message to ciphertext to protect it from an intruder, but we also want to be able to get the original message back so that the receiver can read it properly.

Encryption Keys

A cryptosystem involves a set of rules for how to encrypt the plaintext and decrypt the ciphertext. The encryption and decryption rules, called **algorithms**, often use a device called a **key**, denoted by K , so that the resulting ciphertext depends on the original plaintext message, the algorithm, and the key value. We write this dependence as $C = E(K, P)$. Essentially, E is a set of encryption algorithms, and the key K selects one specific algorithm from the set.

This process is similar to using mass-produced locks in houses. As a homeowner, you would pay dearly to contract with someone to invent and make a lock just for your house. In addition, you would not know whether a particular inventor's lock was really solid or how it compared with those of other inventors. A better solution is to have a few well-known, well-respected companies producing standard locks that differ according to the (physical) key. Then, you and your neighbor might have the same brand and style of lock, but your key will open only your lock. In the same way, it is useful to have a few well-examined encryption algorithms for everyone to use, but differing keys would prevent someone from breaking into data you are trying to protect.

Sometimes the encryption and decryption keys are the same, so $P = D(K, E(K, P))$, meaning that the same key, K , is used both to encrypt a message and later to decrypt it. This form is called **symmetric** or **single-key** or **secret** key encryption because D and E are mirror-image processes. As a trivial example, the encryption algorithm might be to shift each plaintext letter forward n positions in the alphabet. For $n = 1$, A is changed to b, B to c, ... P to q, ... and Z to a, so we say the key value is n , moving n positions forward for encryption and backward for decryption. (You might notice that we have written the plaintext in uppercase letters and the corresponding ciphertext in lowercase; cryptographers sometimes use that convention to help them distinguish the two.)

Symmetric encryption: one key encrypts and decrypts.

At other times, encryption and decryption keys come in pairs. Then, a decryption key, K_D , inverts the encryption of key K_E , so that $P = D(K_D, E(K_E, P))$. Encryption algorithms of this form are called **asymmetric** or **public** key because converting C back to P involves a series of steps and a key that are different from the steps and key of E . The difference between symmetric and asymmetric encryption is shown in [Figure 2-19](#).

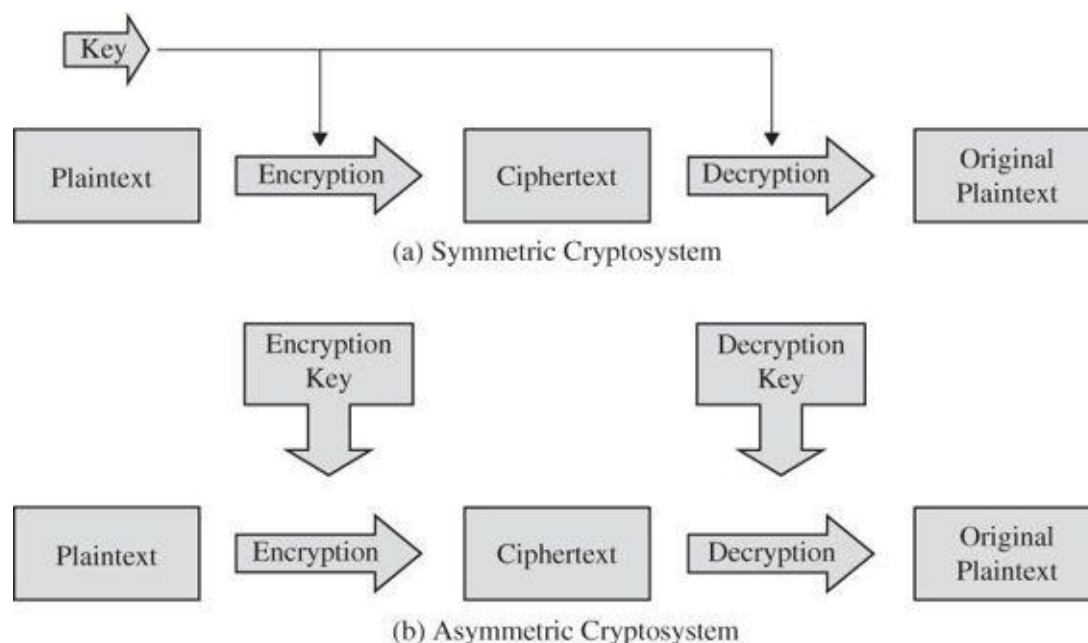


FIGURE 2-19 Symmetric and Asymmetric Encryption

Asymmetric encryption: one key encrypts, a different key decrypts.

A key gives us flexibility in using an encryption scheme. We can create different encryptions of one plaintext message just by changing the key. Moreover, using a key provides additional security. If the encryption algorithm should fall into the interceptor's hands, future messages can still be kept secret because the interceptor will not know the key value. [Sidebar 2-14](#) describes how the British dealt with written keys and codes in World War II. An encryption scheme that does not require the use of a key is called a **keyless cipher**.

Sidebar 2-14 Silken Codes

Leo Marks [[MAR98](#)] describes his life as a code-maker in Britain during World War II. That is, the British hired Marks and others to devise codes that could be used by spies and soldiers in the field. In the early days, the encryption scheme depended on poems that were written for each spy, and it relied on the spy's ability to memorize and recall the poems correctly.

Marks reduced the risk of error by introducing a coding scheme that was printed on pieces of silk. Silk hidden under clothing could not be felt when the spy was patted down and searched. And, unlike paper, silk burns quickly and completely, so the spy could destroy incriminating evidence, also ensuring that the enemy could not get even fragments of the valuable code. When pressed by superiors as to why the British should use scarce silk (which was also needed for war-time necessities like parachutes) for codes, Marks said that it was a choice

“between silk and cyanide.”

The history of encryption is fascinating; it is well documented in David Kahn’s book [KAH96]. Claude Shannon is considered the father of modern cryptography because he laid out a formal, mathematical foundation for information security and expounded on several principles for secure cryptography at the naissance of digital computing [SHA49]. Encryption has been used for centuries to protect diplomatic and military communications, sometimes without full success. The word **cryptography** refers to the practice of using encryption to conceal text. A **cryptanalyst** studies encryption and encrypted messages, hoping to find the hidden meanings. A cryptanalyst might also work defensively, probing codes and ciphers to see if they are solid enough to protect data adequately.

Both a **cryptographer** and a cryptanalyst attempt to translate coded material back to its original form. Normally, a cryptographer works on behalf of a legitimate sender or receiver, whereas a cryptanalyst works on behalf of an unauthorized interceptor. Finally, **cryptology** is the research into and study of encryption and decryption; it includes both cryptography and cryptanalysis.

Cryptanalysis

A cryptanalyst’s chore is to **break** an encryption. That is, the cryptanalyst attempts to deduce the original meaning of a ciphertext message. Better yet, the cryptanalyst hopes to determine which decrypting algorithm, and ideally which key, matches the encrypting algorithm to be able to break other messages encoded in the same way.

For instance, suppose two countries are at war and the first country has intercepted encrypted messages of the second. Cryptanalysts of the first country want to decipher a particular message so as to anticipate the movements and resources of the second. But even better is to discover the actual decryption method; then the first country can penetrate the encryption of *all* messages sent by the second country.

An analyst works with a variety of information: encrypted messages, known encryption algorithms, intercepted plaintext, data items known or suspected to be in a ciphertext message, mathematical or statistical tools and techniques, and properties of languages, as well as plenty of ingenuity and luck. Each piece of evidence can provide a clue, and the analyst puts the clues together to try to form a larger picture of a message’s meaning in the context of how the encryption is done. Remember that there are no rules. An interceptor can use any means available to tease out the meaning of the message.

Work Factor

An encryption algorithm is called **breakable** when, given enough time and data, an analyst can determine the algorithm. However, an algorithm that is theoretically breakable may in fact be impractical to try to break. To see why, consider a 25-character message that is expressed in just uppercase letters. A given cipher scheme may have 26^{25} (approximately 10^{35}) possible decipherments, so the task is to select the right one out of the 26^{25} . If your computer could perform on the order of 10^{10} operations per second, finding this decipherment would require on the order of 10^{25} seconds, or roughly 10^{17} years. In this case, although we know that theoretically we could generate the solution,

determining the deciphering algorithm by examining all possibilities can be ignored as infeasible with current technology.

The difficulty of breaking an encryption is called its **work factor**. Again, an analogy to physical locks may prove helpful. As you know, physical keys have notches or other irregularities, and the notches cause pins to move inside a lock, allowing the lock to open. Some simple locks, such as those sold with suitcases, have only one notch, so these locks can often be opened with just a piece of bent wire; worse yet, some manufacturers produce only a few (and sometimes just one!) distinct internal pin designs; you might be able to open any such lock with a ring of just a few keys. Clearly these locks are cosmetic only.

Common house locks have five or six notches, and each notch can have any of ten depths. To open such a lock requires finding the right combination of notch depths, of which there may be up to a million possibilities, so carrying a ring of that many keys is infeasible. Even though in theory someone could open one of these locks by trying all possible keys, in practice the number of possibilities is prohibitive. We say that the work factor to open one of these locks without the appropriate key is large enough to deter most attacks. So too with cryptography: An encryption is adequate if the work to decrypt without knowing the encryption key is greater than the value of the encrypted data.

Work factor: amount of effort needed to break an encryption (or mount a successful attack)

Two other important issues must be addressed when considering the breakability of encryption algorithms. First, the cryptanalyst cannot be expected to try only the hard, long way. In the example just presented, the obvious decryption might require 26^{25} machine operations, but a more ingenious approach might require only 10^{15} operations. At the speed of 10^{10} operations per second, 10^{15} operations take slightly more than one day. The ingenious approach is certainly feasible. In fact, newspapers sometimes print cryptogram puzzles that humans solve with pen and paper alone, so there is clearly a shortcut to our computer machine time estimate of years or even one day of effort. The newspaper games give hints about word lengths and repeated characters, so humans are solving an easier problem. As we said, however, cryptanalysts also use every piece of information at their disposal.

Some of the algorithms we study in this book are based on known “hard” problems that take an unreasonably long time to solve. But the cryptanalyst does not necessarily have to solve the underlying problem to break the encryption of a single message. Sloppy use of controls can reveal likely words or phrases, and an analyst can use educated guesses combined with careful analysis to generate all or much of an important message. Or the cryptanalyst might employ a spy to obtain the plaintext entirely outside the system; analysts might then use the pair of plaintext and corresponding ciphertext to infer the algorithm or key used to apply to subsequent messages.

In cryptanalysis there are no rules: Any action is fair play.

Second, estimates of breakability are based on current technology. An enormous

advance in computing technology has occurred since 1950. Things that were infeasible in 1940 became possible by the 1950s, and every succeeding decade has brought greater improvements. A conjecture known as “Moore’s Law” asserts that the speed of processors doubles every 1.5 years, and this conjecture has been true for over three decades. We dare not pronounce an algorithm secure just because it cannot be broken with *current* technology, or worse, that it has not been broken yet.

In this book we write that something is impossible; for example, it is impossible to obtain plaintext from ciphertext without the corresponding key and algorithm. Please understand that in cryptography few things are truly impossible: infeasible or prohibitively difficult, perhaps, but impossible, no.

Symmetric and Asymmetric Encryption Systems

Recall that the two basic kinds of encryptions are symmetric (also called “secret key”) and asymmetric (also called “public key”). Symmetric algorithms use one key, which works for both encryption and decryption. Usually, the decryption algorithm is closely related to the encryption one, essentially running the encryption in reverse.

The symmetric systems provide a two-way channel to their users: A and B share a secret key, and they can both encrypt information to send to the other as well as decrypt information from the other. As long as the key remains secret, the system also provides **authenticity**, proof that a message received was not fabricated by someone other than the declared sender.³ Authenticity is ensured because only the legitimate sender can produce a message that will decrypt properly with the shared key.

³. This being a security book, we point out that the proof is actually that the message was sent by someone who had or could simulate the effect of the sender’s key. With many security threats there is a small, but non-zero, risk that the message is not actually from the sender but is a complex forgery.

Symmetry is a major advantage of this type of encryption, but it also leads to a problem: How do two users A and B obtain their shared secret key? And only A and B can use that key for their encrypted communications. If A wants to share encrypted communication with another user C, A and C need a different shared secret key. Managing keys is the major difficulty in using symmetric encryption. In general, n users who want to communicate in pairs need $n * (n - 1)/2$ keys. In other words, the number of keys needed increases at a rate proportional to the *square* of the number of users! So a property of symmetric encryption systems is that they require a means of **key distribution**.

Asymmetric or public key systems, on the other hand, typically have precisely matched pairs of keys. The keys are produced together or one is derived mathematically from the other. Thus, a process computes both keys as a set.

But for both kinds of encryption, a key must be kept well secured. Once the symmetric or private key is known by an outsider, all messages written previously or in the future can be decrypted (and hence read or modified) by the outsider. So, for all encryption algorithms, **key management** is a major issue. It involves storing, safeguarding, and activating keys.

Asymmetric systems excel at key management. By the nature of the public key approach, you can send a public key in an email message or post it in a public directory. Only the corresponding private key, which presumably is not disclosed, can decrypt what

has been encrypted with the public key.

Stream and Block Ciphers

One final characterization of encryption algorithms relates to the nature of the data to be concealed. Suppose you are streaming video, perhaps a movie, from a satellite. The stream may come in bursts, depending on such things as the load on the satellite and the speed at which the sender and receiver can operate. For such application you may use what is called **stream encryption**, in which each bit, or perhaps each byte, of the data stream is encrypted separately. A model of stream enciphering is shown in [Figure 2-20](#). Notice that the input symbols are transformed one at a time. The advantage of a stream cipher is that it can be applied immediately to whatever data items are ready to transmit. But most encryption algorithms involve complex transformations; to do these transformations on one or a few bits at a time is expensive.

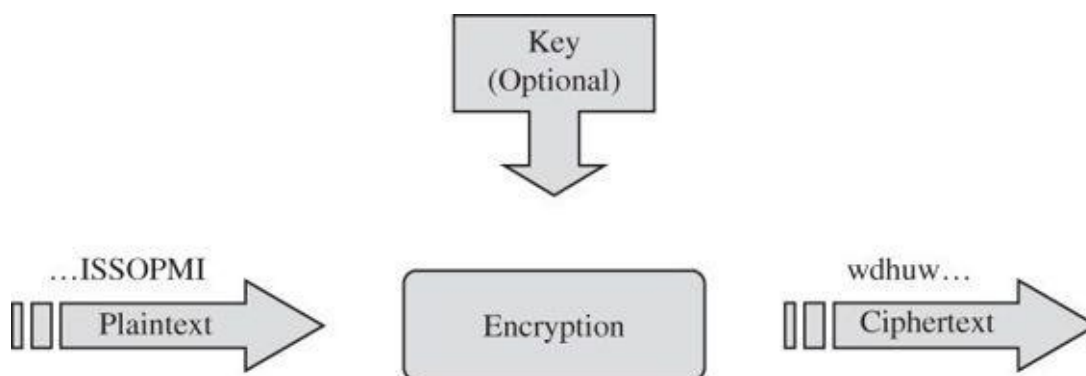


FIGURE 2-20 Stream Enciphering

To address this problem and make it harder for a cryptanalyst to break the code, we can use block ciphers. A **block cipher** encrypts a group of plaintext symbols as a single block. A block cipher algorithm performs its work on a quantity of plaintext data all at once. Like a machine that cuts out 24 cookies at a time, these algorithms capitalize on economies of scale by operating on large amounts of data at once. Blocks for such algorithms are typically 64, 128, 256 bits or more. The block size need not have any particular relationship to the size of a character. Block ciphers work on blocks of plaintext and produce blocks of ciphertext, as shown in [Figure 2-21](#). In the figure, the central box represents an encryption machine: The previous plaintext pair is converted to po, the current one being converted is IH, and the machine is soon to convert ES.

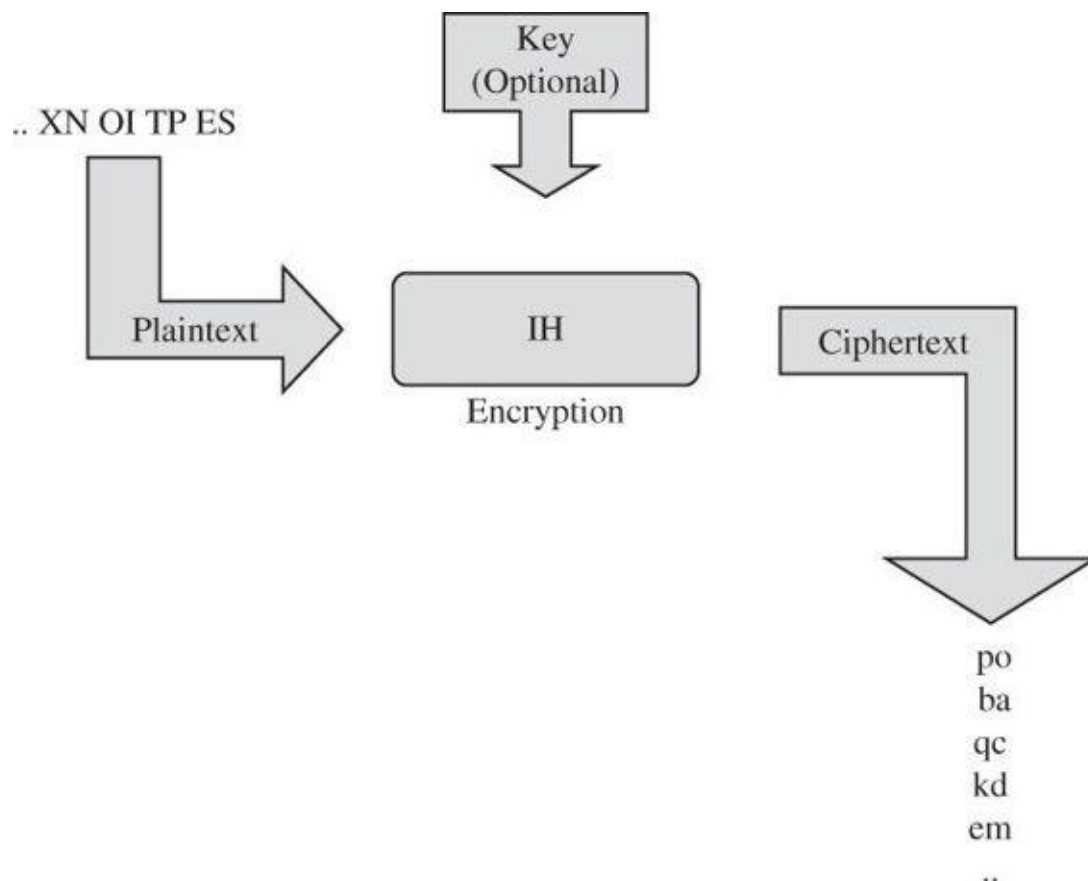


FIGURE 2-21 Block Cipher

Stream ciphers encrypt one bit or one byte at a time; block ciphers encrypt a fixed number of bits as a single chunk.

[Table 2-10](#) lists the advantages and disadvantages of stream and block encryption algorithms.

	Stream	Block
Advantages	• <i>Speed of transformation.</i> Because each symbol is encrypted without regard for any other plaintext symbols, each symbol can be encrypted as soon as it is read. Thus, the time to encrypt a symbol depends only on the encryption algorithm itself, not on the time it takes to receive more plaintext. • <i>Low error propagation.</i> Because each symbol is separately encoded, an error in the encryption process affects only that character.	• <i>High diffusion.</i> Information from the plaintext is diffused into several ciphertext symbols. One ciphertext block may depend on several plaintext letters. • <i>Immunity to insertion of symbol.</i> Because blocks of symbols are enciphered, it is impossible to insert a single symbol into one block. The length of the block would then be incorrect, and the decipherment would quickly reveal the insertion.
Disadvantages	• <i>Low diffusion.</i> Each symbol is separately enciphered. Therefore, all the information of that symbol is contained in one symbol of ciphertext. • <i>Susceptibility to malicious insertions and modifications.</i> Because each symbol is separately enciphered, an active interceptor who has broken the code can splice pieces of previous messages and transmit a spurious new message that may look authentic.	• <i>Slowness of encryption.</i> The person or machine doing the block ciphering must wait until an entire block of plaintext symbols has been received before starting the encryption process. • <i>Padding.</i> A final short block must be filled with irrelevant data to make a full-sized block. • <i>Error propagation.</i> An error will affect the transformation of all other characters in the same block.

TABLE 2-10 Stream and Block Encryption Algorithms

With this description of the characteristics of different encryption algorithms we can now turn to some widely used encryption algorithms. We present how each works, a bit of the historical context and motivation for each, and some strengths and weaknesses. We identify these algorithms by name because these names appear in the popular literature. We also introduce other symmetric algorithms in [Chapter 12](#). Of course you should recognize that these are just examples of popular algorithms; over time these algorithms may be superseded by others. To a large degree cryptography has become plug-and-play, meaning that in an application developers can substitute one algorithm for another of the same type and similar characteristics. In that way advancements in the field of cryptography do not require that all applications using cryptography be rewritten.

DES: The Data Encryption Standard

The Data Encryption Standard (DES) [[NBS77](#)], a system developed for the U.S. government, was intended for use by the general public. Standards organizations have officially accepted it as a cryptographic standard both in the United States and abroad. Moreover, many hardware and software systems have been designed with DES. For many years it was the algorithm of choice for protecting financial, personal, and corporate data; however, researchers increasingly questioned its adequacy as it aged.

Overview of the DES Algorithm

The DES algorithm [[NBS77](#)] was developed in the 1970s by IBM for the U.S. National Institute of Standards and Technology (NIST), then called the National Bureau of Standards (NBS). DES is a careful and complex combination of two fundamental building blocks of encryption: substitution and transposition. The algorithm derives its strength from repeated application of these two techniques, one on top of the other, for a total of 16 cycles. The sheer complexity of tracing a single bit through 16 iterations of substitutions and transpositions has so far stopped researchers in the public from identifying more than a handful of general properties of the algorithm.

The algorithm begins by encrypting the plaintext as blocks of 64 bits. The key is 64 bits long, but in fact it can be any 56-bit number. (The extra 8 bits are often used as check digits but do not affect encryption in normal implementations. Thus we say that DES uses a key, the strength of which is 56 bits.) The user can pick a new key at will any time there is uncertainty about the security of the old key.

DES encrypts 64-bit blocks by using a 56-bit key.

DES uses only standard arithmetic and logical operations on binary data up to 64 bits long, so it is suitable for implementation in software on most current computers. Encrypting with DES involves 16 iterations, each employing replacing blocks of bits (called a substitution step), shuffling the bits (called a permutation step), and mingling in bits from the key (called a key transformation). Although complex, the process is table driven and repetitive, making it suitable for implementation on a single-purpose chip. In fact, several such chips are available on the market for use as basic components in devices that use DES encryption in an application.

Double and Triple DES

As you know, computing power has increased rapidly over the last few decades, and it promises to continue to do so. For this reason, the DES 56-bit key length is not long enough for some people's comfort. Since the 1970s, researchers and practitioners have been interested in a longer-key version of DES. But we have a problem: The DES algorithm design is fixed to a 56-bit key.

Double DES

To address the discomfort, some researchers suggest using a double encryption for greater secrecy. The double encryption works in the following way. Take two keys, k_1 and k_2 , and perform two encryptions, one on top of the other: $E(k_2, E(k_1, m))$. In theory, this approach should multiply the difficulty of breaking the encryption, just as two locks are harder to pick than one.

Unfortunately, that assumption is false. Ralph Merkle and Martin Hellman [[MER81](#)] showed that two encryptions are scarcely better than one: two encryptions with different 56-bit keys are equivalent in work factor to one encryption with a 57-bit key. Thus, the double encryption adds only a small amount of extra work for the attacker who is trying to infer the key(s) under which a piece of ciphertext was encrypted. As we soon describe, some 56-bit DES keys have been derived in just days; two times days is still days, when the hope was to add months if not years of work for the second encryption. Alas, double DES adds essentially no more security.

Triple DES

However, a simple trick does indeed enhance the security of DES. Using three keys adds significant strength.

The so-called **triple DES** procedure is $C = E(k_3, E(k_2, E(k_1, m)))$. That is, you encrypt with one key, then with the second, and finally with a third. This process gives a strength roughly equivalent to a 112-bit key (because the double DES attack defeats the strength of one of the three keys, but it has no effect on the third key).

A minor variation of triple DES, which some people also confusingly call triple DES, is $C = E(k_1, D(k_2, E(k_1, m)))$. That is, you encrypt with one key, decrypt with a second, and encrypt with the first again. This version requires only two keys. (The second decrypt step also makes this process work for single encryptions with one key: The decryption cancels the first encryption, so the net result is one encryption. The encrypt–decrypt–encrypt form is handy because one algorithm can produce results for both conventional single-key DES and the more secure two-key method.) This two-key, three-step version is subject to another tricky attack, so its strength is rated at only about 80 bits. Still, 80 bits is beyond reasonable cracking capability for current hardware.

In summary, ordinary DES has a key space of 56 bits, double DES is scarcely better, but two-key triple DES gives an effective length of 80 bits, and three-key triple DES gives a strength of 112 bits. Remember why we are so fixated on key size: If no other way succeeds, the attacker can always try all possible keys. A longer key means significantly more work for this attack to bear fruit, with the work factor doubling for each additional

bit in key length. Now, roughly a half century after DES was created, a 56-bit key is inadequate for any serious confidentiality, but 80- and 112-bit effective key sizes afford reasonable security. We summarize these forms of DES in [Table 2-11](#).

Form	Operation	Properties	Strength
DES	Encrypt with one key	56-bit key	Inadequate for high-security applications by today's computing capabilities
Double DES	Encrypt with first key; then encrypt result with second key	Two 56-bit keys	Only doubles strength of 56-bit key version
Two-key triple DES	Encrypt with first key, then encrypt (or decrypt) result with second key, then encrypt result with first key (E-D-E)	Two 56-bit keys	Gives strength equivalent to about 80-bit key (about 16 million times as strong as 56-bit version)
Three-key triple DES	Encrypt with first key, then encrypt or decrypt result with second key, then encrypt result with third key (E-E-E)	Three 56-bit keys	Gives strength equivalent to about 112-bit key about 72 quintillion (72×10^{15}) times as strong as 56-bit version

TABLE 2-11 Forms of DES

Security of DES

Since it was first announced, DES has been controversial. Many researchers have questioned the security it provides. Because of its association with the U.S. government, specifically the U.S. National Security Agency (NSA) that made certain unexplained changes between what IBM proposed and what the NBS actually published, some people have suspected that the algorithm was somehow weakened, to allow the government to snoop on encrypted data. Much of this controversy has appeared in the open literature, but certain DES features have neither been revealed by the designers nor inferred by outside analysts.

Whitfield Diffie and Martin Hellman [[DIF77](#)] argued in 1977 that a 56-bit key is too short. In 1977, it was prohibitive to test all 256 (approximately 10^{15}) keys on then current computers. But they argued that over time, computers would become more powerful and the DES algorithm would remain unchanged; eventually, the speed of computers would exceed the strength of DES. Exactly that happened about 20 years later. In 1997, researchers using a network of over 3,500 machines in parallel were able to infer a DES key in four months' work. And in 1998 for approximately \$200,000 U.S. researchers built a special "DES cracker" machine that could find a DES key in approximately four days, a result later improved to a few hours [[EFF98](#)].

Does this mean DES is insecure? No, not exactly. No one has yet shown serious flaws in the DES algorithm itself. The 1997 attack required a great deal of cooperation, and the 1998 machine is rather expensive. But even if conventional DES can be attacked, triple DES is still well beyond the power of these attacks. Remember the impact of exponential growth: Let us say, for simplicity, that single-key DES can be broken in one hour. The simple double-key version could then be broken in two hours. But $2^{80}/2^{56} = 2^{24}$, which is over 16,700,000, meaning it would take 16 million hours, nearly 2,000 years, to defeat a two-key triple DES encryption, and considerably longer for the three-key version.

Nevertheless, the basic structure of DES with its fixed-length 56-bit key and fixed number of iterations makes evident the need for a new, stronger, and more flexible algorithm. In 1995, the NIST began the search for a new, strong encryption algorithm. The response to that search has become the Advanced Encryption Standard, or AES.

AES: Advanced Encryption System

After a public competition and review, NIST selected an algorithm named Rijndael as the new advanced encryption system; Rijndael is now known more widely as AES. AES was adopted for use by the U.S. government in December 2001 and became Federal Information Processing Standard 197 [[NIS01](#)]. AES is likely to be the commercial-grade symmetric algorithm of choice for years, if not decades. Let us look at it more closely.

Overview of Rijndael

Rijndael is a fast algorithm that can easily be implemented on simple processors. Although it has a strong mathematical foundation, it primarily uses substitution, transposition, the shift, exclusive OR, and addition operations. Like DES, AES uses repeat cycles.

There are 10, 12, or 14 cycles for keys of 128, 192, and 256 bits, respectively. In Rijndael, the cycles are called “rounds.” Each round consists of four steps that substitute and scramble bits. Bits from the key are frequently combined with intermediate result bits, so key bits are also well diffused throughout the result. Furthermore, these four steps are extremely fast. The AES algorithm is depicted in [Figure 2-22](#).

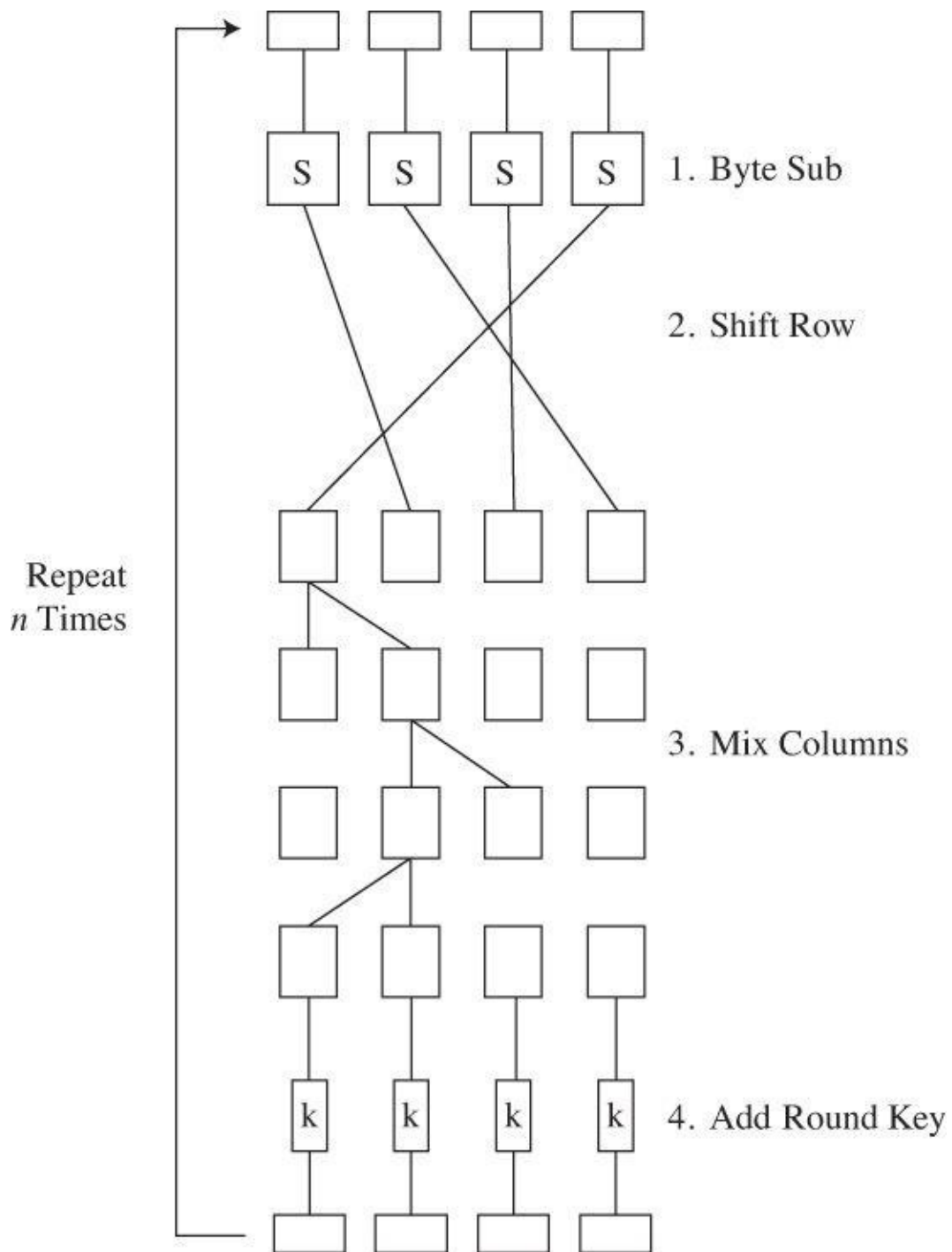


FIGURE 2-22 AES Encryption Algorithm

Strength of the Algorithm

The characteristics and apparent strength of DES and AES are compared in [Table 2-12](#). Remember, of course, that these strength figures apply only if the implementation and use are robust; a strong algorithm loses strength if used with a weakness that lets outsiders determine key properties of the encrypted data.

	DES	AES
Date designed	1976	1999
Block size	64 bits	128 bits
Key length	56 bits (effective length); up to 112 bits with multiple keys	128, 192, 256 (and possibly more) bits
Operations	16 rounds	10, 12, 14 (depending on key length); can be increased
Encryption primitives	Substitution, permutation	Substitution, shift, bit mixing
Cryptographic primitives	Confusion, diffusion	Confusion, diffusion
Design	Open	Open
Design rationale	Closed	Open
Selection process	Secret	Secret, but open public comments and criticisms invited
Source	IBM, enhanced by NSA	Independent Dutch cryptographers

TABLE 2-12 Comparison of DES and AES

Moreover, the number of cycles can be extended in a natural way. With DES the algorithm was defined for precisely 16 cycles; to extend that number would require substantial redefinition of the algorithm. The internal structure of AES has no a priori limitation on the number of cycles. If a cryptanalyst ever concluded that 10 or 12 or 14 rounds were too low, the only change needed to improve the algorithm would be to change the limit on a repeat loop.

A mark of confidence is that the U.S. government has approved AES for protecting Secret and Top Secret classified documents. This is the first time the United States has ever approved the use of a commercial algorithm derived outside the government (and furthermore, outside the United States) to encrypt classified data.

However, we cannot rest on our laurels. No one can predict now what limitations cryptanalysts might identify in the future. Fortunately, talented cryptologists continue to investigate even stronger algorithms that will be able to replace AES when it becomes obsolete. At present, AES seems to be a significant improvement over DES, and it can be improved in a natural way if necessary. DES is still in widespread use, but AES is also widely adopted, particularly for new applications.

Public Key Cryptography

So far, we have looked at encryption algorithms from the point of view of making the scrambling easy for the sender (so that encryption is fast and simple) and the decryption easy for the receiver but not for an intruder. But this functional view of transforming plaintext to ciphertext is only part of the picture. We must also figure out how to distribute encryption keys. We have noted how useful keys can be in deterring an intruder, but the key must remain secret for it to be effective. The encryption algorithms we have presented so far are called **symmetric** or **secret-key** algorithms. The two most widely used symmetric algorithms, DES and AES, operate similarly: Two users have copies of the

same key. One user uses the algorithm to encrypt some plaintext under the key, and the other user uses an inverse of the algorithm with the same key to decrypt the ciphertext. The crux of this issue is that all the power of the encryption depends on the secrecy of the key.

In 1976, Whitfield Diffie and Martin Hellman [[DIF76](#)] invented public key cryptography, a new kind of encryption. With a public key encryption system, each user has two keys, one of which does not have to be kept secret. Although counterintuitive, in fact the public nature of the key does not compromise the secrecy of the system. Instead, the basis for public key encryption is to allow the key to be divulged but to keep the decryption technique secret. Public key cryptosystems accomplish this goal by using two keys: one to encrypt and the other to decrypt. Although these keys are produced in mathematically related pairs, an outsider is effectively unable to use one key to derive the other.

In this section, we look at ways to allow the key to be public but still protect the message. We also focus on the RSA algorithm, a popular, commercial-grade public key system. Other algorithms, such as elliptic curve cryptosystems [[MIL85](#), [KOB87](#)] and the El Gamal algorithm [[ELG85](#)], both of which we cover in [Chapter 12](#), operate similarly (although the underlying mathematics are very different). We concentrate on RSA because many applications use it. We also present a mathematical scheme by which two users can jointly construct a secret encryption key without having any prior secrets.

Motivation

Why should making the key public be desirable? With a conventional symmetric key system, each pair of users needs a separate key. But with public key systems, anyone using a single public key can send a secret message to a user, and the message remains adequately protected from being read by an interceptor. Let us investigate why this is so.

Recall that in general, an n -user system requires $n * (n - 1)/2$ keys, and each user must track and remember a key for each other user with whom he or she wants to communicate. As the number of users grows, the number of keys increases rapidly, as shown in [Figure 2-23](#). Determining and distributing these keys is a problem. A more serious problem is maintaining security for the keys already distributed—we cannot expect users to memorize so many keys. Worse, loss or exposure of one user's keys requires setting up a new key pair with each of that user's correspondents.

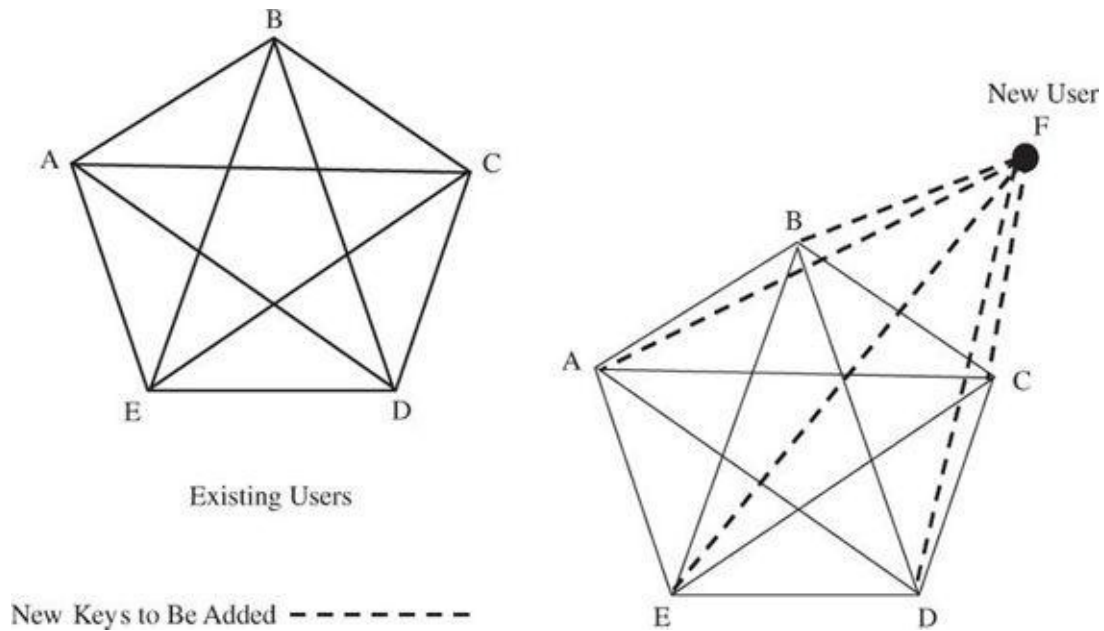


FIGURE 2-23 Explosion in Number of Keys

Characteristics

We can reduce the problem of key proliferation by using a public key approach. In a **public key** or **asymmetric encryption system**, each user has two keys: a **public key** and a **private key**. The user may freely publish the public key because each key does only encryption or decryption, but not both. The keys operate as inverses, meaning that one key undoes the encryption provided by the other key. But deducing one key from the other is effectively impossible.

To see how, let k_{PRIV} be a user's private key, and let k_{PUB} be the corresponding public key. Then, encrypted plaintext using the public key is decrypted by application of the private key; we write the relationship as

$$P = D(k_{PRIV}, E(k_{PUB}, P))$$

That is, a user can decode with a private key what someone else has encrypted with the corresponding public key. Furthermore, with some public key encryption algorithms, including RSA, we have this relationship:

$$P = D(k_{PUB}, E(k_{PRIV}, P))$$

In other words, a user can encrypt a message with a private key, and the message can be revealed only with the corresponding public key.

These two properties tell us that public and private keys can be applied in either order. In particular, the decryption function D can be applied to any argument so that we can decrypt before we encrypt. With conventional encryption, we seldom think of decrypting *before* encrypting. But the concept makes sense with public keys, where it simply means applying the private transformation first and then the public one.

We have noted that a major problem with symmetric encryption is the sheer number of keys a single user has to store and track. With public keys, only two keys are needed per user: one public and one private. Let us see what difference this makes in the number of keys needed. Suppose we have three users, B, C, and D, who must pass protected

messages to user A as well as to each other. Since each distinct pair of users needs a key, each user would need three different keys; for instance, A would need a key for B, a key for C, and a key for D. But using public key encryption, each of B, C, and D can encrypt messages for A by using A's public key. If B has encrypted a message using A's public key, C *cannot* decrypt it, even if C knew it was encrypted with A's public key. Applying A's public key twice, for example, would not decrypt the message. (We assume, of course, that A's private key remains secret.) Thus, the number of keys needed in the public key system is only two per user.

The Rivest–Shamir–Adelman (RSA) Algorithm

The **Rivest–Shamir–Adelman (RSA) cryptosystem** is a public key system. Based on an underlying hard problem and named after its three inventors (Ronald Rivest, Adi Shamir, and Leonard Adleman), this algorithm was introduced in 1978 [[RIV78](#)]. Cryptanalysts have subjected RSA to extensive cryptanalysis, but they have found no serious flaws.

The two keys used in RSA, d and e , are used for decryption and encryption. They are actually interchangeable: Either can be chosen as the public key, but one having been chosen, the other one must be kept private. For simplicity, we call the encryption key e and the decryption key d . We denote plaintext as P and its corresponding ciphertext as C . $C = \text{RSA}(P, e)$. Also, because of the nature of the RSA algorithm, the keys can be applied in either order:

$$P = E(D(P)) = D(E(P))$$

or

$$P = \text{RSA}(\text{RSA}(P, e), d) = \text{RSA}(\text{RSA}(P, d), e)$$

(You can think of E and D as two complementary functions, each of which can “undo” the other's effect.)

RSA does have the unfortunate property that the keys are long: 256 bits is considered the minimum usable length, but in most contexts experts prefer keys on the order of 1000 to 2000 bits. Encryption in RSA is done by exponentiation, raising each plaintext block to a power; that power is the key e . In contrast to fast substitution and transposition of symmetric algorithms, exponentiation is extremely time-consuming on a computer. Even worse, the time to encrypt increases exponentially as the exponent (key) grows longer. Thus, RSA is markedly slower than DES and AES.

The encryption algorithm is based on the underlying problem of factoring large numbers in a finite set called a field. So far, nobody has found a shortcut or easy way to factor large numbers in a field. In a highly technical but excellent paper, Dan Boneh [[BON99](#)] reviews all the known cryptanalytic attacks on RSA and concludes that none is significant. Because the factorization problem has been open for many decades, most cryptographers consider this problem a solid basis for a secure cryptosystem.

To summarize, the two symmetric algorithms DES and AES provide solid encryption of blocks of 64 to 256 bits of data. The asymmetric algorithm RSA encrypts blocks of various sizes. DES and AES are substantially faster than RSA, by a factor of 10,000 or more, and their rather simple primitive operations have been built into some computer

chips, making their encryption even more efficient than RSA. Therefore, people tend to use DES and AES as the major cryptographic workhorses, and reserve slower RSA for limited uses at which it excels.

The characteristics of secret key (symmetric) and public key (asymmetric) algorithms are compared in [Table 2-13](#).

	Secret Key (Symmetric)	Public Key (Asymmetric)
Number of keys	1	2
Key size (bits)	Depends on the algorithm; 56–112 (DES), 128–256 (AES)	Unlimited; typically no less than 256; 1000 to 2000 currently considered desirable for most uses
Protection of key	Must be kept secret	One key must be kept secret; the other can be freely exposed
Best uses	Cryptographic workhorse. Secrecy and integrity of data, from single characters to blocks of data, messages and files	Key exchange, authentication, signing
Key distribution	Must be out-of-band	Public key can be used to distribute other keys
Speed	Fast	Slow, typically by a factor of up to 10,000 times slower than symmetric algorithms

TABLE 2-13 Comparison of Secret Key and Public Key Encryption

Public Key Cryptography to Exchange Secret Keys

Encryption algorithms alone are not the answer to everyone's encryption needs. Although encryption implements protected communications channels, it can also be used for other duties. In fact, combining symmetric and asymmetric encryption often capitalizes on the best features of each.

Suppose you need to send a protected message to someone you do not know and who does not know you. This situation is more common than you may think. For instance, you may want to send your income tax return to the government. You want the information to be protected, but you do not necessarily know the person who is receiving the information. Similarly, you may want to purchase from a shopping website, exchange private (encrypted) email, or arrange for two hosts to establish a protected channel. Each of these situations depends on being able to exchange an encryption key in such a way that nobody else can intercept it. The problem of two previously unknown parties exchanging cryptographic keys is both hard and important. Indeed, the problem is almost circular: To establish an encrypted session, you need an encrypted means to exchange keys.

Public key cryptography can help. Since asymmetric keys come in pairs, one half of the pair can be exposed without compromising the other half. In fact, you might think of the public half of the key pair as truly public—posted on a public website, listed in a public directory similar to a telephone listing, or sent openly in an email message. That is the beauty of public key cryptography: As long as the private key is not disclosed, a public key can be open without compromising the security of the encryption.

Simple Key Exchange Protocol

Suppose that a sender, Amy, and a receiver, Bill, both have pairs of asymmetric keys for a common encryption algorithm. We denote any public key encryption function as $E(k, X)$, meaning perform the public key encryption function on X by using key k . Call the keys $k_{\text{PRIV-A}}$, $k_{\text{PUB-A}}$, $k_{\text{PRIV-B}}$, and $k_{\text{PUB-B}}$, for the private and public keys for Amy and Bill, respectively.

The problem we want to solve is for Amy and Bill to be able to establish a secret (symmetric algorithm) encryption key that only they know. The simplest solution is for Amy to choose any symmetric key K , and send $E(k_{\text{PUB-A}}, K)$ to Bill. Bill takes Amy's public key, removes the encryption, and obtains K .

This analysis is flawed, however. How does the sender know the public key really belongs to the intended recipient? Consider, for example, the following scenario. Suppose Amy and Bill do not have a convenient bulletin board. So, Amy just asks Bill for his key. Basically, the key exchange protocol, depicted in [Figure 2-24](#), would work like this:

1. Amy says: Bill, please send me your public key.
2. Bill replies: Here, Amy; this is my public key.
3. Amy responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

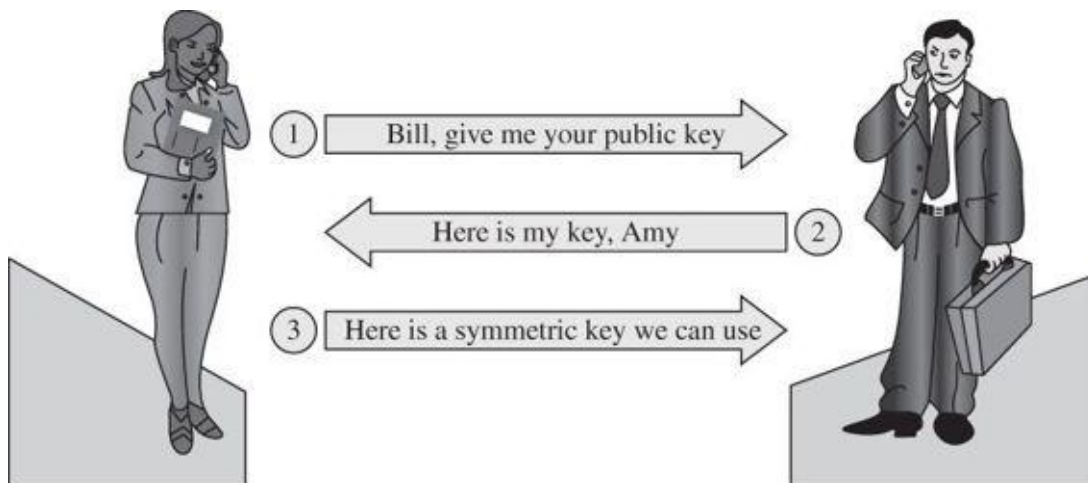


FIGURE 2-24 Key Exchange Protocol

In the subversion shown in [Figure 2-25](#), we insert an attacker, Malvolio, into this communication.

1. Amy says: Bill, please send me your public key.
 - 1a. Malvolio intercepts the message and fashions a new message to Bill, purporting to come from Amy but with Malvolio's return address, asking for Bill's public key.
 2. Bill replies: Here, Amy; this is my public key. (Because of the return address in step 1a, this reply goes to Malvolio.)
 - 2a. Malvolio holds Bill's public key and sends Malvolio's own public key to Amy, alleging it is from Bill.
3. Amy responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

3a. Malvolio intercepts this message and obtains and holds the symmetric key Amy has generated.

3b. Malvolio generates a new symmetric key and sends it to Bill, with a message purportedly from Amy: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

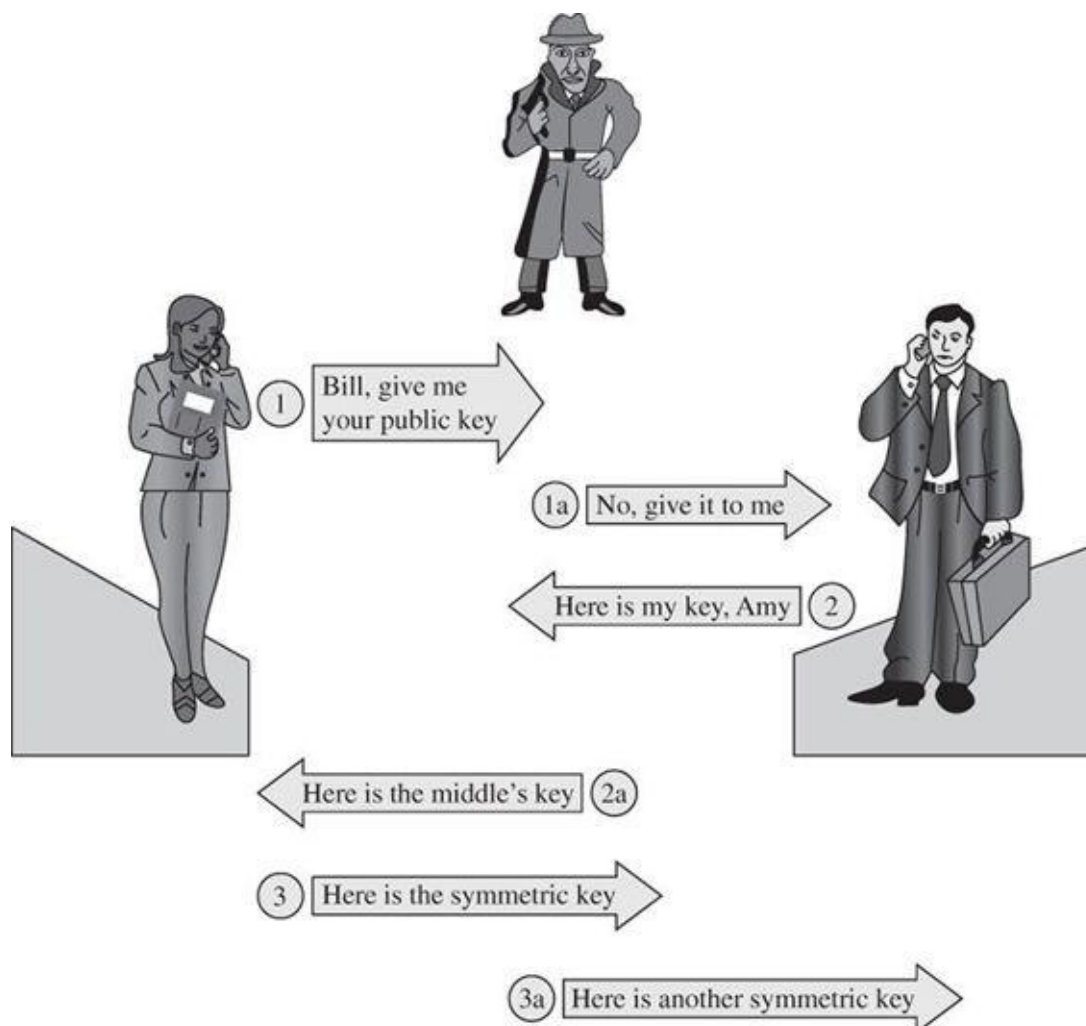


FIGURE 2-25 Key Exchange Protocol with a Man in the Middle

In summary, Malvolio now holds two symmetric encryption keys, one each shared with Amy and Bill. Not only can Malvolio stealthily obtain all their interchanges, but Amy and Bill cannot communicate securely with each other because neither shares a key with the other.

From this point on, all communications pass through Malvolio. Having both symmetric keys, Malvolio can decrypt anything received, modify it, encrypt it under the other key, and transmit the modified version to the other party. Neither Amy nor Bill is aware of the switch. This attack is a type of **man-in-the-middle**⁴ failure, in which an unauthorized third party intercedes in an activity presumed to be exclusively between two people. See [Sidebar 2-15](#) for an example of a real-world man-in-the-middle attack.

⁴ Alas, this terminology is hopelessly sexist. Even if we called these attacks person-in-the-middle or intruder-in-the-middle in this book, you would find only the term man-in-the-middle used by other writers, who also use terms like man-in-the-browser and man-in-the-phone, which arise in [Chapter 4](#) of this book. Thus, we are regrettably stuck with the conventional term.

Sidebar 2-15 Aspidistra, a WW II Man in the Middle

During World War II Britain used a man-in-the-middle attack to delude German pilots and civilians. Aspidistra, the name of a common houseplant also known as cast-iron plant for its seeming ability to live forever, was also the name given to a giant radio transmitter the British War Office bought from RCA in 1942. The transmitter broadcast at 500 kW of power, ten times the power allowed to any U.S. station at the time, which meant Aspidistra was able to transmit signals from Britain into Germany.

Part of the operation of Aspidistra was to delude German pilots by broadcasting spurious directions (land, go here, turn around). Although the pilots also received valid flight instructions from their own controllers, this additional chatter confused them and could result in unnecessary flight and lost time. This part of the attack was only an impersonation attack.

Certain German radio stations in target areas were turned off to prevent their being beacons by which Allied aircraft could home in on the signal; bombers would follow the signal and destroy the antenna and its nearby transmitter if the stations broadcast continually. When a station was turned off, the British immediately went on the air using Aspidistra on the same frequency as the station the Germans just shut down. They copied and rebroadcast a program from another German station, but they interspersed propaganda messages that could demoralize German citizens and weaken support for the war effort.

The Germans tried to counter the phony broadcasts by advising listeners that the enemy was transmitting and advising the audience to listen for the official German broadcast announcement—which, of course, the British duly copied and broadcast themselves. (More details and pictures are at <http://www.qsl.net/g0crw/Special%20Events/Aspidistra2.htm>, and <http://bobrowen.com/nymas/radiopropaper.pdf>.)

Revised Key Exchange Protocol

Remember that we began this discussion with a man-in-the-middle attack against a simple key exchange protocol. The faulty protocol was

1. A says: B, please send me your public key.
2. B replies: Here, A; this is my public key.
3. A responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

At step 2 the intruder intercepts B's public key and passes along the intruder's. The intruder can be foiled if A and B exchange half a key at a time. Half a key is useless to the intruder because it is not enough to encrypt or decrypt anything. Knowing half the key does not materially improve the intruder's ability to break encryptions in the future.

Rivest and Shamir [[RIV84](#)] have devised a solid protocol as follows.

1. Amy sends her public key to Bill.

2. Bill sends his public key to Amy.
3. Amy creates a symmetric key, encrypts it using Bill's public key, and sends half of the result to Bill. (Note: half of the result might be the first $n/2$ bits, all the odd numbered bits, or some other agreed-upon form.)
4. Bill responds to Amy that he received the partial result (which he cannot interpret at this point, so he is confirming only that he received some bits). Bill encrypts any random number with his private key and sends half the bits to Amy.
5. Amy sends the other half of the encrypted result to Bill.
6. Bill puts together the two halves of Amy's result, decrypts it using his private key and thereby obtains the shared symmetric key. Bill sends the other half of his encrypted random number to Amy.
7. Amy puts together the two halves of Bill's random number, decrypts it using her private key, extracts Bill's random number, encrypts it using the now-shared symmetric key, and sends that to Bill.
8. Bill decrypts Amy's transmission with the symmetric key and compares it to the random number he selected in step 6. A match confirms the validity of the exchange.

To see why this protocol works, look at step 3. Malvolio, the intruder, can certainly intercept both public keys in steps 1 and 2 and substitute his own. However, at step 3 Malvolio cannot take half the result, decrypt it using his private key, and reencrypt it under Bill's key. Bits cannot be decrypted one by one and reassembled.

At step 4 Bill picks any random number, which Amy later returns to Bill to show she has successfully received the encrypted value Bill sent. Such a random value is called a **nonce**, a value meaningless in and of itself, to show activity (liveness) and originality (not a replay). In some protocols the receiver decrypts the nonce, adds 1 to it, reencrypts the result, and returns it. Other times the nonce includes a date, time, or sequence number to show that the value is current. This concept is used in computer-to-computer exchanges that lack some of the characteristics of human interaction.

Authenticity

The problem of the person in the middle can be solved in another way: Amy should send to Bill

$$E(k_{PUB-B}, E(k_{PRIV-A}, K))$$

This function ensures that only Bill, using k_{PRIV-B} , can remove the encryption applied with k_{PUB-B} , and Bill knows that only Amy could have applied k_{PRIV-A} that Bill removes with k_{PUB-A} .

We can think of this exchange in terms of locks and seals. Anyone can put a letter into a locked mailbox (through the letter slot), but only the holder of the key can remove it. In olden days, important people had seals that they would impress into molten wax on a letter; the seal's imprint showed authenticity, but anyone could break the seal and read the letter. Putting these two pieces together, a sealed letter inside a locked mailbox enforces

the authenticity of the sender (the seal) and the confidentiality of the receiver (the locked mailbox).

If Amy wants to send something protected to Bill (such as a credit card number or a set of medical records), then the exchange works something like this. Amy seals the protected information with her private key so that it can be opened only with Amy's public key. This step ensures authenticity: only Amy can have applied the encryption that is reversed with Amy's public key. Amy then locks the information with Bill's public key. This step adds confidentiality because only Bill's private key can decrypt data encrypted with Bill's public key. Bill can use his private key to open the letter box (something only he can do) and use Amy's public key to verify the inner seal (proving that the package came from Amy).

Thus, as we have seen, asymmetric cryptographic functions are a powerful means for exchanging cryptographic keys between people who have no prior relationship. Asymmetric cryptographic functions are slow, but they are used only once, to exchange symmetric keys. Furthermore, if the keys being exchanged are for a symmetric encryption system such as AES or DES, the key length is relatively short, up to 256 bits for AES or 64 for DES. Even if we were to use an expanded form of AES with a key length of 1000 bits, the slow speed of public key cryptography would not be a significant problem because it is performed only once, to establish shared keys.

Asymmetric cryptography is also useful for another important security construct: a digital signature. A human signature on a paper document is a strong attestation: it signifies both agreement (you agree to the terms in the document you signed) and understanding (you know what you are signing). People accept written signatures as a surrogate for an in-person confirmation. We would like a similarly powerful construct for confirming electronic documents. To build a digital signature we introduce integrity codes, key certificates, and, finally, signatures themselves.

Error Detecting Codes

Communications are notoriously prone to errors in transmission. You may have noticed that occasionally a mobile phone conversation will skip or distort a small segment of the conversation, and television signals sometimes show problems commonly called noise. In these cases, complete and accurate reception is not important as long as the noise is relatively slight or infrequent. You can always ask your phone partner to repeat a sentence, and a winning goal on television is always rebroadcast numerous times.

With important data, however, we need some way to determine that the transmission is complete and intact. Mathematicians and engineers have designed formulas called error detection and correction codes to make transmission errors apparent and to perform minor repairs.

Error detecting codes come under many names, such as hash codes, message digests, checksums, integrity checks, error detection and correction codes, and redundancy tests. Although these terms have fine differences of meaning, the basic purpose of all is to demonstrate that a block of data has been modified. That sentence is worded carefully: A message digest will (*sometimes*) signal that content has changed, but it is less solid at demonstrating no modification, even though that is what we really want. We want

something to show no tampering—malicious or not; we get something that usually shows tampering.

Sam writes a letter, makes and keeps a photocopy, and sends the original to Theresa. Along the way, Fagin intercepts the letter and makes changes; being a skilled forger, Fagin deceives Theresa. Only when Theresa and Sam meet and compare the (modified) original do they detect the change.

The situation is different if Sam and Theresa suspect a forger is nigh. Sam carefully counts the letters in his document, tallying 1 for an a, 2 for a b, and so on up to 26 for a z. He adds those values and writes the sum in tiny digits at the bottom of the letter. When Theresa receives the letter she does the same computation and compares her result to the one written at the bottom. Three cases arise:

- the counts do not agree, in which case Theresa suspects a change
- there is no count, in which case Theresa suspects either that Sam was lazy or forgetful or that a forger overlooked their code
- Teresa's count is the same as written at the bottom

The last case is the most problematic. Theresa probably concludes with relief that there was no change. As you may have already determined, however, she may not be thinking correctly. Fagin might catch on to the code and correctly compute a new sum to match the modifications. Even worse, perhaps Fagin's changes happen to escape detection. Suppose Fagin removes one letter c (value=3) and replaces it with three copies of the letter a (value=1+1+1=3); the sum is the same, or if Fagin only permutes letters, the sum remains the same, because it is not sensitive to order.

These problems all arise because the code is a many-to-one function: two or more inputs produce the same output. Two inputs that produce the same output are called a **collision**. In fact, all message digests are many-to-one functions, and thus when they report a change, one did occur, but when they report no change, it is only likely—not certain—that none occurred because of the possibility of a collision.

Collisions are usually not a problem for two reasons. First, they occur infrequently. If plaintext is reduced to a 64-bit digest, we expect the likelihood of a collision to be 1 in 2^{64} , or about 1 in 10^{19} , most unlikely, indeed. More importantly, digest functions are unpredictable, so given one input, finding a second input that results in the same output is infeasible. Thus, with good digest functions collisions are infrequent, and we cannot cause or predict them.

We can use error detecting and error correcting codes to guard against modification of data. Detection and correction codes are procedures or functions applied to a block of data; you may be familiar with one type of detecting code: parity. These codes work as their names imply: Error detecting codes detect when an error has occurred, and error correcting codes can actually correct errors without requiring a copy of the original data. The error code is computed and stored safely on the presumed intact, original data; later anyone can recompute the error code and check whether the received result matches the expected value. If the values do not match, a change has certainly occurred; if the values match, it is probable—but not certain—that no change has occurred.

Parity

The simplest error detection code is a **parity** check. An extra bit, which we call a fingerprint, is added to an existing group of data bits, depending on their sum. The two kinds of parity are called even and odd. With even parity the fingerprint is 0 if the sum of the data bits is even, and 1 if the sum is odd; that is, the parity bit is set so that the sum of all data bits plus the parity bit is even. Odd parity is the same except the overall sum is odd. For example, the data stream 01101101 would have an even parity bit of 1 (and an odd parity bit of 0) because $0+1+1+0+1+1+0+1 = 5 + 1 = 6$ (or $5 + 0 = 5$ for odd parity).

One parity bit can reveal the modification of a single bit. However, parity does not detect two-bit errors—cases in which two bits in a group are changed. One parity bit can detect all single-bit changes, as well as changes of three, five and seven bits. [Table 2-14](#) shows some examples of detected and undetected changes. The changed bits (each line shows changes from the original value of 00000000) are in bold, underlined; the table shows whether parity properly detected that at least one change occurred.

Original Data	Parity Bit	Modified Data	Modification Detected?
0 0 0 0 0 0 0 0	1	0 0 0 0 0 0 0 <u>1</u>	Yes
0 0 0 0 0 0 0 0	1	<u>1</u> 0 0 0 0 0 0 0	Yes
0 0 0 0 0 0 0 0	1	<u>1</u> 0 0 0 0 0 0 <u>1</u>	No
0 0 0 0 0 0 0 0	1	0 0 0 0 0 0 0 <u>1</u> <u>1</u>	No
0 0 0 0 0 0 0 0	1	0 0 0 0 0 <u>1</u> <u>1</u> <u>1</u>	Yes
0 0 0 0 0 0 0 0	1	0 0 0 0 <u>1</u> <u>1</u> <u>1</u> <u>1</u>	No
0 0 0 0 0 0 0 0	1	0 <u>1</u> 0 <u>1</u> 0 <u>1</u> 0 <u>1</u>	No
0 0 0 0 0 0 0 0	1	<u>1</u> <u>1</u> <u>1</u> <u>1</u> <u>1</u> <u>1</u> <u>1</u> <u>1</u>	No

TABLE 2-14 Changes Detected by Parity

Detecting odd numbers of changed bits leads to a change detection rate of about 50 percent, which is not nearly good enough for our purposes. We can improve this rate with more parity bits (computing a second parity bit of bits 1, 3, 5, and 7, for example), but more parity bits increase the size of the fingerprint; each time we increase the fingerprint size we also increase the size of storing these fingerprints.

Parity signals only that a bit has been changed; it does not identify which bit has been changed, much less when, how, or by whom. On hardware storage devices, a code called a **cyclic redundancy** check detects errors in recording and playback. Some more complex codes, known as **error correction codes**, can detect multiple-bit errors (two or more bits changed in a data group) and may be able to pinpoint the changed bits (which are the bits to reset to correct the modification). Fingerprint size, error detection rate, and correction lead us to more powerful codes.

Hash Codes

In most files, the elements or components of the file are not bound together in any way. That is, each byte or bit or character is independent of every other one in the file. This lack of binding means that changing one value affects the integrity of the file but that one change can easily go undetected.

What we would like to do is somehow put a seal or shield around the file so that we can detect when the seal has been broken and thus know that something has been changed. This notion is similar to the use of wax seals on letters in medieval days; if the wax was broken, the recipient would know that someone had broken the seal and read the message inside. In the same way, cryptography can be used to **seal** a file, encasing it so that any change becomes apparent. One technique for providing the seal is to compute a function, sometimes called a **hash** or **checksum** or **message digest** of the file.

The code between Sam and Theresa is a hash code. Hash codes are often used in communications where transmission errors might affect the integrity of the transmitted data. In those cases the code value is transmitted with the data. Whether the data or the code value was marred, the receiver detects some problem and simply requests a retransmission of the data block.

Such a protocol is adequate in cases of unintentional errors but is not intended to deal with a dedicated adversary. If Fagin knows the error detection function algorithm, then he can change content and fix the detection value to match. Thus, when a malicious adversary might be involved, secure communication requires a stronger form of message digest.

One-Way Hash Functions

As a first step in defeating Fagin, we have to prevent him from working backward from the digest value to see what possible inputs could have led to that result. For instance, some encryptions depend on a function that is easy to understand but difficult to compute. For a simple example, consider the cube function, $y = x^3$. Computing x^3 by hand, with pencil and paper, or with a calculator is not hard. But the inverse function, $\sqrt[3]{y}$, is much more difficult to compute. And the function $y = x^2$ has no inverse function since there are two possibilities for $\sqrt{y}$: $+x$ and $-x$. Functions like these, which are much easier to compute than their inverses, are called **one-way functions**.

File Change Detection

A one-way function can be useful in creating a change detection algorithm. The function must depend on all bits of the file being sealed, so any change to even a single bit will alter the checksum result. The checksum value is stored with the file. Then, each time someone accesses or uses the file, the system recomputes the checksum. If the computed checksum matches the stored value, the file is likely to be intact.

The one-way property guards against malicious modification: An attacker cannot “undo” the function to see what the original file was, so there is no simple way to find a set of changes that produce the same function value. (Otherwise, the attacker could find undetectable modifications that also have malicious impact.)

Tripwire [[KIM98](#)] is a utility program that performs integrity checking on files. With Tripwire a system administrator computes a hash of each file and stores these hash values somewhere secure, typically offline. Later the administrator reruns Tripwire and compares the new hash values with the earlier ones.

Cryptographic Checksum

Malicious modification must be handled in a way that also prevents the attacker from modifying the error detection mechanism as well as the data bits themselves. One way to handle this is to use a technique that shrinks and transforms the data according to the value of the data bits.

A **cryptographic checksum** is a cryptographic function that produces a checksum. It is a digest function using a cryptographic key that is presumably known only to the originator and the proper recipient of the data. The cryptography prevents the attacker from changing the data block (the plaintext) and also changing the checksum value (the ciphertext) to match. The attacker can certainly change the plaintext, but the attacker does not have a key with which to recompute the checksum. One example of a cryptographic checksum is to first employ any noncryptographic checksum function to derive an n -bit digest of the sensitive data. Then apply any symmetric encryption algorithm to the digest. Without the key the attacker cannot determine the checksum value that is hidden by the encryption. We present other cryptographic hash functions in [Chapter 12](#).

Two major uses of cryptographic checksums are code-tamper protection and message-integrity protection in transit. Code tamper protection is implemented in the way we just described for detecting changes to files. Similarly, a checksum on data in communication identifies data that have been changed in transmission, maliciously or accidentally. The U.S. government defined the **Secure Hash Standard** or **Algorithm** (SHS or SHA), actually a collection of algorithms, for computing checksums. We examine SHA in [Chapter 12](#).

Checksums are important countermeasures to detect modification. In this section we applied them to the problem of detecting malicious modification to programs stored on disk, but the same techniques are applicable to protecting against changes to data, as we show later in this book.

A strong cryptographic algorithm, such as for DES or AES, is especially appropriate for sealing values, since an outsider will not know the key and thus will not be able to modify the stored value to match with data being modified. For low-threat applications, algorithms even simpler than those of DES or AES can be used. In block encryption schemes, chaining means linking each block to the previous block's value (and therefore to all previous blocks), for example, by using an exclusive OR to combine the encrypted previous block with the current one. A file's cryptographic checksum could be the last block of the chained encryption of a file because that block will depend on all other blocks. We describe chaining in more detail in [Chapter 12](#).

As we see later in this chapter, these techniques address the non-alterability and non-reusability required in a digital signature. A change or reuse will probably be flagged by the checksum so the recipient can tell that something is amiss.

Signatures

The most powerful technique to demonstrate authenticity is a digital signature. Like its counterpart on paper, a digital signature is a way by which a person or organization can affix a bit pattern to a file such that it implies confirmation, pertains to that file only, cannot be forged, and demonstrates authenticity. We want a means by which one party can sign something and, as on paper, have the signature remain valid for days, months, years—indefinitely. Furthermore, the signature must convince all who access the file. Of course, as with most conditions involving digital methods, the caveat is that the assurance is limited by the assumed skill and energy of anyone who would try to defeat the assurance.

A digital signature often uses asymmetric or public key cryptography. As you just saw, a public key protocol is useful for exchange of cryptographic keys between two parties who have no other basis for trust. Unfortunately, the public key cryptographic protocols involve several sequences of messages and replies, which can be time consuming if either party is not immediately available to reply to the latest request. It would be useful to have a technique by which one party could reliably precompute some protocol steps and leave them in a safe place so that the protocol could be carried out even if only one party were active. This situation is similar to the difference between a bank teller and an ATM. You can obtain cash, make a deposit or payment, or check your balance because the bank has pre-established steps for an ATM to handle those simple activities 24 hours a day, even if the bank is not open. But if you need a certified check or foreign currency, you may need to interact directly with a bank agent.

In this section we define digital signatures and compare their properties to those of handwritten signatures on paper. We then describe the infrastructure surrounding digital signatures that lets them be recognizable and valid indefinitely.

Components and Characteristics of Signatures

A digital signature is just a binary object associated with a file. But if we want that signature to have the force of a paper-based signature, we need to understand the properties of human signatures. Only then can we express requirements for our digital version.

Properties of Secure Paper-Based Signatures

Consider a typical situation that parallels a common human need: an order to transfer funds from one person to another. In other words, we want to be able to send the electronic equivalent of a computerized check. We understand the properties of this transaction for a conventional paper check:

- A check is a *tangible object* authorizing a financial transaction.
- The signature on the check *confirms authenticity* because (presumably) only the legitimate signer can produce that signature.
- In the case of an alleged forgery, a third party can be called in to *judge authenticity*.
- Once a check is cashed, it is canceled so that it *cannot be reused*.
- The paper check is *not alterable*. Or, most forms of alteration are easily detected.

Transacting business by check depends on *tangible objects* in a *prescribed form*. But tangible objects do not exist for transactions on computers. Therefore, authorizing payments by computer requires a different model. Let us consider the requirements of such a situation, from the standpoint both of a bank and of a user.

Properties of Digital Signatures

Suppose Sheila sends her bank a message authorizing it to transfer \$100 to Rob. Sheila's bank must be able to verify and prove that the message really came from Sheila if she should later disavow sending the message. (This property is called **non-repudiation**.) The bank also wants to know that the message is entirely Sheila's, that it has not been altered along the way. For her part, Sheila wants to be certain that her bank cannot forge such messages. (This property is called **authenticity**.) Both parties want to be sure that the message is new, not a reuse of a previous message, and that it has not been altered during transmission. Using electronic signals instead of paper complicates this process.

But we have ways to make the process work. A **digital signature** is a protocol that produces the same effect as a real signature: It is a mark that only the sender can make but that other people can easily recognize as belonging to the sender. Just like a real signature, a digital signature confirms agreement to a message.

A digital signature must meet two primary conditions:

- It must be *unforgeable*. If person S signs message M with signature $Sig(S,M)$, no one else can produce the pair $[M, Sig(S,M)]$.
- It must be *authentic*. If a person R receives the pair $[M, Sig(S,M)]$ purportedly from S , R can check that the signature is really from S . Only S could have created this signature, and the signature is firmly attached to M .

These two requirements, shown in [Figure 2-26](#), are the major hurdles in computer transactions. Two more properties, also drawn from parallels with the paper-based environment, are desirable for transactions completed with the aid of digital signatures:

- It is *not alterable*. After being transmitted, M cannot be changed by S , R , or an interceptor.
- It is *not reusable*. A previous message presented again will be instantly detected by R .

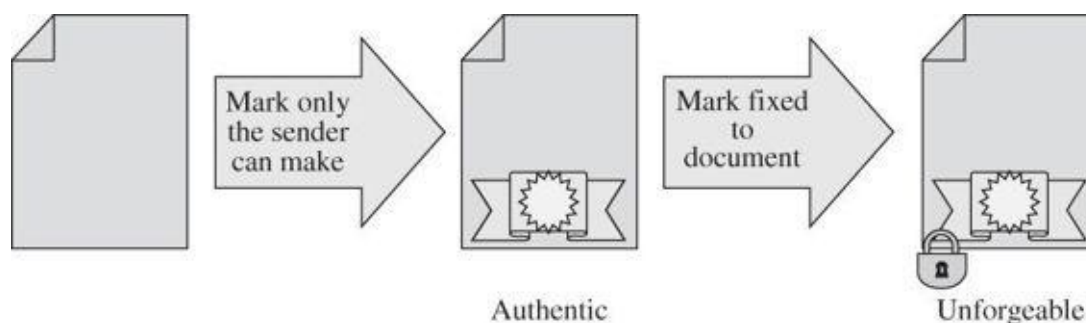


FIGURE 2-26 Digital Signature Requirements

To see how digital signatures work, we first present a mechanism that meets the first two requirements. We then add to that solution to satisfy the other requirements. We develop digital signatures in pieces: first building a piece to address alterations, then

describing a way to ensure authenticity, and finally developing a structure to establish identity. Eventually all these parts tie together in a conceptually simple framework.

We have just described the pieces for a digital signature: public key cryptography and secure message digests. These two pieces together are technically enough to make a digital signature, but they do not address authenticity. For that, we need a structure that binds a user's identity and public key in a trustworthy way. Such a structure is called a certificate. Finally, we present an infrastructure for transmitting and validating certificates.

Public Keys for Signatures

Public key encryption systems are ideally suited to signing. For simple notation, let us assume that the public key encryption for user U is accessed through $E(M, K_U)$ and that the private key transformation for U is written as $D(M, K_U)$. We can think of E as the privacy transformation (since only U can decrypt it) and D as the authenticity transformation (since only U can produce it). Remember, however, that under some asymmetric algorithms such as RSA, D and E are commutative and either one can be applied to any message. Thus,

$$D(E(M, K_U), K_U) = M = E(D(M, K_U), K_U)$$

If S wishes to send M to R , S uses the authenticity transformation to produce $D(M, K_S)$. S then sends $D(M, K_S)$ to R . R decodes the message with the public key transformation of S , computing $E(D(M, K_S), K_S) = M$. Since only S can create a message that makes sense under $E(-, K_S)$, the message must genuinely have come from S . This test satisfies the authenticity requirement.

R will save $D(M, K_S)$. If S should later allege that the message is a forgery (not really from S), R can simply show M and $D(M, K_S)$. Anyone can verify that since $D(M, K_S)$ is transformed to M with the public key transformation of S —but only S could have produced $D(M, K_S)$ —then $D(M, K_S)$ must be from S . This test satisfies the unforgeable requirement.

There are other approaches to signing; some use symmetric encryption, others use asymmetric. The approach shown here illustrates how the protocol can address the requirements for unforgeability and authenticity. To add secrecy, S applies $E(M, K_R)$ as shown in [Figure 2-27](#).

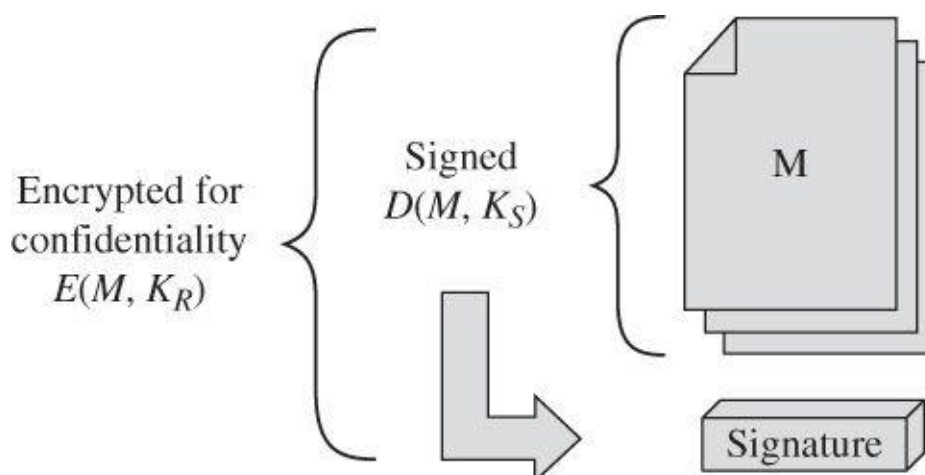


FIGURE 2-27 Use of Two Keys in an Asymmetric Digital Signature

These pieces, a hash function, public key cryptography, and a protocol, give us the technical pieces of a digital signature. However, we also need one nontechnical component. Our signer *S* can certainly perform the protocol to produce a digital signature, and anyone who has *S*'s public key can determine that the signature did come from *S*. But who is *S*? We have no reliable way to associate a particular human with that public key. Even if someone says "this public key belongs to *S*," on what basis do we believe that assertion? Remember the man-in-the-middle attack earlier in this chapter when Amy and Bill wanted to establish a shared secret key? Next we explore how to create a trustworthy binding between a public key and an identity.

Trust

A central issue of digital commerce is trust: How do you know that a Microsoft web page really belongs to Microsoft, for example? This section is less about technology and more about the human aspects of trust, because that confidence underpins the whole concept of a digital signature.

In real life you may trust a close friend in ways you would not trust a new acquaintance. Over time your trust in someone may grow with your experience but can plummet if the person betrays you. You try out a person, and, depending on the outcome, you increase or decrease your degree of trust. These experiences build a personal trust framework.

Web pages can be replaced and faked without warning. To some extent, you assume a page is authentic if nothing seems unusual, if the content on the site seems credible or at least plausible, and if you are not using the site for critical decisions. If the site is that of your bank, you may verify that the URL looks authentic. Some sites, especially those of financial institutions, have started letting each customer pick a security image, for example, a hot red sports car or an iconic landmark; users are warned to enter sensitive information only if they see the personal image they previously chose.

In a commercial setting, certain kinds of institutions connote trust. You may trust (the officials at) certain educational, religious, or social organizations. Big, well-established companies such as banks, insurance companies, hospitals, and major manufacturers have developed a measure of trust. Age of an institution also inspires trust. Indeed, trust is the basis for the notion of branding, in which you trust something's quality because you know the brand. As you will see shortly, trust in such recognized entities is an important component in digital signatures.

Establishing Trust Between People

As humans we establish trust all the time in our daily interactions with people. We identify people we know by recognizing their voices, faces, or handwriting. At other times, we use an affiliation to convey trust. For instance, if a stranger telephones us and we hear, "I represent the local government ..." or "I am calling on behalf of this charity ..." or "I am calling from the school/hospital/police about your mother/father/son/daughter/brother/sister ...," we may decide to trust the caller even if we do not know him or her. Depending on the nature of the call, we may decide to believe the caller's affiliation or to seek independent verification. For example, we may obtain the

affiliation's number from the telephone directory and call the party back. Or we may seek additional information from the caller, such as "What color jacket was she wearing?" or "Who is the president of your organization?" If we have a low degree of trust, we may even act to exclude an outsider, as in "I will mail a check directly to your charity rather than give you my credit card number."

For each of these interactions, we have what we might call a "trust threshold," a degree to which we are willing to believe an unidentified individual. This threshold exists in commercial interactions, too. When Acorn Manufacturing Company sends Big Steel Company an order for 10,000 sheets of steel, to be shipped within a week and paid for within ten days, trust abounds. The order is printed on an Acorn form, signed by someone identified as Helene Smudge, Purchasing Agent. Big Steel may begin preparing the steel even before receiving money from Acorn. Big Steel may check Acorn's credit rating to decide whether to ship the order without payment first. If suspicious, Big Steel might telephone Acorn and ask to speak to Ms. Smudge in the purchasing department. But more likely Big Steel will actually ship the goods without knowing who Ms. Smudge is, whether she is actually the purchasing agent, whether she is authorized to commit to an order of that size, or even whether the signature is actually hers. Sometimes a transaction like this occurs by fax, so that Big Steel does not even have an original signature on file. In cases like this one, which occur daily, trust is based on appearance of authenticity (such as a printed, signed form), outside information (such as a credit report), and urgency (Acorn's request that the steel be shipped quickly).

Establishing Trust Electronically

For electronic communication to succeed, we must develop similar ways for two parties to establish trust without having met. A common thread in our personal and business interactions is the ability to have someone or something vouch for the existence and integrity of one or both parties. The police, the Chamber of Commerce, or the Better Business Bureau vouches for the authenticity of a caller. Acorn indirectly vouches for the fact that Ms. Smudge is its purchasing agent by transferring the call to her in the purchasing department when Big Steel calls for her. In a sense, the telephone company vouches for the authenticity of a party by listing someone in the directory. This concept of "vouching for" by a third party can be a basis for trust in commercial settings where two parties do not know each other.

The trust issue we need to address for digital signatures is authenticity of the public key. If Monique signs a document with her private key, anyone else can decrypt the signature with her public key to verify that only Monique could have signed it. The only problem is being able to obtain Monique's public key in a way in which we can adequately trust that the key really belongs to her, that is, that the key was not circulated by some evil actor impersonating Monique. In the next section we present a trustworthy means to bind a public key with an identity.

Trust Based On a Common Respected Individual

A large company may have several divisions, each division may have several departments, each department may have several projects, and each project may have several task groups (with variations in the names, the number of levels, and the degree of

completeness of the hierarchy). The top executive may not know by name or sight every employee in the company, but a task group leader knows all members of the task group, the project leader knows all task group leaders, and so on. This hierarchy can become the basis for trust throughout the organization.

To see how, suppose two people meet: Ann and Andrew. Andrew says he works for the same company as Ann. Ann wants independent verification that he does. She finds out that Bill and Betty are two task group leaders for the same project (led by Camilla); Ann works for Bill and Andrew for Betty. (The organizational relationships are shown in [Figure 2-28](#).) These facts give Ann and Andrew a basis for trusting each other's identity. The chain of verification might be something like this:

- Ann asks Bill who Andrew is.
- Bill either asks Betty, if he knows her directly, and if not, he asks Camilla.
- (If asked, Camilla then asks Betty.)
- Betty replies to Camilla or Bill that Andrew works for her.
- (Camilla tells Bill, if she was involved.)
- Bill tells Ann.

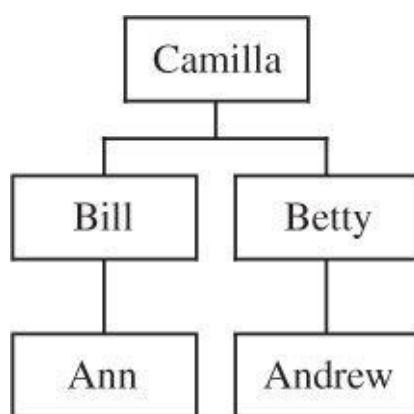


FIGURE 2-28 Trust Relationships

If Andrew is in a different task group, it may be necessary to go higher in the organizational tree before a common point is found.

We can use a similar process for cryptographic key exchange, as shown in [Figure 2-29](#). If Andrew and Ann want to communicate, Andrew can give his public key to Betty, who passes it to Camilla, then Bill, or directly to Bill, who gives it to Ann. But this sequence is not exactly the way it would work in real life. The key would probably be accompanied by a note saying it is from Andrew, ranging from a bit of yellow paper to a form 947 Statement of Identity. And if a form 947 is used, then Betty would also have to attach a form 632a Transmittal of Identity, Camilla would attach another 632a, and Bill would attach a final one, as shown in [Figure 2-29](#). This chain of forms 632a would say, in essence, “I am Betty and I received this key and the attached statement of identity personally from a person I know to be Andrew,” “I am Camilla and I received this key and the attached statement of identity and the attached transmittal of identity personally from a person I know to be Betty,” and so forth. When Ann receives the key, she can review the chain of evidence and conclude with reasonable assurance that the key really did come from Andrew. This protocol is a way of obtaining authenticated public keys, a binding of a

key and a reliable identity.

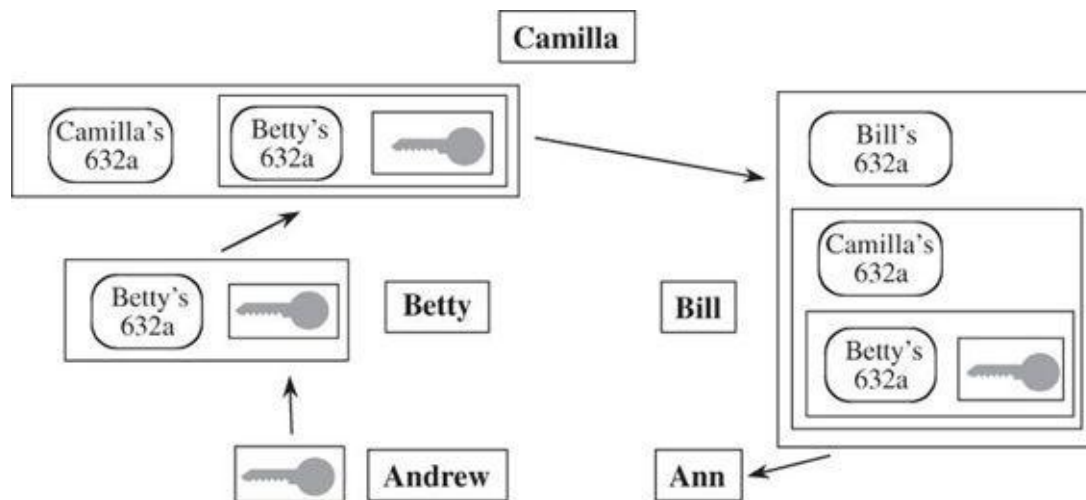


FIGURE 2-29 Key Relationships in a Certificate

This model works well within a company because there is always someone common to any two employees, even if the two employees are in different divisions so that the only common person is the president. The process bogs down, however, if Ann, Bill, Camilla, Betty, and Andrew all have to be available whenever Ann and Andrew want to communicate. If Betty is away on a business trip or Bill is off sick, the protocol falters. It also does not work well if the president cannot get any meaningful work done because every day is occupied with handling forms 632a.

To address the first of these problems, Andrew can ask for his complete chain of forms 632a from the president down to him. Andrew can then give a copy of this full set to anyone in the company who wants his key. Instead of working from the bottom up to a common point, Andrew starts at the top and documents his full chain. He gets these signatures any time his superiors are available, so they do not need to be available when he wants to give away his authenticated public key.

We can resolve the second problem by reversing the process. Instead of starting at the bottom (with task members) and working to the top of the tree (the president), we start at the top. Andrew thus has a preauthenticated public key for unlimited use in the future. Suppose the expanded structure of our hypothetical company, showing the president and other levels, is as illustrated in [Figure 2-30](#).

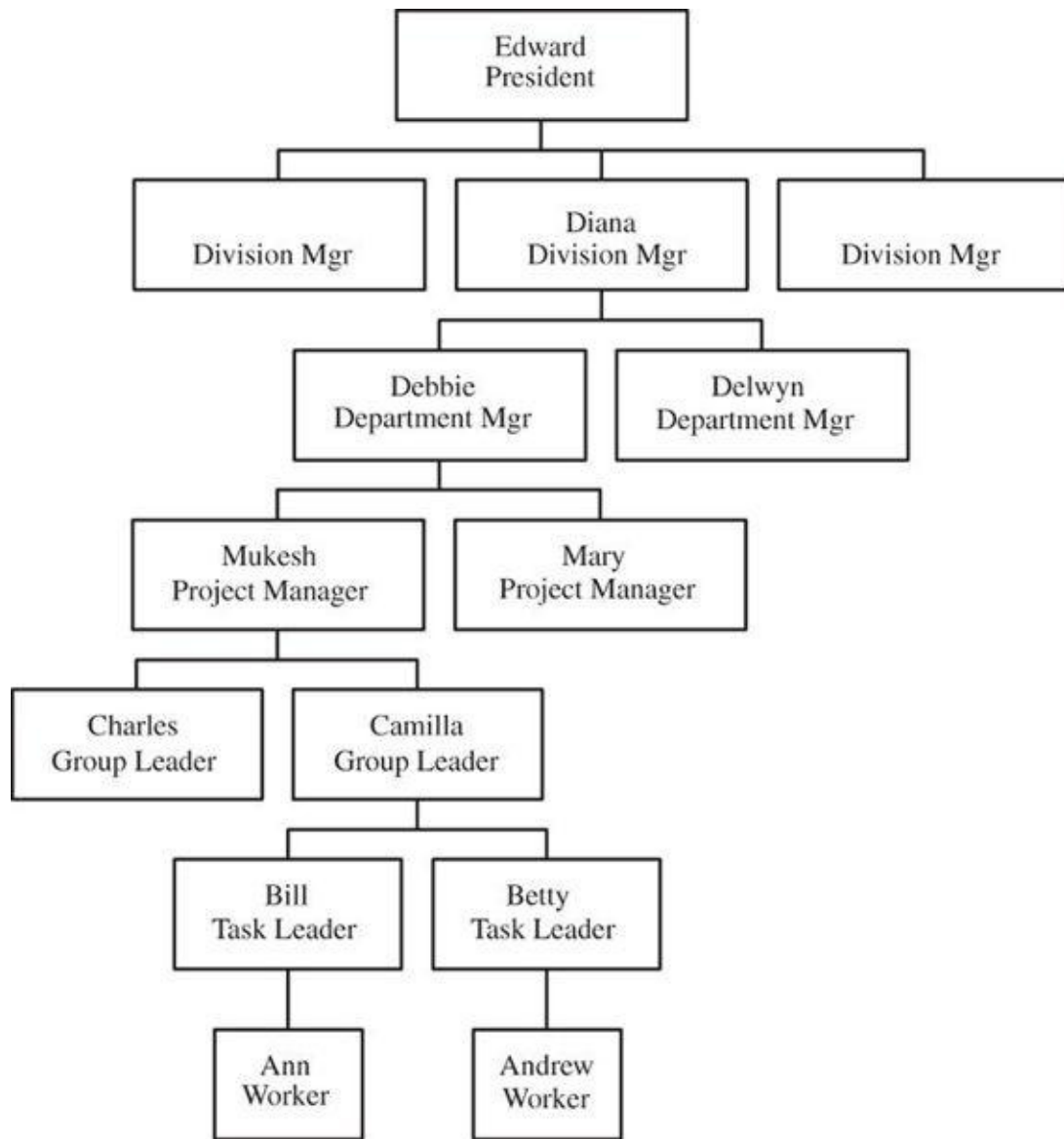


FIGURE 2-30 Delegation of Trust

The president creates a letter for each division manager saying “I am Edward, the president, I attest to the identity of division manager Diana, whom I know personally, and I trust Diana to attest to the identities of her subordinates.” Each division manager does similarly, copying the president’s letter with each letter the manager creates, and so on. Andrew receives a packet of letters, from the president down through his task group leader, each letter linked by name to the next. If every employee in the company receives such a packet, any two employees who want to exchange authenticated keys need only compare each other’s packets; both packets will have at least Edward in common, perhaps some other managers below Edward, and at some point will deviate. Andrew and Ann, for example, could compare their chains, determine that they were the same through Camilla, and trace just from Camilla down. Andrew knows the chain from Edward to Camilla is authentic because it is identical to his chain, and Ann knows the same. Each knows the rest of the chain is accurate because it follows an unbroken line of names and signatures.

Certificates: Trustable Identities and Public Keys

You may have concluded that this process works, but it is far too cumbersome to apply in real life; perhaps you have surmised that we are building a system for computers. This protocol is represented more easily electronically than on paper. With paper, people must

guard against forgeries, to prevent part of one chain from being replaced and to ensure that the public key at the bottom is bound to the chain. The whole thing can be done electronically with digital signatures and hash functions. Kohnfelder [KOH78] seems to be the originator of the concept of using an electronic certificate with a chain of authenticators; Merkle's paper [MER80] expands the concept.

A public key and user's identity are bound together in a **certificate**, which is then signed by someone called a **certificate authority**, certifying the accuracy of the binding. In our example, the company might set up a certificate scheme in the following way. First, Edward selects a public key pair, posts the public part where everyone in the company can retrieve it, and retains the private part. Then, each division manager, such as Diana, creates her public key pair, puts the public key in a message together with her identity, and passes the message securely to Edward. Edward signs it by creating a hash value of the message and then encrypting the hash with his private key. By signing the message, Edward affirms that the public key (Diana's) and the identity (also Diana's) in the message are for the same person. This message is called Diana's certificate.

All of Diana's department managers create messages with their public keys, Diana hashes and signs each, and returns them. She also appends to each a copy of the certificate she received from Edward. In this way, anyone can verify a manager's certificate by starting with Edward's well-known public key, decrypting Diana's certificate to retrieve her public key (and identity), and using Diana's public key to decrypt the manager's certificate. [Figure 2-31](#) shows how certificates are created for Diana and one of her managers, Delwyn. This process continues down the hierarchy to Ann and Andrew. As shown in [Figure 2-32](#), Andrew's certificate is really his individual certificate combined with all certificates for those above him in the line to the president.

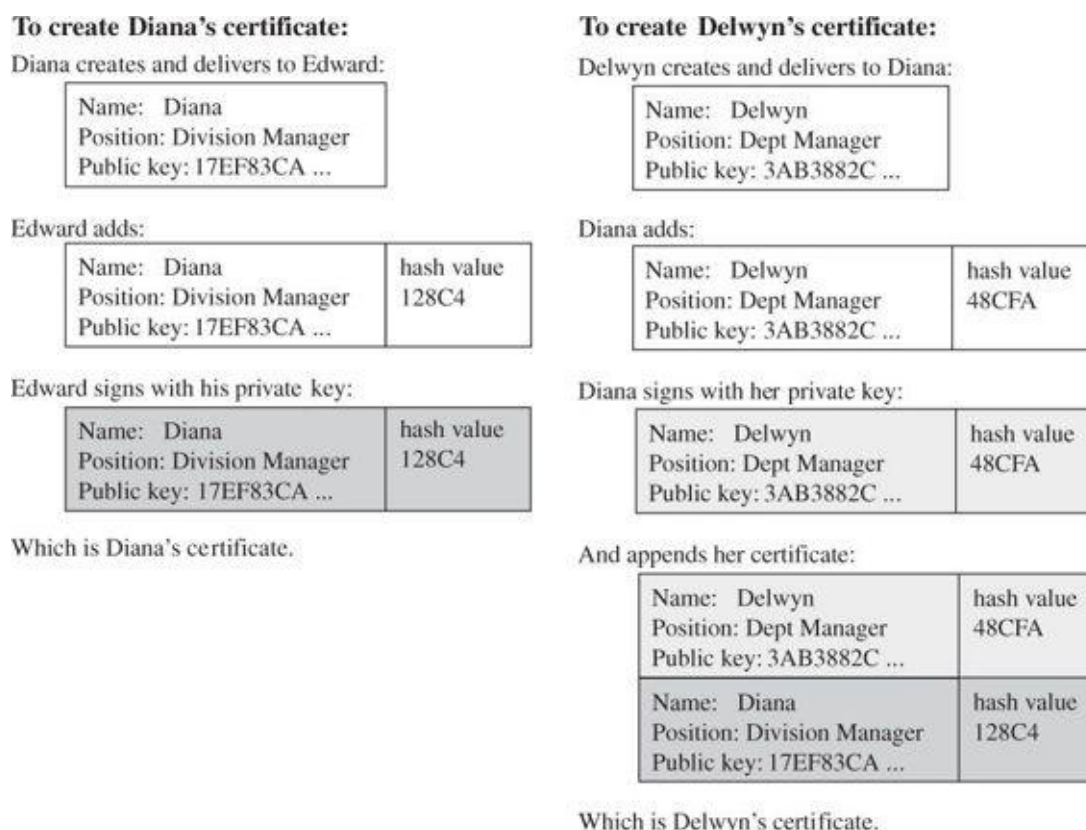


FIGURE 2-31 Creating Certificates

Key to encryptions

-  Encrypted under Betty's private key
-  Encrypted under Camilla's private key
-  Encrypted under Mukesh's private key
-  Encrypted under Delwyn's private key
-  Encrypted under Diana's private key
-  Encrypted under Edward's private key

Name: Andrew Position: Worker Public key: 7013F82A ...	hash value 60206
Name: Betty Position: Task Leader Public key: 2468ACE0 ...	hash value 00002
Name: Camilla Position: Group Leader Public key: 44082CCA ...	hash value 12346
Name: Mukesh Position: Project Manager Public key: 47F0F008 ...	hash value 16802
Name: Delwyn Position: Dept Manager Public key: 3AB3882C ...	hash value 48CFA
Name: Diana Position: Division Manager Public key: 17EF83CA ...	hash value 128C4

FIGURE 2-32 Certificate Hierarchy

Certificate Signing Without a Single Hierarchy

In our examples, certificates were issued on the basis of the managerial structure. But we do not require such a structure nor do we have to follow such a convoluted process in order to use certificate signing for authentication. Anyone who is considered acceptable as an authority can sign a certificate. For example, if you want to determine whether a person received a degree from a university, you would not contact the president or chancellor but would instead go to the office of records or the registrar. To verify someone's employment, you might ask the personnel office or the director of human resources. And to check if someone lives at a particular address, you might consult the office of public records.

Sometimes, a particular person is designated to attest to the authenticity or validity of a document or person. For example, a notary public attests to the validity of a (written) signature on a document. Some companies have a security officer to verify that an employee has appropriate security clearances to read a document or attend a meeting. Many companies have a separate personnel office for each site or each plant location; the personnel officer vouches for the employment status of the employees at that site. Any of these officers or heads of offices could credibly sign certificates for people under their purview. Natural hierarchies exist in society, and these same hierarchies can be used to validate certificates.

The only problem with a hierarchy is the need for trust of the top level. The entire chain of authenticity is secure because each certificate contains the key that decrypts the next certificate, except for the top. Within a company, employees naturally trust the person at the top. But if certificates are to become widely used in electronic commerce, people must be able to exchange certificates securely across companies, organizations, and countries.

The Internet is a large federation of networks for interpersonal, intercompany, interorganizational, and international (as well as intracompany, intraorganizational, and intranational) communication. It is not a part of any government, nor is it a privately

owned company. It is governed by a board called the Internet Society. The Internet Society has power only because its members, the governments and companies that make up the Internet, agree to work together. But there really is no “top” for the Internet. Different companies, such as C&W HKT, SecureNet, VeriSign, Baltimore Technologies, Deutsche Telecom, Società Interbancaria per l’Automatizzazione di Milano, Entrust, and Certiposte are root certification authorities, which means each is a highest authority that signs certificates. So, instead of one root and one top, there are many roots, largely structured around national boundaries.

Distributing Keys and Certificates

Earlier in this chapter we introduced several approaches to key distribution, ranging from direct exchange to distribution through a central distribution facility to certified advance distribution. But no matter what approach is taken to key distribution, each has its advantages and disadvantages. Points to keep in mind about any key distribution protocol include the following:

- What operational restrictions are there? For example, does the protocol require a continuously available facility, such as the key distribution center?
- What trust requirements are there? Who and what entities must be trusted to act properly?
- What is the protection against failure? Can an outsider impersonate any of the entities in the protocol and subvert security? Can any party of the protocol cheat without detection?
- How efficient is the protocol? A protocol requiring several steps to establish an encryption key that will be used many times is one thing; it is quite another to go through several time-consuming steps for a one-time use.
- How easy is the protocol to implement? Notice that complexity in computer implementation may be different from manual use.

Digital Signatures—All the Pieces

Putting these pieces together we can now outline a complete digital signature scheme. Assume user S wants to apply a digital signature to a file (or other data object), meeting the four objectives of a digital signature: unforgeable, authentic, unalterable, and not reusable.

A **digital signature** consists of

- a file
- demonstration that the file has not been altered
- indication of who applied the signature
- validation that the signature is authentic, that is, that it belongs to the signer
- connection of the signature to the file

With these five components we can construct a digital signature.

We start with the file. If we use a secure hash code of the file to compute a message digest and include that hash code in the signature, the code demonstrates that the file has

not been changed. A recipient of the signed file can recompute the hash function and, if the hash values match, conclude with reasonable trust that the received file is the same one that was signed. So far, our digital signature looks like the object in [Figure 2-33](#).

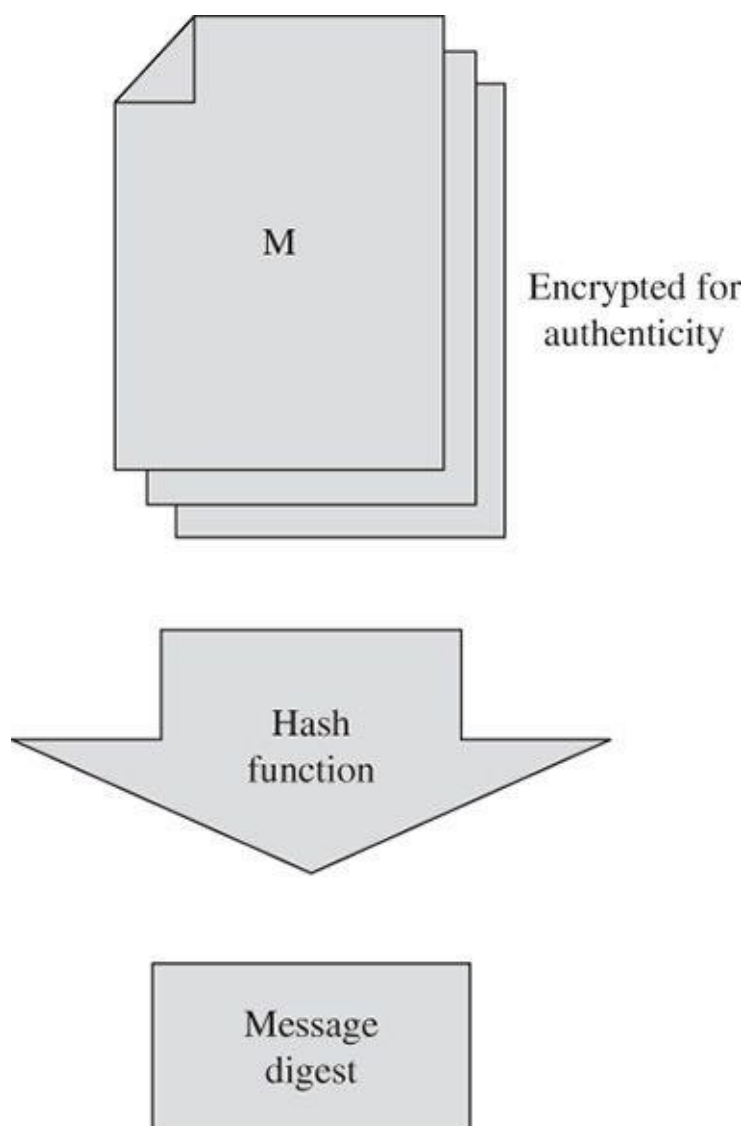


FIGURE 2-33 Hash Code to Detect Changes

Next, we apply the signer’s private encryption key to encrypt the message digest. Because only the signer knows that key, the signer is the only one who could have applied it. Now the signed object looks like [Figure 2-34](#).

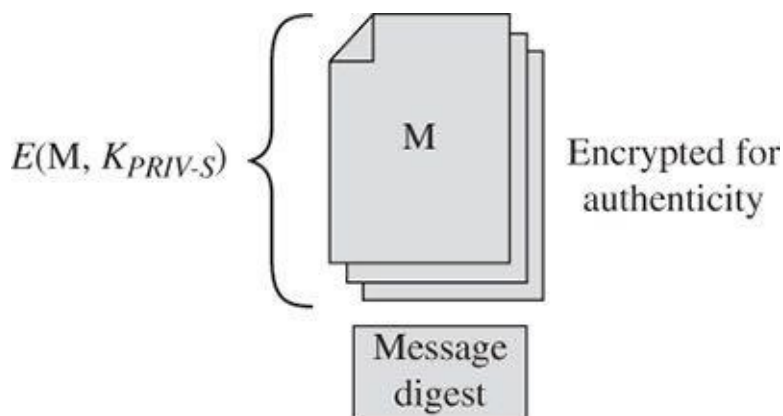


FIGURE 2-34 Encryption to Show Authenticity

The only other piece to add is an indication of who the signer was, so that the receiver

knows which public key to use to unlock the encryption, as shown in [Figure 2-35](#). The signer's identity has to be outside the encryption because if it were inside, the identity could not be extracted.

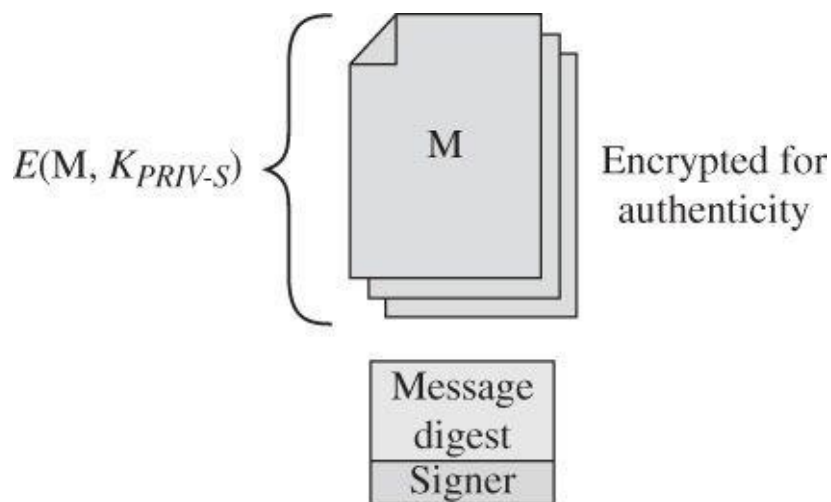


FIGURE 2-35 Indication of Signer

Two extra flourishes remain to be added. First, depending on the file's size, this object can be large, and asymmetric encryption is slow, not suited to encrypting large things. However, S's authenticating encryption needs to cover only the secure hash code, not the entire file itself. If the file were modified, it would no longer match the hash code, so the recipient would know not to trust the object as authentic from S. And if the hash code were broken off and attached to a different file, it would not match there, either. So for efficiency we need encrypt only the hash value with S's private key, as shown in [Figure 2-36](#).

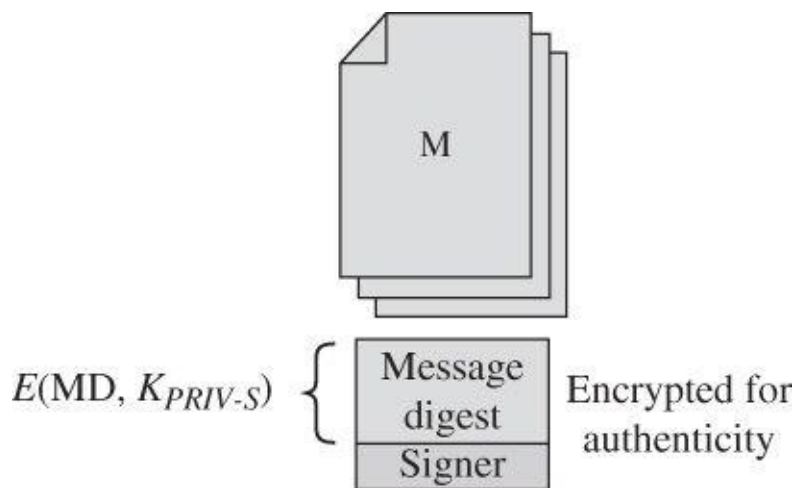


FIGURE 2-36 Asymmetric Encryption Covering the Hash Value

Second, the file, the data portion of the object, is exposed for anyone to read. If S wants confidentiality, that is, so that only one recipient can see the file contents, S can select a symmetric encryption key, encrypt the file, and store the key under user U's asymmetric public encryption key. This final addition is shown in [Figure 2-37](#).

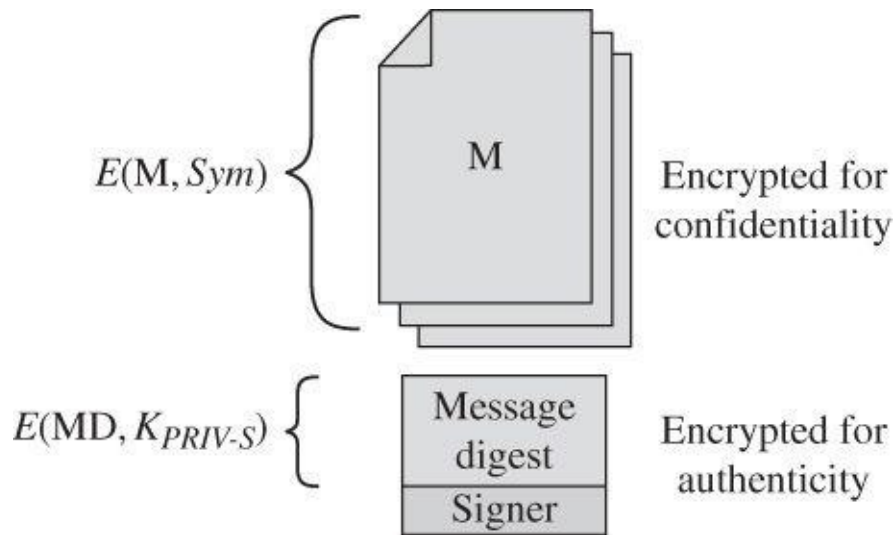


FIGURE 2-37 Digitally Signed Object Protected for Both Integrity and Confidentiality

In conclusion, a digital signature can indicate the authenticity of a file, especially a piece of code. When you attempt to install a piece of signed code, the operating system will inspect the certificate and file and notify you if the certificate and hash are not acceptable. Digital signatures, coupled with strong hash functions and symmetric encryption, are an effective way to ensure that a file is precisely what the originator stored for download.

This description of digital signatures concludes our section on tools from cryptography. We summarize the tools in [Table 2-15](#). In this section we have introduced important pieces we call upon later in this book.

Tool	Uses
Secret key (symmetric) encryption	Protecting confidentiality and integrity of data at rest or in transit
Public key (asymmetric) encryption	Exchanging (symmetric) encryption keys Signing data to show authenticity and proof of origin
Error detection codes	Detect changes in data
Hash codes and functions (forms of error detection codes)	Detect changes in data
Cryptographic hash functions	Detect changes in data, using a function that only the data owner can compute (so an outsider cannot change both data and the hash code result to conceal the fact of the change)
Error correction codes	Detect and repair errors in data
Digital signatures	Attest to the authenticity of data
Digital certificates	Allow parties to exchange cryptographic keys with confidence of the identities of both parties

TABLE 2-15 Tools Derived from Cryptography

Our point in this chapter is not to train a new corps of cryptographers or cryptologists; to do that would require far more material than this book can contain. Rather, we want you to know and understand the basic concepts of cryptography so in later chapters you can appreciate the difficulty, strengths, and weaknesses of, for example, securing a wireless

network signal or establishing a protected communication between a browser user and a website.

In the next chapter we put the three tools of this chapter to use in dealing with security problems in programs and programming.

2.4 Exercises

1. Describe each of the following four kinds of access control mechanisms in terms of (a) ease of determining authorized access during execution, (b) ease of adding access for a new subject, (c) ease of deleting access by a subject, and (d) ease of creating a new object to which all subjects by default have access.
 - per-subject access control list (that is, one list for each subject tells all the objects to which that subject has access)
 - per-object access control list (that is, one list for each object tells all the subjects who have access to that object)
 - access control matrix
 - capability
2. Suppose a per-subject access control list is used. Deleting an object in such a system is inconvenient because all changes must be made to the control lists of all subjects who did have access to the object. Suggest an alternative, less costly means of handling deletion.
3. File access control relates largely to the secrecy dimension of security. What is the relationship between an access control matrix and the integrity of the objects to which access is being controlled?
4. One feature of a capability-based protection system is the ability of one process to transfer a copy of a capability to another process. Describe a situation in which one process should be able to transfer a capability to another.
5. Suggest an efficient scheme for maintaining a per-user protection scheme. That is, the system maintains one directory per user, and that directory lists all the objects to which the user is allowed access. Your design should address the needs of a system with 1000 users, of whom no more than 20 are active at any time. Each user has an average of 200 permitted objects; there are 50,000 total objects in the system.
6. Calculate the timing of password-guessing attacks:
 - (a) If passwords are three uppercase alphabetic characters long, how much time would it take to determine a particular password, assuming that testing an individual password requires 5 seconds? How much time if testing requires 0.001 seconds?
 - (b) Argue for a particular amount of time as the starting point for “secure.” That is, suppose an attacker plans to use a brute-force attack to determine a password. For what value of x (the total amount of time to try as many passwords as necessary) would the attacker find this attack prohibitively long?
 - (c) If the cutoff between “insecure” and “secure” were x amount of time,

how long would a secure password have to be? State and justify your assumptions regarding the character set from which the password is selected and the amount of time required to test a single password.

7. Design a protocol by which two mutually suspicious parties can authenticate each other. Your protocol should be usable the first time these parties try to authenticate each other.
8. List three reasons people might be reluctant to use biometrics for authentication. Can you think of ways to counter those objections?
9. False positive and false negative rates can be adjusted, and they are often complementary: Lowering one raises the other. List two situations in which false negatives are significantly more serious than false positives.
10. In a typical office, biometric authentication might be used to control access to employees and registered visitors only. We know the system will have some false negatives, some employees falsely denied access, so we need a human override, someone who can examine the employee and allow access in spite of the failed authentication. Thus, we need a human guard at the door to handle problems, as well as the authentication device; without biometrics we would have had just the guard. Consequently, we have the same number of personnel with or without biometrics, plus we have the added cost to acquire and maintain the biometrics system. Explain the security advantage in this situation that justifies the extra expense.
11. Outline the design of an authentication scheme that “learns.” The authentication scheme would start with certain primitive information about a user, such as name and password. As the use of the computing system continued, the authentication system would gather such information as commonly used programming languages; dates, times, and lengths of computing sessions; and use of distinctive resources. The authentication challenges would become more individualized as the system learned more information about the user.
 - Your design should include a list of many pieces of information about a user that the system could collect. It is permissible for the system to ask an authenticated user for certain additional information, such as a favorite book, to use in subsequent challenges.
 - Your design should also consider the problem of presenting and validating these challenges: Does the would-be user answer a true-false or a multiple-choice question? Does the system interpret natural language prose?
12. How are passwords stored on your personal computer?
13. Describe a situation in which a weak but easy-to-use password may be adequate.
14. List three authentication questions (but not the answers) your credit card company could ask to authenticate you over the phone. Your questions should be ones to which an imposter could not readily obtain the answers. How difficult would it be for you to provide the correct answer (for example, you would have to look something up or you would have to do a quick arithmetical calculation)?
15. If you forget your password for a website and you click [Forgot my password], sometimes the company sends you a new password by email but sometimes it sends

you your old password by email. Compare these two cases in terms of vulnerability of the website owner.

16. Defeating authentication follows the method–opportunity–motive paradigm described in [Chapter 1](#). Discuss how these three factors apply to an attack on authentication.

17. Suggest a source of some very long unpredictable numbers. Your source must be something that both the sender and receiver can readily access but that is not obvious to outsiders and not transmitted directly from sender to receiver.

18. What are the risks of having the United States government select a cryptosystem for widespread commercial use (both inside and outside the United States). How could users from outside the United States overcome some or all of these risks?

19. If the useful life of DES was about 20 years (1977–1999), how long do you predict the useful life of AES will be? Justify your answer.

20. Humans are said to be the weakest link in any security system. Give an example for each of the following:

(a) a situation in which human failure could lead to a compromise of encrypted data

(b) a situation in which human failure could lead to a compromise of identification and authentication

(c) a situation in which human failure could lead to a compromise of access control

21. Why do cryptologists recommend changing the encryption key from time to time? Is it the same reason security experts recommend changing a password from time to time? How can one determine how frequently to change keys or passwords?

22. Explain why hash collisions occur. That is, why must there always be two different plaintexts that have the same hash value?

23. What property of a hash function means that collisions are not a security problem? That is, why can an attacker not capitalize on collisions and change the underlying plaintext to another form whose value collides with the hash value of the original plaintext?

24. Does a PKI perform encryption? Explain your answer.

25. Does a PKI use symmetric or asymmetric encryption? Explain your answer.

26. Should a PKI be supported on a firewall (meaning that the certificates would be stored on the firewall and the firewall would distribute certificates on demand)? Explain your answer.

27. Why does a PKI need a means to cancel or invalidate certificates? Why is it not sufficient for the PKI to stop distributing a certificate after it becomes invalid?

28. Some people think the certificate authority for a PKI should be the government, but others think certificate authorities should be private entities, such as banks, corporations, or schools. What are the advantages and disadvantages of each approach?

29. If you live in country A and receive a certificate signed by a government

certificate authority in country B, what conditions would cause you to trust that signature as authentic?

30. A certificate contains an identity, a public key, and signatures attesting that the public key belongs to the identity. Other fields that may be present include the organization (for example, university, company, or government) to which that identity belongs and perhaps suborganizations (college, department, program, branch, office). What security purpose do these other fields serve, if any? Explain your answer.